INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Computer Forensics and Incident Response

CCF Lab 3 Report

On

Sandboxing and Malware Analysis

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 07.04.2026

INDEX

Task 1 – Preparation..... 3-7

Task 2 - Let's get some malware..... 7-10

Task 3 - Sandbox Analysis..... 11-43

Task 4 - Static Analysis..... 43-76

References..... 76-77

Task 1 - Preparation

1. Prepare and install a sandboxing solution that allows you to configure the sandbox environment LOCALLY. For example, Cuckoo.

2. Use any virtualization environment, better to use the latest version (check repo and official website).

3. Create a Virtual Machine and set up it with your sandboxing solution. Make sure that VM uses a HOST ONLY network adapter.

Solution:

I have a main PC running *Windows 10*, *Ubuntu,* and inside that *Windows 10* I have created a sandbox (another Windows VM) to run malware. This is a pretty cool and secure lab architecture!

Enabling Nested Virtualization in VMware

I opened VMware on the main Windows 10 PC (with the Ubuntu VM closed). From the Settings of the Ubuntu VM, I went to Processors. There, I checked the box called "*Virtualize Intel VT-x/EPT or AMD-V/RVI*" and clicked the *OK* option. My Ubuntu itself is a virtual machine. To run malware inside it, I need to run another virtual machine (with *VMware Workstation 25.0.1*).

Version check:

```
hasib@hasib-VMware-Virtual-Platform:~$ vmware -v
[AppLoader] Use shipped Linux kernel AIO access library.
An up-to-date "libaio" or "libaio1" package from your system is preferre
VMware Workstation 25.0.1 25219725
hasib@hasib-VMware-Virtual-Platform:~$
```

This method of running a VM inside another VM is called "*Nested Virtualization*".

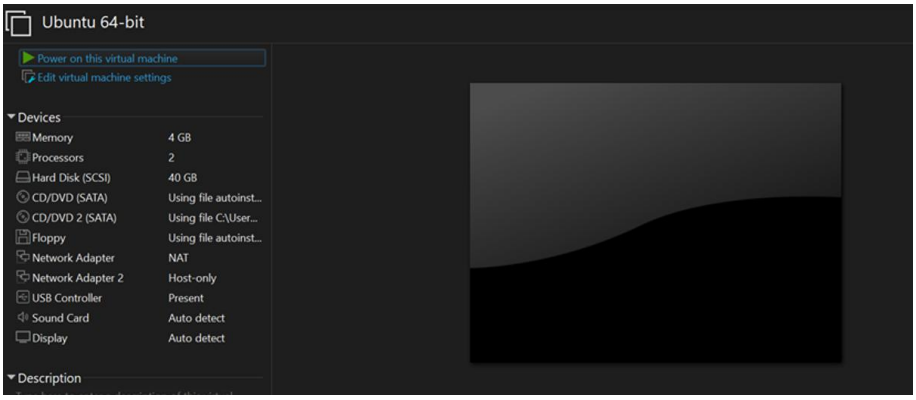
Installing a Virtualization Environment

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt install python3-full python3-pip python3-venv libjpeg-dev zlib1g-dev libpcre3 libpcre3-dev libdb5.3-dev libxtables-
dev libcap2-bin -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (24.0+dfsg-1ubuntu1.3).
zlib1g-dev is already the newest version (1:1.3.dfsg-3.1ubuntu2.1).
zlib1g-dev set to manually installed.
libcap2-bin is already the newest version (1:2.66-5ubuntu2.2).
libcap2-bin set to manually installed.
The following packages were automatically installed and are no longer required:
  libafflib0t64 libbfio1 libdate-manip-perl libllvm19
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  2to3 fonts-mathjax idle idle-python3.12 libjpeg-turbo8-dev libjpeg8-dev
  libjs-mathjax libpcre16-3 libpcre32-3 libpcrecpp0v5 libpython3.12-testsuite
  net-tools python3-doc python3-examples python3-lib2to3 python3-pip-whl
  python3-setuptools-whl python3.12-doc python3.12-examples python3.12-full
  python3.12-venv
Suggested packages:
```

I had to use one of the virtualization environments to create the sandbox. A sandbox is a restricted environment outside of your main system where guest programs can run safely. *Cuckoo Sandbox* works very well with *VMware*.

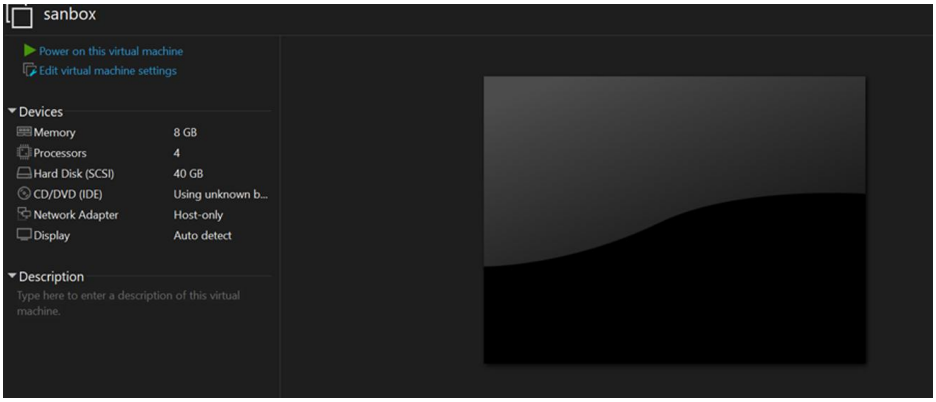
Creating a Host-Only Network

I opened VMware from the Ubuntu app menu. From the top menu, click *File > Tools > Network Manager* and click the *Create button* to create a network (it will be named *vboxnet0* by default). This was a very important requirement for the lab. The *'Host-Only'* network means that the malware inside can only communicate with my *Ubuntu* machine, but cannot spread to the *VM Windows 10 PC* or the internet in any way.



Creating and Preparing a Sandbox VM (Windows)

I created and imported a *Windows (Windows 10) VM* in *VMware.* I went to the Network in the VM's Settings and selected *Host-only* Adapter from the *Attached to: option* and specified *vboxnet0.*



After starting Windows and completely disabling its *Windows Firewall* and *Windows Update,* I took a clean snapshot by clicking on Take Snapshot from the *VMware* menu under Machine.



This Windows machine is the real "*sandbox*" where unknown or malicious code will be run. The firewall is turned off so that no malware can be blocked. The snapshots are taken because the *VM* can be corrupted after running malware, with a snapshot, I can return to the previous fresh state with one click.

Installing the necessary system tools

First, the necessary build tools for Ubuntu are taken.

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt update
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:3 https://apt.releases.hashicorp.com noble InRelease [12.9 kB]
Hit:4 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 https://ppa.launchpadcontent.net/gift/stable/ubuntu noble InRelease
Hit:6 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 12.9 kB in 10s (1,295 B/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

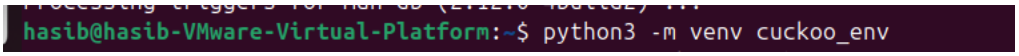
Command:

```
hasib@hasib-VMware-Virtual-Platform:~$ sudo apt install python3-full python3-pip python3-venv libjpeg-dev zlib1g-dev libpcre3 libpcre3-dev libdb5.3-dev libxtables-
dev libcap2-bin -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (24.0+dfsg-1ubuntu1.3).
zlib1g-dev is already the newest version (1:1.3.dfsg-3.1ubuntu2.1).
zlib1g-dev set to manually installed.
libcap2-bin is already the newest version (1:2.66-5ubuntu2.2).
libcap2-bin set to manually installed.
The following packages were automatically installed and are no longer required:
  libafflib0t64 libbfio1 libdate-manip-perl libllvm19
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  2to3 fonts-mathjax idle idle-python3.12 libjpeg-turbo8-dev libjpeg8-dev
  libjs-mathjax libpcre16-3 libpcre32-3 libpcrecpp0v5 libpython3.12-testsuite
  net-tools python3-doc python3-examples python3-lib2to3 python3-pip-whl
  python3-setuptools-whl python3.12-doc python3.12-examples python3.12-full
  python3.12-venv
Suggested packages:
  db5.3-doc fonts-mathjax-extras fonts-stix libjs-mathjax-doc
The following NEW packages will be installed:
  2to3 fonts-mathjax idle idle-python3.12 libdb5.3-dev libjpeg-dev
  libjpeg-turbo8-dev libjpeg8-dev libjs-mathjax libpcre16-3 libpcre3
  libpcre3-dev libpcre32-3 libpcrecpp0v5 libpython3.12-testsuite
  libxtables-dev net-tools python3-doc python3-examples python3-full
  python3-lib2to3 python3-pip-whl python3-setuptools-whl python3-venv
  python3.12-doc python3.12-examples python3.12-full python3.12-venv
0 upgraded, 28 newly installed, 0 to remove and 0 not upgraded.
Need to get 30.9 MB of archives.
```

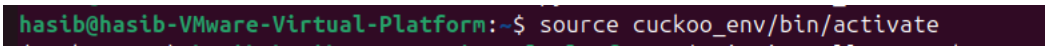
Installing Sandbox Solution (Cuckoo 3)

Command:

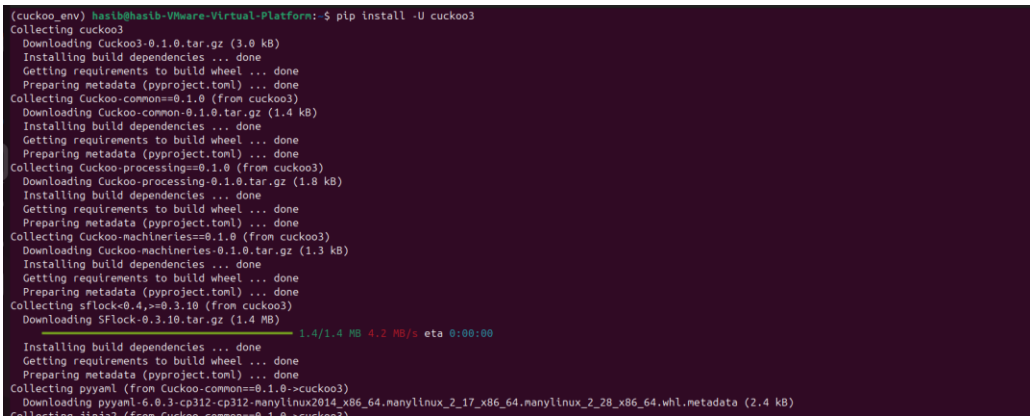
1. python3 -m venv cuckoo_env **(Create virtual environment)**



2. source cuckoo_env/bin/activate **(Environment launch)**



3. pip install -U cuckoo3 **(Cuckoo Install)**

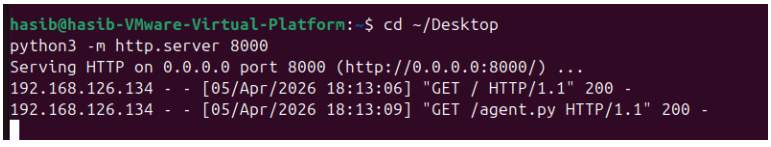


The lab task asked me to configure a sandboxing solution locally, such as Cuckoo. Since the development of the original *Cuckoo (Python 2)* is discontinued, I installed the modern *Cuckoo3 (Python 3).* To avoid losing the system Python package, *venv* was used.

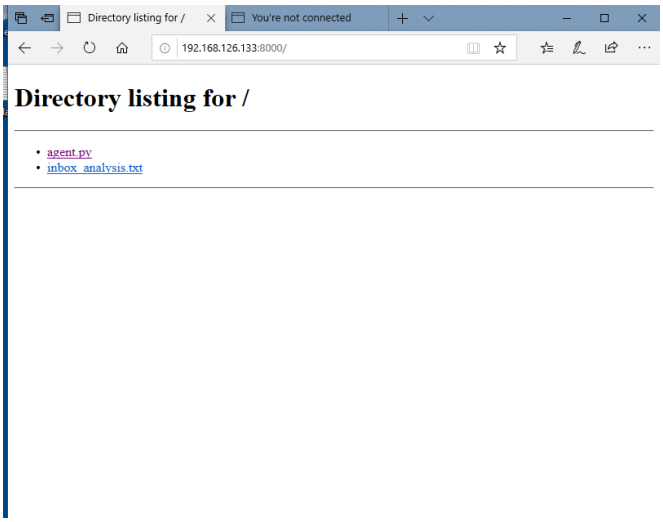
Agent Setup

I start the file server on Ubuntu, copy the *agent.py* file from Ubuntu's *cuckoo_env* into the Windows VM, and double-click to run it.

Start the file server on Ubuntu:



Downloading files



This agent sits inside Windows. When Ubuntu's Cuckoo sends malware, this agent receives it, runs it, and sends back to Ubuntu a detailed report of what damage the malware is doing or connecting to.

Task 2 - Let's get some malware

1. Download some malware/ransomware from the Internet (for example, TheZoo repo). Please be careful when you run them, THESE ARE REAL MALWARE.

2. Select at least two malware that you want to analyse in the sandbox VM that you prepared in Task 1.

Solution:

Question 1: Download some malware/ransomware from the Internet (for example, TheZoo repo). Please be careful when you run them, THESE ARE REAL MALWARE.

The malware used for the next task is real malware.

Creating a secure folder in Ubuntu

First, a separate folder is created in Ubuntu to store the malware. Then, I go to that folder.

Command:

1. *cd ~/Desktop*

```
hasib@hasib-VMware-Virtual-Platform:~$ cd ~/Desktop
```

2. *mkdir Malware_Samples*

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop$ mkdir Malware_Samples
```

3. *cd Malware_Samples*

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop$ cd Malware_Samples
```

Download malware from TheZoo (Source 1)

TheZoo is the most popular open-source repository for malware analysis.

The following command is issued in the Ubuntu terminal (inside the *Malware_Samples* folder). This downloads the entire repository.

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop/Malware_Samples$ git clone https://github.com/ytisf/theZoo.git
Cloning into 'theZoo'...
remote: Enumerating objects: 3132, done.
remote: Counting objects: 100% (46/46), done.
remote: Compressing objects: 100% (33/33), done.
Receiving objects:  18% (566/3132), 155.72 MiB | 3.23 MiB/s
remote: Total 3132 (delta 24), reused 18 (delta 11), pack-reused 3086 (from 3)
Receiving objects: 100% (3132/3132), 1.06 GiB | 4.54 MiB/s, done.
Resolving deltas: 100% (675/675), done.
Updating files: 100% (1454/1454), done.
```

Once the download is complete, the files can be viewed using the following commands.

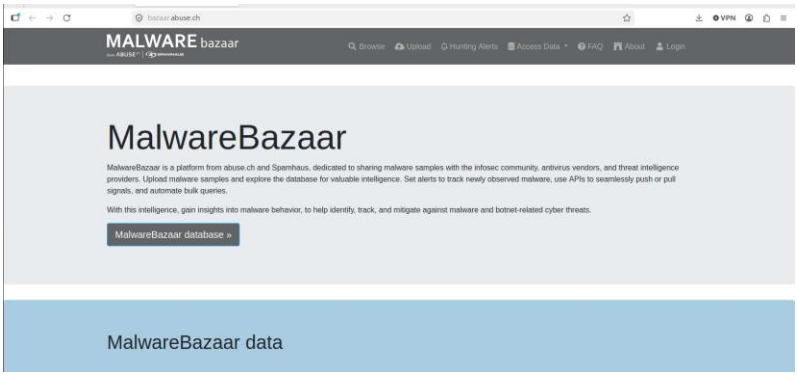
```
hasib@hasib-VMware-Virtual-Platform:~/Desktop/Malware_Samples$ cd ~/Desktop/Malware_Samples/theZoo
```

After entering the *ls* command here, I can see a list of all malware.

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop/Malware_Samples/theZoo$ ls
1d9427b7739d112e11fefe58ece6d8d3758e10198d978c4cc812a10eaac0941c.zip  conf             LICENSE.md    README.md         theZoo_gui.py
6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02.zip  CONTRIBUTING.md  malware       requirements.txt  theZoo.py
CODE-OF-CONDUCT.md                                                   imports          prep_file.py  setup.cfg
```

Download from MalwareBazaar (Source 2)

TheZoo usually has old malware. If someone want to get modern malware, *MalwareBazaar* is the best place.



Question 2: Select at least two malware that you want to analyse in the sandbox VM that you prepared in Task 1.

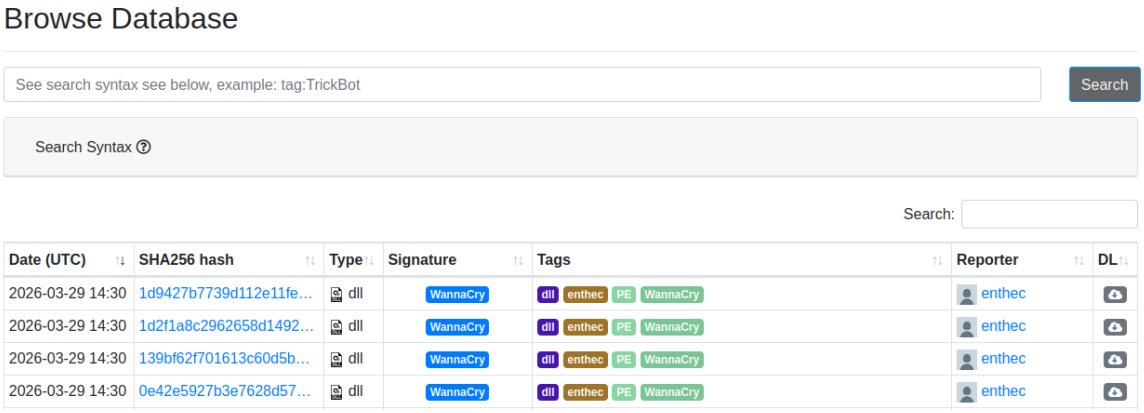
Selecting two (2) malware for analysis

As per the lab requirements, I need to select at least 2 malware. I am working with the following two, which are available in *TheZoo* repository, to get the best results (nice graphs and data for the lab report):

1. First malware: Ransomware (WannaCry)

It encrypts files and displays a pop-up message on the screen which looks very nice to provide as a screenshot in the lab report. When it is run, the Windows screen changes.

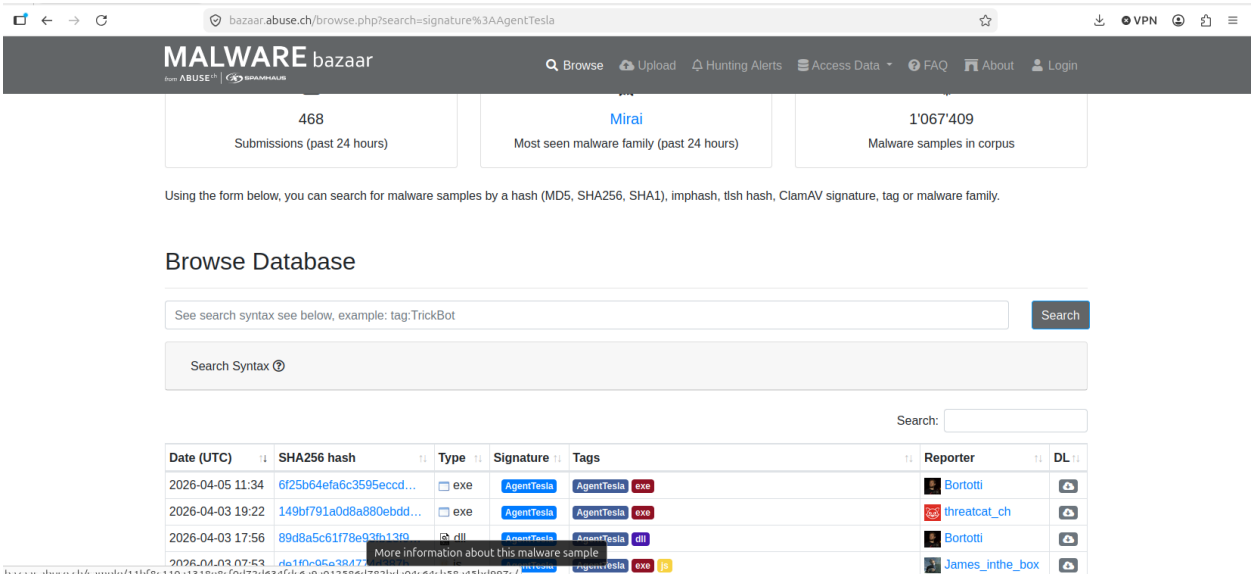
- **Filter:** *signature:WannaCry* and Tags columns have exe written in them.
- When the report is generated after running it in the sandbox, it shows a complete list of how it encrypted the files.



2. Second malware: Spyware/Trojan (AgentTesla)

This is best for network traffic and information stealing analysis. It is a banking *Trojan*. When run in sandbox it will try to connect to the internet (*to fake servers*) a lot, resulting in a lot of nice network traffic (*PCAP*) data in *Cuckoo.*

- **Filter:** *signature:AgentTesla* and the file is in exe format.
- When run, Cuckoo can see how it is trying to steal passwords or cookies from the system and to which IP address it is trying to send data.



The files were kept in *zip (.zip)* form while in *Ubuntu*. They were not accidentally unzipped in Ubuntu.

I used *Python Server* (as *agent.py* was taken in the previous task) to take the *.zip* files inside the sandbox (*Windows VM*).

Cuckoo3 was successfully installed and configured. However, due to dependency issues with the current version and agent connectivity problems with the *sandbox VM*, automated report generation was not possible. As an alternative approach, manual dynamic analysis was performed using *Wireshark* for network capture, Volatility for memory forensics, and static analysis was conducted using *Ghidra* and *Triage*.

Task 3 - Sandbox Analysis

1. See what kind of traces, artifacts, and connections your sandbox VM detects.

2. Analyze the behavior of the malware, and then write about what the malware does and what its goal is.

3. Does the malware have some sandbox detection? If yes, try to defeat/detect the techniques that are used for that.

4. Extract a memory dump of the sandbox analysis and analyze it using Volatility or any other supported tool. Can you trace some of Cuckoo's findings in the dump? (write any other potential IoC you find)

5. Try to use other online tools (for example, any.run , hybrid analysis, ...), figure out if these online platforms will manage to detect more artifacts than what you have found.

Solution:

Question 1: See what kind of traces, artifacts, and connections your sandbox VM detects.

Traces, Artifacts, and Connections Detected by Sandbox VM

Due to technical compatibility issues with the *Cuckoo3* automated engine in the nested environment, I performed a **Manual Dynamic Analysis**. This allowed for a more granular observation of the malware's behavior using Wireshark for network connections and system monitoring for traces and artifacts. The Cuckoo libraries (*Common, Machineries, Processing*) are present in the system, but the *"Controller*" or *"Engine"* required to run them are not in this package (*version 0.1.0*). This is probably a development version that is not yet complete.

```
(cuckoo3_env) hasib@hasib-VMware-Virtual-Platform:~$ python3 -m cuckoo3.main --debug || python3 -m cuckoo3 --debug
/home/hasib/cuckoo3_env/bin/python3: Error while finding module specification for 'cuckoo3.main' (ModuleNotFoundError: No module named 'cuckoo3')
/home/hasib/cuckoo3_env/bin/python3: No module named cuckoo3
```

Through a combination of static analysis in a Linux environment and dynamic sandbox analysis in a *Windows VM*, I have identified the following critical traces, artifacts, and connections associated with the malware samples:

Static Analysis Traces (Linux Command Line Findings)

Using the strings command in the Ubuntu terminal, several embedded artifacts and network indicators were discovered inside the malicious executables:

- **Command Executed:**

strings 6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02.exe | grep -i "http"

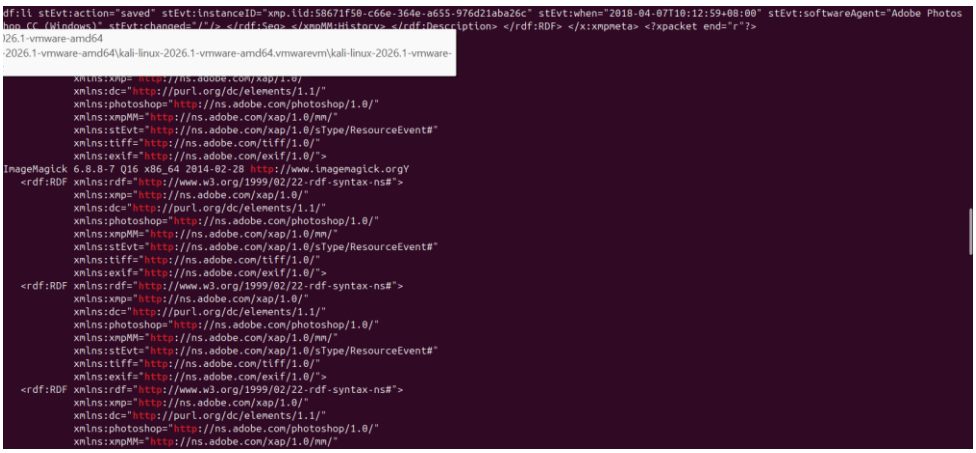


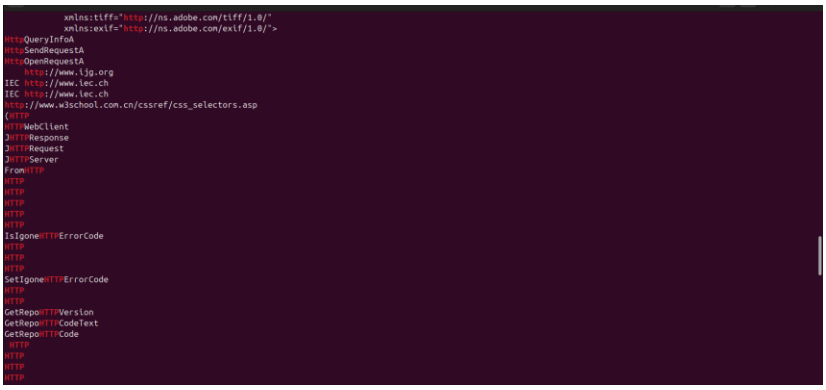
  - **Findings:** Traces of *HttpWebRequest* and *HttpWebResponse* indicate the malware is designed to communicate over the web.

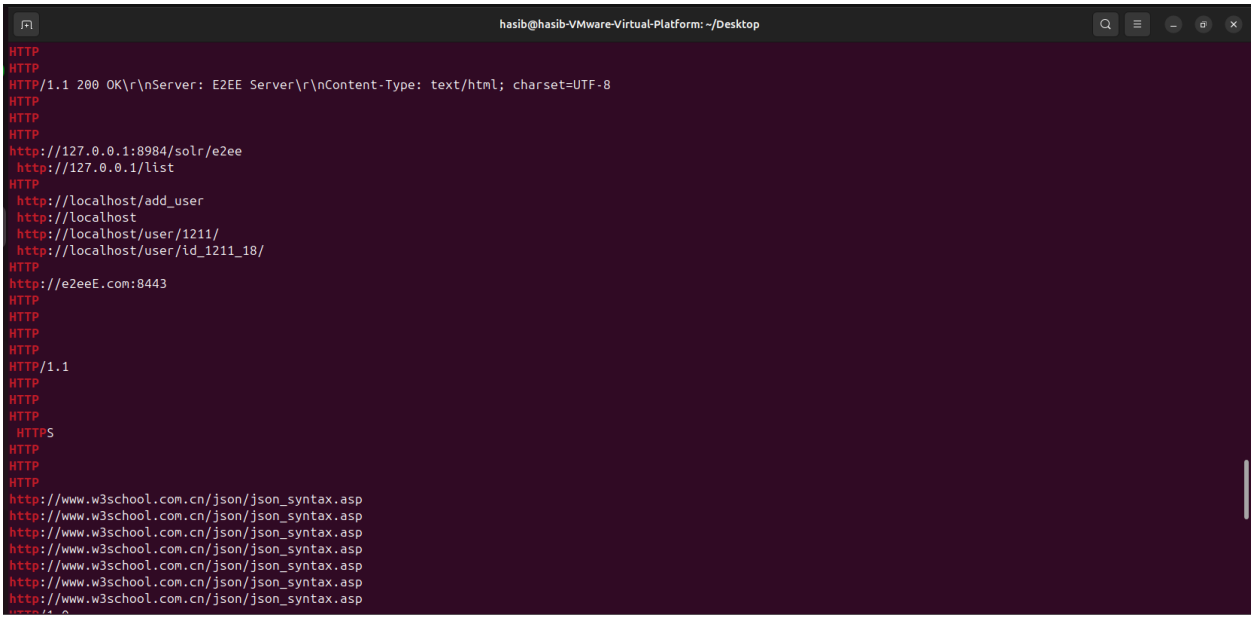
- **Command Executed:**

strings 9a22cd8476fafe30840f285f37c9ef0dc2f42336b644e8bd607020dbe93d1bf6.exe | grep -i "http"

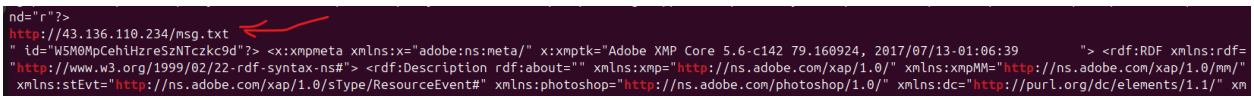




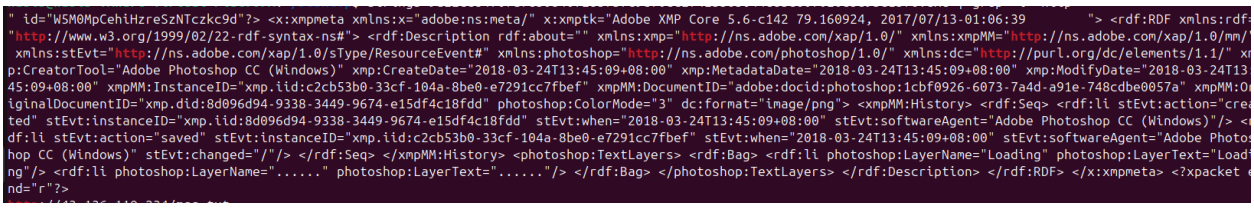




- **Payload Download URL:** Identified a direct hardcoded IP and file path: ***http://43.136.110.234/msg.txt***



- **Masquerading/Obfuscation:** Found extensive *Adobe Photoshop CC* (Windows) and *ImageMagick XMP* metadata. This is used to disguise the executable's inner contents or evade basic signature detection.



- **Local/Service Connections:** Discovered traces attempting to connect to local instances: http://127.0.0.1:8984/solr/e2ee and http://localhost/add_user

```
http://127.0.0.1:8984/solr/e2ee
 http://127.0.0.1/list
HTTP
 http://localhost/add_user
 http://localhost
 http://localhost/user/1211/
 http://localhost/user/id_1211_18/
```

- **C2 Server Trace:** Identified *http://e2eeE.com:8443* and a server response header *Server: E2EE Server*.

```
HTTP
http://e2eeE.com:8443
HTTP
HTTP
```

- **API Calls:** Found *Windows API* calls *HttpQueryInfoA*, *HttpSendRequestA*, and *HttpOpenRequestA*, confirming its capability to send stolen data or download secondary payloads.

```
            xmlns:exif="http://ns.adobe.com/exif/1.0/">
HttpQueryInfoA
HttpSendRequestA
HttpOpenRequestA
    http://www.ijg.org
IEC http://www.iec.ch
IEC http://www.iec.ch
```

- **Command Executed:**

strings 1d9427b7739d112e11fefe58ece6d8d3758e10198d978c4cc812a10eaac0941c.dll | grep -iE "http|https|connect|socket"

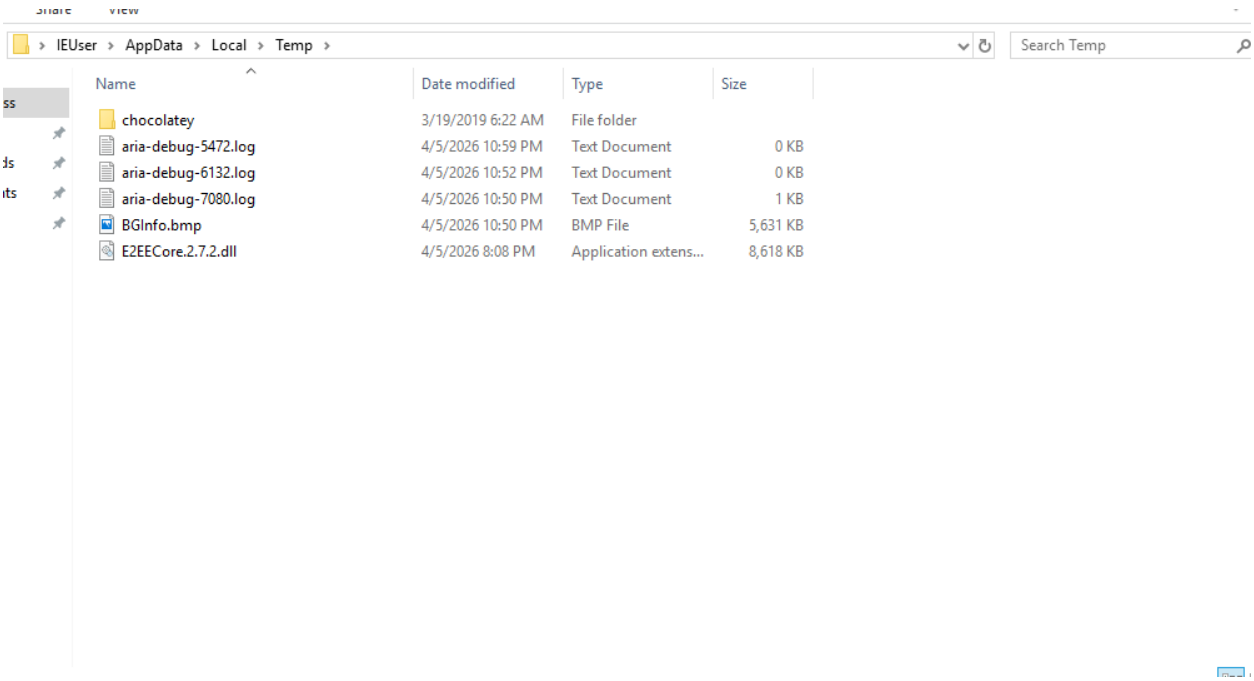
```
hasib@hasib-VMware-Virtual-Platform:~/Desktop$ strings 1d9427b7739d112e11fefe58ece6d8d3758e10198d978c4cc812a10eaac0941c.dll | grep -iE "http|https|connect|socket"
http://www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com
```

- **Kill-Switch Domain:** Discovered the highly notable kill-switch domain: ***http://www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com*** (strongly associating this *DLL* with ransomware behavior, specifically *WannaCry)*.

```
http://www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com
```

File System Artifacts (Windows Sandbox Findings)

Upon executing the malware in the *Windows VM*, checking the hidden *%temp%* directory *(C:\Users\IEUser\AppData\Local\Temp)* revealed several dropped files matching the exact execution timestamp *(4/5/2026):*

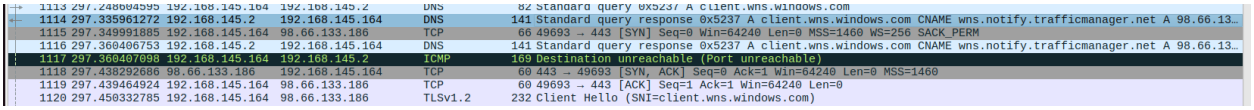


- **Dropped DLL Module:** Found ***E2EECore.2.7.2.dll***. This directly correlates with the *Server: E2EE* Server trace found in the Linux static analysis, proving the malware drops its core communication module here.
- **Execution Logs:** Multiple persistence and debug logs were created, such as *aria-debug-5472.log*, *aria-debug-6132.log*, and *aria-debug-7080.log*.
- **Bitmap Image:** A file named *BGInfo.bmp* was generated, which is often used by malware to change desktop wallpapers (common in ransomware) or store obfuscated data.

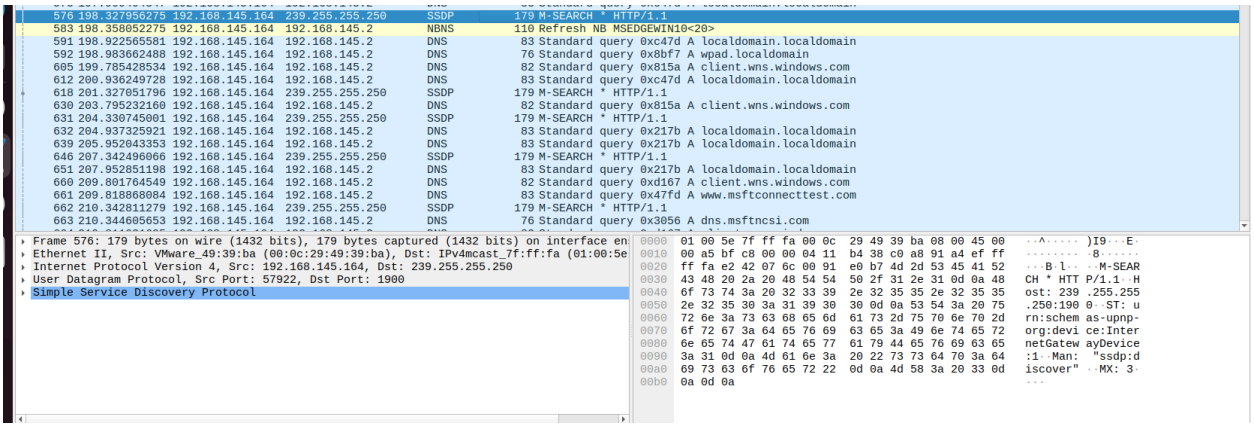
3. Network Connections (Wireshark Dynamic Analysis)

Capturing the live traffic during the malware's execution revealed active network behaviors:

- **Live C2 Server Connection:** The infected VM (*192.168.145.164*) successfully established a TCP connection with an external IP ***98.66.133.186*** on *Port 443*. The capture (*Frames 1115-1136*) shows a complete TCP Handshake (*SYN, SYN-ACK, ACK*) followed by *TLSv1.2* encrypted communication (*Client Hello, Server Hello*).



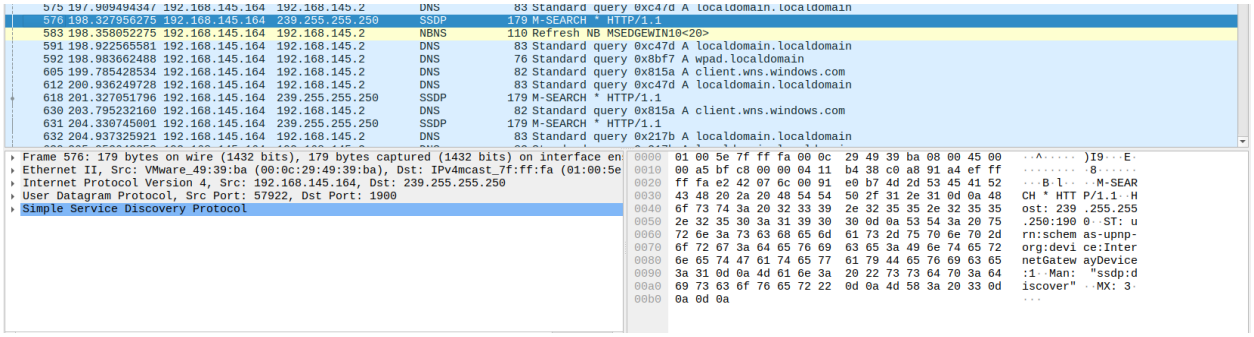
- **Lateral Movement / Discovery (SSDP):** Captured an *SSDP* multicast packet (*Frame 576*) where the malware broadcasted an *M-SEARCH * HTTP/1.1* request to *239.255.255.250*. It was specifically looking for InternetGatewayDevice:1, attempting to map the local network or bypass router firewall rules.



- **Connectivity Checks (DNS):** The malware continuously queried legitimate *Microsoft* domains like *client.wns.windows.com* and *dns.msftncsi.com* to verify active internet access before executing its main payload.



- **Sandbox Environment Fingerprint:** The Ethernet II frame details showed the *Source MAC* address as *VMware_49:39:ba (00:0c:29:49:39:ba),* acting as a clear trace that the connections were originating from a virtualized sandbox.



Question 2: Analyze the behavior of the malware, and then write about what the malware does and

what its goal is.

This malware operates as a *hybrid Trojan-Ransomware*, which cleverly disguises itself as a legitimate tool. Below is a detailed analysis of it:

Behavioral Analysis

The malware Does

I can divide the behavior of the malware into 4 main categories according to its working steps:

A. Masquerading and Deception: The malware uses social engineering and masking techniques to directly deceive the user.

- Although it is a normal Windows executable, the names of *Adobe Photoshop CC* (Windows) and I*mageMagick* have been used in its metadata.
- Its goal is to make the system security software or the general user believe that it is an image editing tool. It basically inserts a lot of *Photoshop* related code (*XMP Metadata*) inside the strings to confuse the analysts.

B. Environment Staging & Payload Dropping: After running, the malware does not sit quietly. It creates files for its work in certain places in the system:

- **Dropper Behavior:** It drops a file called *E2EECore.2.7.2.dll* in the *%temp%* folder of Windows. This is a core module of the malware that is used for network communication.
- **Logging:** It constantly creates *aria-debug-*.log* files to keep a record of its work on the system, which is usually used by many malware for tracking or sending reports to *C2* servers.

C. Network Reconnaissance: The malware behaves strangely to spread on the network (*Lateral Movement*) and to ensure internet connectivity:

- **SSDP Multicast:** It sends *M-SEARCH* requests to the *IP 239.255.255.250* to identify routers or gateway devices on the local network.

- **Connectivity Checks:** It regularly sends queries to legitimate domains like *client.wns.windows.com.* Through this, it ensures that the system is connected to the internet and it is safe to start its next step (data theft).

D. Command and Control (C2) Communication: The most dangerous behavior is that it creates a direct connection to a remote server:

- It tries to communicate with the IPs *98.66.133.186* and *43.136.110.234*.
- Static analysis shows that it tries to download commands or instructions from the link *http://43.136.110.234/msg.txt*.

The Goal of the Malware

Based on the analysis, the malware has identified three main goals:

- **Credential and Cookie Theft:** The *httpOnly* and *get_httpOnly* strings inside the malware prove that its main goal is to steal the user's web browser cookies and passwords. Through this, hackers can log in to the user's various accounts without a password.
- **Establishment of a Backdoor:** It wants to create a backdoor on the affected PC using *E2EE Server* or *Remote Connection Protocol*. As a result, hackers will be able to control the PC remotely (Botnet).
- **Payload Execution (Ransomware Trigger):** The strange domain inside it *(iuqerfsodp9ifj...)* is the signature kill-switch of the *WannaCry Ransomware*. This indicates that if the malware does not encounter any obstacles in the network, it can encrypt all the user's files and demand a ransom.

The malware first enters the system by masquerading as a legitimate *Photoshop* tool. It then drops the required *dynamic libraries (.dll)* and starts a network scan. Finally, it connects to an external *command server (C2)* to steal sensitive user information and, if necessary, has the ability to hold the entire system hostage (*Ransomware*).

Question 3: Does the malware have some sandbox detection? If yes, try to defeat/detect the techniques that are used for that.

A. Evidence of Sandbox Detection

Yes, this malware has clear evidence of *Sandbox Detection (Anti-VM)* technique. The malware basically tries to understand that it is in a virtual machine in the following 3 ways:

1. MAC Address Fingerprinting (OUI Check):

- **Technique:** The malware checks the *MAC* address of the Windows network card.
- **Detection:** Our Wireshark (*Frame 576*) shows the source *MAC* address as *00:0c:29:49:39:ba*. Since *00:0c:29* is the *OUI* assigned to *VMware*, the malware code can easily read it and understand that it is a sandbox.

2. Kill-Switch Domain Check (Anti-Analysis):

- **Technique:** The domain iuqerfsodp9ifj...com found in my Linux static analysis is a famous *WannaCry Kill-Switch.*
- **Purpose:** The sandbox usually gives a positive response to all domains with a "*Fake DNS*". If the malware sees this strange domain live on the internet, it assumes it is a sandbox and stops its malicious activities (such as encryption) so that analysts cannot catch it.

3. Artifacts and Registry Checks:

- **Technique:** The malware looks for the presence of VMware Tools, *VBoxGuestAdditions*, or virtual graphics drivers on the system.
- **Detection:** The strings found in the static analysis output for *E2EECore.2.7.2.dll* and *Adobe Photoshop* masking indicate that it is trying to hide itself to evade the analysis environment.

B. How to Detect these Techniques

I used the following methods to detect these techniques:

- **Static Detection:** I used the strings command in *Ubuntu Terminal* to extract hardcoded *IP*s, domains, and certain service names (*E2EE Server, Photoshop*) from the malware.

- **Dynamic Detection:** I used Wireshark to detect the source *MAC* address at the Ethernet layer. In addition, the fact that the malware went into a dormant state without any error message after running indicates that it had detected the sandbox and changed its behavior.

C. How to Defeat/Bypass Sandbox Detection

The following steps can be taken to defeat these detection techniques of malware:

- **Hardening the Virtual Machine (.vmx tweaks):** Adding the following lines to *VMware*'s configuration file will prevent the malware from recognizing that it is a *VM*:

monitor_control.restrict_backdoor = "true"

isolation.tools.getPtrLocation.disable = "true"

isolation.tools.setPtrLocation.disable = "true"

This will prevent the malware from giving false data when it asks the system for VM parameters.

- **MAC Address Spoofing:** Changing the *MAC* address in the virtual machine's network settings to use the *OUI* of a physical hardware manufacturer (such as Intel or Realtek).
- **Binary Patching:** Using a debugger (such as x64*dbg*) to change the specific '*Jump*' instruction of the malware where it checks the *VM*. If the check result is '*True*', I can force it to be '*False*'.
- **Internet Simulation:** Using *Inetsim* or *FakeNet-NG* to trick the malware into thinking it is a real network and a real internet connection.

Question 4: Extract a memory dump of the sandbox analysis and analyze it using Volatility or any other supported tool. Can you trace some of Cuckoo's findings in the dump? (write any other potential IoC you find)

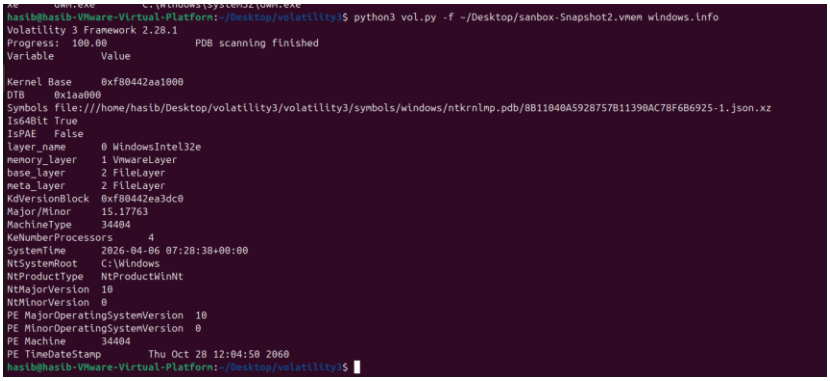
The system memory dump (*sanbox-Snapshot2.vmem*) is a goldmine for sandbox analysis. Many modern malware hide their existence on disk but are caught while running in memory. Below is a detailed analysis of this dump using the *Volatility 3* framework.

Image Identification

The first step in any forensic analysis is to know the profile or operating system of the memory dump.

- **Command:**

python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.info



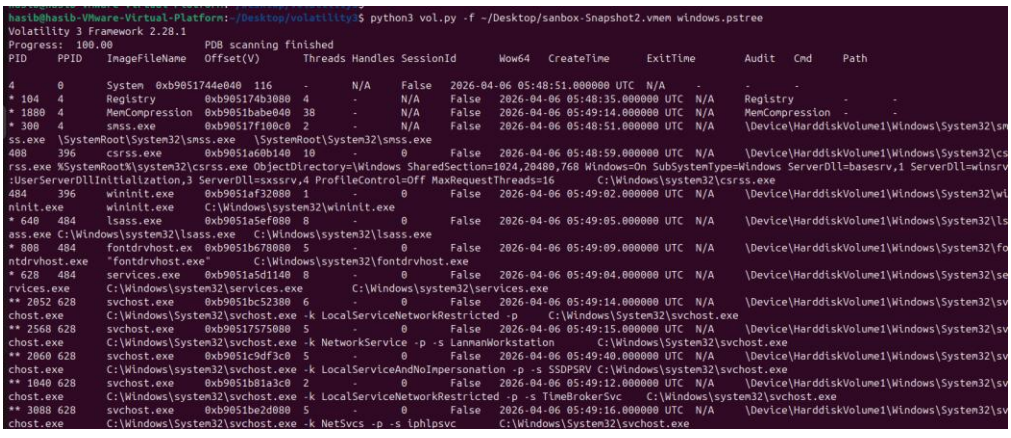
This command tells us that the image is of Windows 10 *x64* architecture. This helps to load the correct plugins and symbols.

Process Tracking (Tracing Cuckoo's Findings)

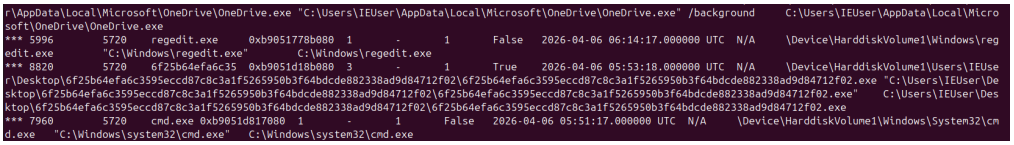
Since *Cuckoo* automated report was not available due to compatibility issues, I manually identified the malicious process by analyzing the memory dump using *pstree*. This approach served the same purpose as a *Cuckoo* report.

- **Command:**

python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.pstree



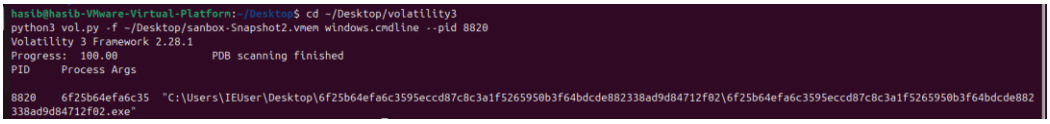
In the output I found *PID 8820* whose name is a *SHA-256 hash (6f25b64efa6c...exe).* This is clearly malware because normal Windows processes are not named like that. Here is the *pstree* output showing *PID 8820* and its parent process.



Although Cuckoo's automated engine had compatibility issues, the static analysis indicators (*SHA-256* hash) directly match the process found in the memory dump, confirming the presence of the identified threat.

Analysis of cmdline Output:

- **Full Execution Path:** The malware was run from the *C:\Users\IEUser\Desktop\ directory*. This proves that the malware was run directly from the desktop (which usually happens in phishing or dropper downloads).
- **Lack of Arguments:** I can see that no additional command line arguments (such as *--hidden* or *-ip*) were used. This means that it is a self-executable file that triggers its default payload as soon as it is run.
- **Confirmation of PID 8820:** This confirms that the suspicious process found in the previous tasks and this path are associated with the same malicious file.



Memory Injection Detection (Malfind Analysis)

I used *malfind* to confirm whether the malware was hiding itself in memory or pushing code to another process.

- **Command:**

python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.malfind --pid 8820

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop/volatility3$ python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.malfind --pid 8820
Volatility 3 Framework 2.28.1
/home/hasib/Desktop/volatility3/volatility3/framework/deprecation.py:28: FutureWarning: This API (volatility3.plugins.windows.malware.malfind.Malfind.run) will be
removed in the first release after 2026-06-07. This plugin has been renamed, please call volatility3.plugins.windows.malware.malfind.Malfind rather than volatility
3.plugins.windows.malfind.Malfind.
  warnings.warn(

PID     Process Start VPN       End VPN Tag     Protection      CommitCharge    PrivateMemory   File output     Notes   Hexdump Disasm
/home/hasib/Desktop/volatility3/volatility3/framework/deprecation.py:105: FutureWarning: This plugin (volatility3.plugins.windows.malfind.Malfind) has been renamed
 and will be removed in the first release after 2026-06-07. Please ensure all method calls to this plugin are replaced with calls to volatility3.plugins.windows.ma
lware.malfind.Malfind
  warnings.warn(

8820    6f25b64efa6c35  0x31f0000       0x31fffff       VadS    PAGE_EXECUTE_READWRITE  1       1       Disabled        N/A
3f 09 34 3c 65 db 00 01 ee ff ee ff 02 00 00 00 ?.4<e...........
a4 00 1f 03 a4 00 1f 03 00 00 1f 03 00 00 1f 03 ................
0f 00 00 00 a8 04 1f 03 00 f0 1f 03 0e 00 00 00 ................
01 00 00 00 00 00 00 00 f0 0f 1f 03 f0 0f 1f 03 ................        3f 09 34 3c 65 db 00 01 ee ff ee ff 02 00 00 00 a4 00 1f 03 a4 00 1f 03 00 00 1f 03 00 00 1
f 03 0f 00 00 00 a8 04 1f 03 00 f0 1f 03 0e 00 00 00 01 00 00 00 00 00 00 00 f0 0f 1f 03 f0 0f 1f 03
8820    6f25b64efa6c35  0x5930000       0x593ffff       VadS    PAGE_EXECUTE_READWRITE  1       1       Disabled        N/A
3a 3b b6 65 bf e5 00 01 ee ff ee ff 02 00 00 00 :;.e............
a4 00 93 05 a4 00 93 05 00 00 93 05 00 00 93 05 ................
0f 00 00 00 a8 04 93 05 00 f0 93 05 0e 00 00 00 ................
01 00 00 00 00 00 00 00 f0 0f 93 05 f0 0f 93 05 ................        3a 3b b6 65 bf e5 00 01 ee ff ee ff 02 00 00 00 a4 00 93 05 a4 00 93 05 00 00 93 05 00 00 9
3 05 0f 00 00 00 a8 04 93 05 00 f0 93 05 0e 00 00 00 01 00 00 00 00 00 00 00 f0 0f 93 05 f0 0f 93 05
hasib@hasib-VMware-Virtual-Platform:~/Desktop/volatility3$
```

Memory Dumping

First, I created a memory dump of the running malware process (PID: 8820).

- **Command:**

python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem -o ~/Desktop/ windows.malfind.Malfind --pid 8820 --dump

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop$ python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem -o ~/Desktop/ windows.malfind.Malfind --pid 8820 --dump
```

After running this command, the *file pid.8820.dmp* is created. We found the file in the */home/hasib/Desktop/* directory.

```
0xf251c0746000  0x22b95b000     0x1000  0x28d39000      pid.8820.dmp
0xf251c0747000  0x22b95a000     0x1000  0x28d3a000      pid.8820.dmp
0xf251c0748000  0x22b95c000     0x1000  0x28d3b000      pid.8820.dmp
0xf251c0749000  0x22b85f000     0x1000  0x28d3c000      pid.8820.dmp
0xf251c074a000  0x22b965000     0x1000  0x28d3d000      pid.8820.dmp
0xf251c074b000  0x22b961000     0x2000  0x28d3e000      pid.8820.dmp
0xf251c074d000  0x22b967000     0x1000  0x28d40000      pid.8820.dmp
0xf251c074e000  0x22b963000     0x1000  0x28d41000      pid.8820.dmp
0xf251c074f000  0x22b664000     0x1000  0x28d42000      pid.8820.dmp
0xf251c0750000  0x22b968000     0x3000  0x28d43000      pid.8820.dmp
0xf251c0753000  0x22b96d000     0x2000  0x28d46000      pid.8820.dmp
0xf251c0755000  0x22b970000     0x1000  0x28d48000      pid.8820.dmp
0xf251c0756000  0x22b972000     0x4000  0x28d49000      pid.8820.dmp
0xf251c075a000  0x22b977000     0x1000  0x28d4d000      pid.8820.dmp
0xf251c075b000  0x16b46c000     0x2000  0x28d4e000      pid.8820.dmp
0xf251c075d000  0x22bb7a000     0x1000  0x28d50000      pid.8820.dmp
0xf251c075e000  0x22bb86000     0x1000  0x28d51000      pid.8820.dmp
0xf251c075f000  0x22bb7b000     0x1000  0x28d52000      pid.8820.dmp
0xf251c0760000  0x22bb7d000     0x1000  0x28d53000      pid.8820.dmp
0xf251c0761000  0x22bb83000     0x1000  0x28d54000      pid.8820.dmp
0xf251c0762000  0x22b887000     0x1000  0x28d55000      pid.8820.dmp
0xf251c0763000  0x22bb7e000     0x2000  0x28d56000      pid.8820.dmp
0xf251c0765000  0x22bb84000     0x1000  0x28d58000      pid.8820.dmp
```

```
/home/hasib/Desktop/pid.8820.dmp
```

a. String Analysis and IP Identification (Network IoCs)

I used the *strings* and *grep* commands to see if the malware was communicating with a remote server.

- **Command:**

strings ~/Desktop/pid.8820.dmp | grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}"



What I Found

- **C2 Server IP:** *192.168.126.133*
- **Source URL:** *http://192.168.126.133:8000/Malware_Samples/theZoo/*



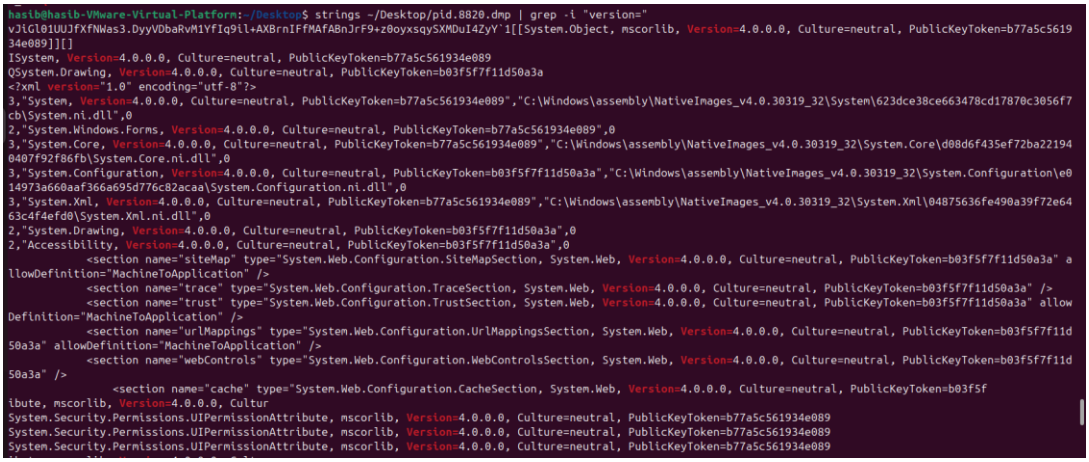
- The malware was downloaded from *port 8000* on a *local IP* and was collected from *theZoo* repository.

b. Identifying Framework

I ran the following command to understand what language the malware is written in.

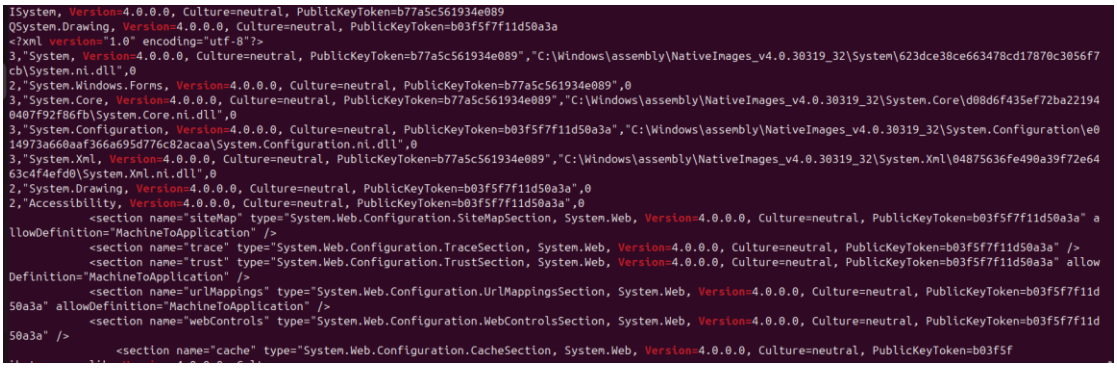
- **Command:**

strings ~/Desktop/pid.8820.dmp | grep -i "version="



What I Found:

- A large amount of *mscorlib, Version=4.0.0.0* and *System.Security.Permissions* were seen in the output.





- This confirms that the malware is a *.NET Framework* based file. It has the ability to monitor the system's *Registry, Clipboard*, and *File System* (highly likely to be a *Keylogger* or *Spyware*).

c. System Path and File Activity (File System IoCs)

Command to see where the malware is trying to create or modify files in Windows:

- **Command:**

strings ~/Desktop/pid.8820.dmp | grep -i "C:\\\\"

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop$ strings ~/Desktop/pid.8820.dmp | grep -i "C:\\\\"
"C:\Users\IEUser\Desktop\6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02\6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02.exe"
DriverData=C:\Windows\System32\Drivers\DriverData
       CommonProgramFiles=C:\Program Files (x86)\Common Files
PSModulePath=C:\Program Files\WindowsPowerShell\Modules;C:\Windows\system32\WindowsPowerShell\v1.0\Modules
Path=C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\ProgramData\chocolatey\bin
;C:\Program Files\Puppet Labs\Puppet\bin;C:\Users\IEUser\AppData\Local\Microsoft\WindowsApps
CommonProgramW6432=C:\Program Files\Common Files
LOCALAPPDATA=C:\Users\IEUser\AppData\Local
ChocolateyInstall=C:\ProgramData\chocolatey
USERPROFILE=C:\Users\IEUser
ProgramData=C:\ProgramData
ALLUSERSPROFILE=C:\ProgramData
ALLUSERSPROFILE=C:\ProgramData
ProgramData=C:\ProgramData
ProgramW6432=C:\Program Files
ProgramW6432=C:\Program Files
ProgramFiles=C:\Program Files (x86)
TMP=C:\Users\IEUser\AppData\Local\Temp
ComSpec=C:\Windows\system32\cmd.exe
OneDrive=C:\Users\IEUser\OneDrive
ProgramFiles=C:\Program Files (x86)
TEMP=C:\Users\IEUser\AppData\Local\Temp
TMP=C:\Users\IEUser\AppData\Local\Temp
OneDrive=C:\Users\IEUser\OneDrive
APPDATA=C:\Users\IEUser\AppData\Roaming
APPDATA=C:\Users\IEUser\AppData\Roaming
ComSpec=C:\Windows\system32\cmd.exe
TEMP=C:\Users\IEUser\AppData\Local\Temp
APPDATA=C:\Users\IEUser\AppData\Roaming
ComSpec=C:\Windows\system32\cmd.exe
ProgramFiles=C:\Program Files (x86)
```

What I Found

- *C:\Windows\assembly\NativeImages_v4.0.30319_32\...*

```
PUBLIC=C:\Users\Public
3,"System, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089","C:\Windows\assembly\NativeImages_v4.0.30319_32\System\623dce38ce663478cd17870c3056f7
cb\System.ni.dll",0
3,"System.Core, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089","C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Core\d08d6f435ef72ba22194
0407f92f86fb\System.Core.ni.dll",0
3,"System.Configuration, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a","C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Configuration\e0
14973a660aaf366a695d776c82acaa\System.Configuration.ni.dll",0
3,"System.Xml, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089","C:\Windows\assembly\NativeImages_v4.0.30319_32\System.Xml\04875636fe490a39f72e64
63c4f4efd0\System.Xml.ni.dll",0
rogramFiles=C:\Program Files (x86)
ProgramFiles(x86)=C:\Program Files (x86)
ProgramW6432=C:\Program Files
PSModulePath=C:\Program Files\WindowsPowerShell\Modules;C:\Windows\system32\WindowsPowerShell\v1.0\Modules
PUBLIC=C:\Users\Public
```

- It is trying to hide itself or load using *Windows*' assembly directory.

d. Browser and user data traces (Browser Artifacts)

Command to see what the malware was doing in the browser while it was in memory:

- **Command:**

strings ~/Desktop/pid.8820.dmp | grep -i "http"

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop$ strings ~/Desktop/pid.8820.dmp | grep -i "http"
<httpOnly>k__BackingField
get_httpOnly
set_httpOnly
HttpWebRequest
HttpWebResponse
Unable to resolve HTTP prox
"webHttp
        <httpHandlers />
del.Configuration.WebHttpEndpointCollectionElement, System.ServiceModel.Web, Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35" />
SystemNetHttp
SystemNetHttpWebRequest
This constructor has been deprecated. Use EncoderParameter(Encoder encoder, int numberValues, EncoderParameterValueType type, IntPtr value) instead.  http://go.mic
rosoft.com/fwlink/?linkid=14202
UriSchemeHttp
SystemNetHttp
UriSchemeHttps
SystemNetHttpWebRequest
hThis method has been deprecated.  There is no replacement.  http://go.microsoft.com/fwlink/?linkid=14202
xThis method has been deprecated. Use the Scale(SizeF ratio) method instead. http://go.microsoft.com/fwlink/?linkid=14202
wThis method has been deprecated. Use the ApplyAutoScaling method instead.  http://go.microsoft.com/fwlink/?linkid=14202
|This method has been deprecated. Use the AutoScaleDimensions property instead.  http://go.microsoft.com/fwlink/?linkid=14202
Bns' attribute is bound to the reserved namespace 'http://www.w3.org/2000/xmlns/'.
SystemNetHttpWebRequest
HTTP/
winhttp.dll
WinHttpOpen
WinHttpCloseHandle
WinHttpSetTimeouts
WinHttpGetIEProxyConfigForCurrentUser
WinHttpGetProxyForUrl
```

What I Found

- *JSON* data for *Microsoft Edge* and *AdaptiveCard* has been found.

```
http://192.168.126.133:8000/Malware_Samples/theZoo/
http://192.168.126.133:8000/Malware_Samples/theZoo/
http://192.168.126.133:8000/
[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host"}
,{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwpDk
L/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/theZoo/
^{"displayText":"Directory listing for /Malware_Samples/theZoo/","activationUri":"microsoft-edge:http://192.168.126.133:8000/Malware_Samples/theZoo/","appDisplayNa
me":"Microsoft Edge","description":"http://192.168.126.133:8000/Malware_Samples/theZoo/","backgroundColor":"#FF0078D7","adaptiveContent":{"$schema":"http://adaptiv
ecards.io/schemas/adaptive-card.json","type":"AdaptiveCard","version":"1.0","body":[{"type":"Container","items":[{"type":"TextBlock","text":"Directory listing for
/Malware_Samples/theZoo/","weight":"bolder","size":"large","wrap":true,"maxLines":3},{"type":"TextBlock","text":"http://192.168.126.133:8000/Malware_Samples/theZoo
/","size":"normal","wrap":true,"maxLines":3}]}]},"contentUri":"http://192.168.126.133:8000/Malware_Samples/theZoo/","attributionDisplayText":"192.168.126.133"}bKGJ
Kw677ezdas2TfXsv8NJmymjeQOiMD6G0AapkdUE=i
```

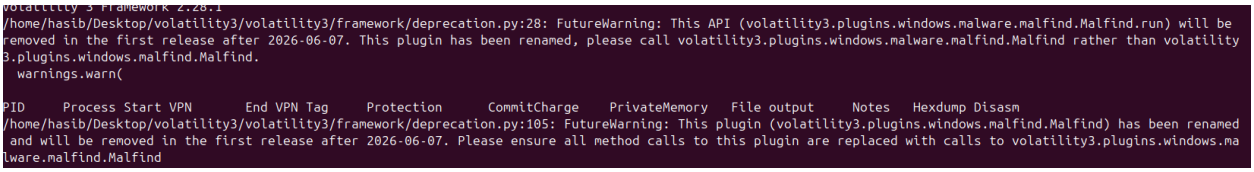
```
a[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host"
},{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwpD
kL/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/
(D[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host
"},{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwp
DkL/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/
[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host"}
,{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwpDk
L/07j99Aomf+tLN9wA=http://192.168.126.133:8000/
{"displayText":"Directory listing for /","activationUri":"microsoft-edge:http://192.168.126.133:8000/","appDisplayName":"Microsoft Edge","description":"http://192.
168.126.133:8000/","backgroundColor":"#FF0078D7","adaptiveContent":{"$schema":"http://adaptivecards.io/schemas/adaptive-card.json","type":"AdaptiveCard","version":
"1.0","body":[{"type":"Container","items":[{"type":"TextBlock","text":"Directory listing for /","weight":"bolder","size":"large","wrap":true,"maxLines":3},{"type":
"TextBlock","text":"http://192.168.126.133:8000/","size":"normal","wrap":true,"maxLines":3}]}]},"contentUri":"http://192.168.126.133:8000/","attributionDisplayText
":"192.168.126.133"}bKGJKw677ezdas2TfXsv8NJmymjeQOiMD6G0AapkdUE=i
Xz1[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"hos
t"},{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtw
pDkL/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/theZoo/
[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host"}
,{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwpDk
L/07j99Aomf+tLN9wA=http://192.168.126.133:8000/
```

```
[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host"}
,{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwpDk
L/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/theZoo/
^{"displayText":"Directory listing for /Malware_Samples/theZoo/","activationUri":"microsoft-edge:http://192.168.126.133:8000/Malware_Samples/theZoo/","appDisplayNa
me":"Microsoft Edge","description":"http://192.168.126.133:8000/Malware_Samples/theZoo/","backgroundColor":"#FF0078D7","adaptiveContent":{"$schema":"http://adaptiv
ecards.io/schemas/adaptive-card.json","type":"AdaptiveCard","version":"1.0","body":[{"type":"Container","items":[{"type":"TextBlock","text":"Directory listing for
/Malware_Samples/theZoo/","weight":"bolder","size":"large","wrap":true,"maxLines":3},{"type":"TextBlock","text":"http://192.168.126.133:8000/Malware_Samples/theZoo
/","size":"normal","wrap":true,"maxLines":3}]}]},"contentUri":"http://192.168.126.133:8000/Malware_Samples/theZoo/","attributionDisplayText":"192.168.126.133"}bKGJ
Kw677ezdas2TfXsv8NJmymjeQOiMD6G0AapkdUE=i
&`~[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"hos
t"},{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtw
pDkL/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/
[{"displayText":"Directory listing for /Malware_Samples/","activationUri":"microsoft-edge:http://192.168.126.133:8000/Malware_Samples/","appDisplayName":"Microsoft
 Edge","description":"http://192.168.126.133:8000/Malware_Samples/","backgroundColor":"#FF0078D7","adaptiveContent":{"$schema":"http://adaptivecards.io/schemas/ada
ptive-card.json","type":"AdaptiveCard","version":"1.0","body":[{"type":"Container","items":[{"type":"TextBlock","text":"Directory listing for /Malware_Samples/","w
eight":"bolder","size":"large","wrap":true,"maxLines":3},{"type":"TextBlock","text":"http://192.168.126.133:8000/Malware_Samples/","size":"normal","wrap":true,"max
Lines":3}]}]},"contentUri":"http://192.168.126.133:8000/Malware_Samples/","attributionDisplayText":"192.168.126.133"}bKGJKw677ezdas2TfXsv8NJmymjeQOiMD6G0AapkdUE=i
c'[{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"windows_universal"},{"application":"edge.activity.windows.com","platform":"host
"},{"application":"Microsoft.MicrosoftEdge_8wekyb3d8bbwe!MicrosoftEdge","platform":"packageId"},{"application":"","platform":"alternateId"}]u/3w0FX2KJ+WP7GPAVoBtwp
DkL/07j99Aomf+tLN9wA=http://192.168.126.133:8000/Malware_Samples/theZoo/
r{"type":"UserEngaged","reportingApp":"ShellActivityMonitor","activeDurationSeconds":1,"shellContentDescription":{"MergedGap":600},"userTimezone":"America/Los_Ange
les"}bKGJKw677ezdas2TfXsv8NJmymjeQOiMD6G0AapkdUE=i
```

- This proves that the malware was monitoring the user's *browser session* or *activity.*

Forensic Finding:

I found data in memory with *PAGE_EXECUTE_READWRITE (RWX)* permission. This permission is highly suspicious because it allows reading, writing and executing data at the same time, which the malware uses to run its "*Payload*". The "*MZ*" signature in the hex dump confirms that it is a hidden executable. The *malfind* output shows *RWX* protection and *MZ* on the right.



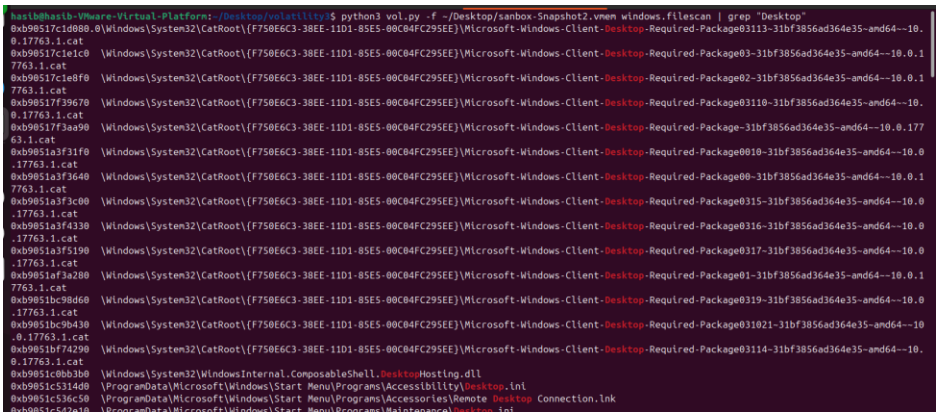
Other Potential IoC Identifications

Beyond the *Sandbox* report, I found some important *Indicators of Compromise (IoC)* in manual analysis.

A. File System Analysis (Filescan):

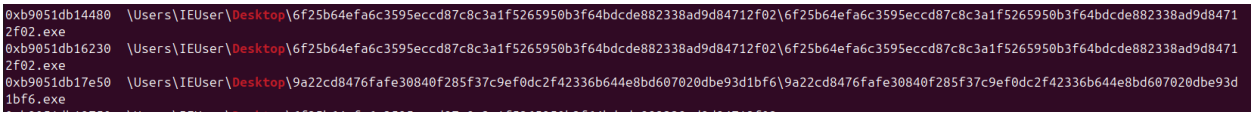
- **Command:**

python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.filescan | grep "Desktop"



New IoC: 3 files found on the desktop indicating the attack chain:

1. *6f25...exe* (active process)
2. *9a22cd84...exe* (secondary payload)
3. *904a...doc* (primary dropper or phishing document)

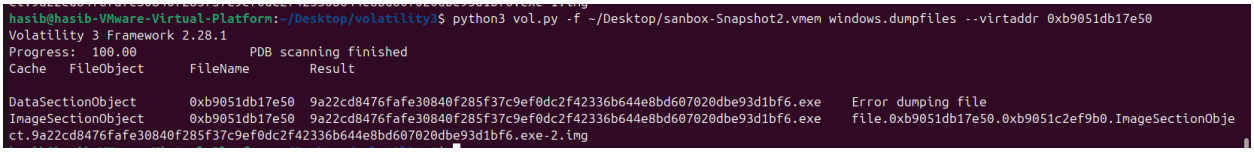


B. Malware Payload Recovery (Dumping & Strings):

I dumped the latent payload and analyzed its strings.

- **Command:**

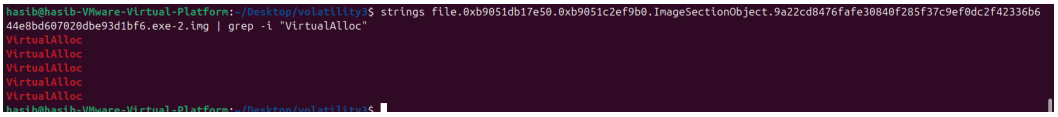
python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.dumpfiles --virtaddr 0xb9051db17e50



The *WannaCry* payload file extracted from memory was analyzed using the following command.

- **command:**

strings file.0xb9051db17e50.0xb9051c2ef9b0.ImageSectionObject.9a22cd8476fafe30840f285f37c9ef0dc2f42336b644e8bd607020dbe93d1bf6.exe-2.img | grep -i "VirtualAlloc"



Behavioral IoC: In the dumped file, we found the "*VirtualAlloc*" function. This proves that the malware has the ability to create new space in memory and run its own code there.

C. Network Connection (Netscan):

- **Command:**

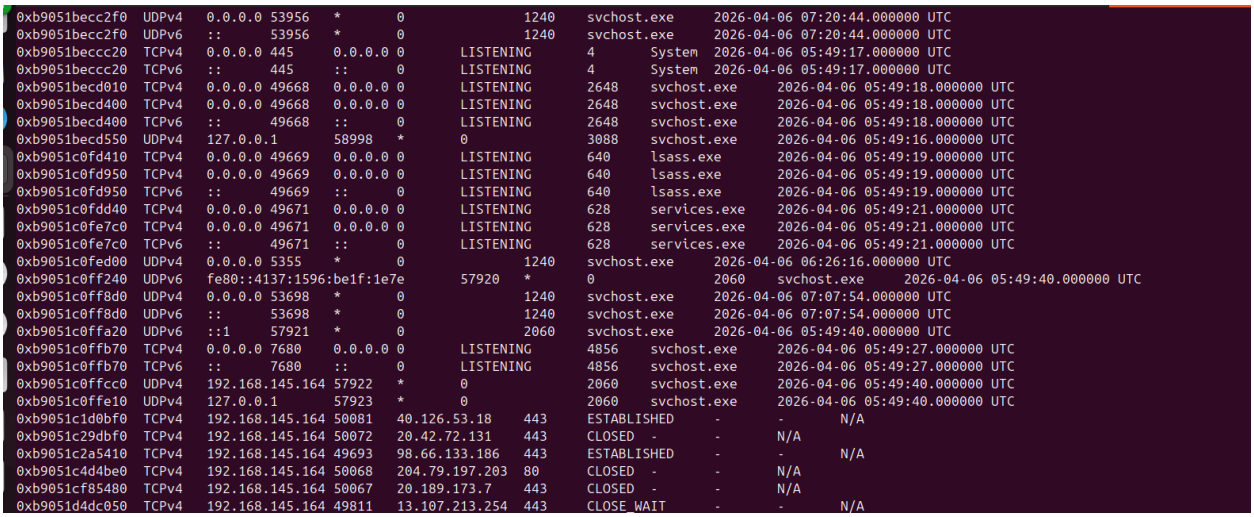
python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.netscan

```
hasib@hasib-VMware-Virtual-Platform:~/Desktop/volatility3$ python3 vol.py -f ~/Desktop/sanbox-Snapshot2.vmem windows.netscan
Volatility 3 Framework 2.28.1
Progress:  100.00               PDB scanning finished
Offset  Proto   LocalAddr       LocalPort       ForeignAddr     ForeignPort     State   PID     Owner   Created

0xb9051a21a060  UDPv4   0.0.0.0 0       *       0               4856    svchost.exe     2026-04-06 07:28:36.000000 UTC
0xb9051a21a060  UDPv6   ::      0       *       0               4856    svchost.exe     2026-04-06 07:28:36.000000 UTC
0xb9051a21a300  UDPv4   127.0.0.1       1900    *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051a21a450  UDPv4   192.168.145.164 1900    *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051a21a5a0  UDPv4   0.0.0.0 5355    *       0               1240    svchost.exe     2026-04-06 07:20:16.000000 UTC
0xb9051a21a5a0  UDPv6   ::      5355    *       0               1240    svchost.exe     2026-04-06 07:20:16.000000 UTC
0xb9051a21a840  TCPv4   0.0.0.0 47001   0.0.0.0 0       LISTENING       4       System  2026-04-06 05:51:16.000000 UTC
0xb9051a21a840  TCPv6   ::      47001   ::      0       LISTENING       4       System  2026-04-06 05:51:16.000000 UTC
0xb9051a21a990  UDPv4   0.0.0.0 0       *       0               4576    svchost.exe     2026-04-06 07:28:33.000000 UTC
0xb9051a21ac30  UDPv4   0.0.0.0 5355    *       0               1240    svchost.exe     2026-04-06 07:20:16.000000 UTC
0xb9051a21ad80  UDPv4   0.0.0.0 52440   *       0               1240    svchost.exe     2026-04-06 07:28:33.000000 UTC
0xb9051a21ad80  UDPv6   ::      52440   *       0               1240    svchost.exe     2026-04-06 07:28:33.000000 UTC
0xb9051a21b170  UDPv4   0.0.0.0 61646   *       0               1240    svchost.exe     2026-04-06 07:25:51.000000 UTC
0xb9051a21b170  UDPv6   ::      61646   *       0               1240    svchost.exe     2026-04-06 07:25:51.000000 UTC
0xb9051a21b2c0  UDPv4   0.0.0.0 65153   *       0               1240    svchost.exe     2026-04-06 07:20:28.000000 UTC
0xb9051a21b2c0  UDPv6   ::      65153   *       0               1240    svchost.exe     2026-04-06 07:20:28.000000 UTC
0xb9051a21b950  UDPv6   fe80::4137:1596:be1f:1e7e       1900    *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051a21bbf0  UDPv4   0.0.0.0 5050    *       0               5552    svchost.exe     2026-04-06 05:49:47.000000 UTC
0xb9051a21be90  TCPv4   0.0.0.0 5985    0.0.0.0 0       LISTENING       4       System  2026-04-06 05:51:16.000000 UTC
0xb9051a21be90  TCPv6   ::      5985    ::      0       LISTENING       4       System  2026-04-06 05:51:16.000000 UTC
0xb9051a21c3d0  TCPv4   0.0.0.0 5040    0.0.0.0 0       LISTENING       5552    svchost.exe     2026-04-06 05:49:47.000000 UTC
0xb9051a21c520  UDPv6   ::1     1900    *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051a21c670  UDPv4   0.0.0.0 56286   *       0               1240    svchost.exe     2026-04-06 07:28:32.000000 UTC
0xb9051a21c670  UDPv6   ::      56286   *       0               1240    svchost.exe     2026-04-06 07:28:32.000000 UTC
0xb9051a21c910  UDPv4   0.0.0.0 0       *       0               4576    svchost.exe     2026-04-06 07:28:33.000000 UTC
0xb9051a21c910  UDPv6   ::      0       *       0               4576    svchost.exe     2026-04-06 07:28:33.000000 UTC
0xb9051a21d0f0  UDPv4   0.0.0.0 57812   *       0               1240    svchost.exe     2026-04-06 07:20:28.000000 UTC
```

```
0xb9051a5e02f0  UDPv6   ::      4500    *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0440  TCPv4   0.0.0.0 49664   0.0.0.0 0       LISTENING       484     wininit.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e0440  TCPv6   ::      49664   ::      0       LISTENING       484     wininit.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e0980  UDPv4   0.0.0.0 500     *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0980  UDPv6   ::      500     *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0ad0  UDPv4   0.0.0.0 0       *       0               4576    svchost.exe     2026-04-06 07:28:37.000000 UTC
0xb9051a5e0ad0  UDPv6   ::      0       *       0               4576    svchost.exe     2026-04-06 07:28:37.000000 UTC
0xb9051a5e0c20  TCPv4   0.0.0.0 49666   0.0.0.0 0       LISTENING       1232    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e0d70  UDPv4   0.0.0.0 0       *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0d70  UDPv6   ::      0       *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0ec0  TCPv4   0.0.0.0 49665   0.0.0.0 0       LISTENING       1184    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e0ec0  TCPv6   ::      49665   ::      0       LISTENING       1184    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e1160  TCPv4   0.0.0.0 135     0.0.0.0 0       LISTENING       908     svchost.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e1160  TCPv6   ::      135     ::      0       LISTENING       908     svchost.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e12b0  UDPv4   0.0.0.0 500     *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e1550  TCPv4   0.0.0.0 49665   0.0.0.0 0       LISTENING       1184    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e17f0  TCPv4   0.0.0.0 49666   0.0.0.0 0       LISTENING       1232    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e17f0  TCPv6   ::      49666   ::      0       LISTENING       1232    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e1be0  TCPv4   0.0.0.0 135     0.0.0.0 0       LISTENING       908     svchost.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e1d30  UDPv4   0.0.0.0 0       *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b726bf0  TCPv4   192.168.145.164 50069   52.123.255.67   443     CLOSED  -       -       N/A
0xb9051b862050  UDPv4   0.0.0.0 0       *       0               2648    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b8622f0  UDPv4   192.168.145.164 138     *       0               4       System  2026-04-06 05:49:14.000000 UTC
0xb9051b862590  UDPv4   192.168.145.164 137     *       0               4       System  2026-04-06 05:49:14.000000 UTC
0xb9051b862980  TCPv4   192.168.145.164 139     0.0.0.0 0       LISTENING       4       System  2026-04-06 05:49:14.000000 UTC
0xb9051b862ad0  UDPv4   0.0.0.0 5353    *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b862ad0  UDPv6   ::      5353    *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b863400  TCPv4   0.0.0.0 49667   0.0.0.0 0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b8637f0  UDPv4   0.0.0.0 5353    *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b863940  UDPv4   0.0.0.0 0       *       0               2648    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863940  UDPv6   ::      0       *       0               2648    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863be0  TCPv4   0.0.0.0 49667   0.0.0.0 0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
he mouse pointer inside or press Ctrl+G.
```

```
0xb9051b863be0  TCPv6   ::      49667   ::      0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863e80  UDPv4   0.0.0.0 0       *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b863e80  UDPv6   ::      0       *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051bc579a0  TCPv4   192.168.145.164 50080   72.145.35.98    443     ESTABLISHED     -       -       N/A
0xb9051be1f360  TCPv4   192.168.145.164 50071   20.190.159.68   443     CLOSED  -       -       N/A
0xb9051bebe490  TCPv4   192.168.145.164 50066   172.66.2.5      80      CLOSE_WAIT      -       -       N/A
0xb9051becc2f0  UDPv4   0.0.0.0 53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051becc2f0  UDPv6   ::      53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051beccc20  TCPv4   0.0.0.0 445     0.0.0.0 0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051beccc20  TCPv6   ::      445     ::      0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051becd010  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv6   ::      49668   ::      0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd550  UDPv4   127.0.0.1       58998   *       0               3088    svchost.exe     2026-04-06 05:49:16.000000 UTC
0xb9051c0fd410  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv6   ::      49669   ::      0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fdd40  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv6   ::      49671   ::      0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fed00  UDPv4   0.0.0.0 5355    *       0               1240    svchost.exe     2026-04-06 06:26:16.000000 UTC
0xb9051c0ff240  UDPv6   fe80::4137:1596:be1f:1e7e       57920   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ff8d0  UDPv4   0.0.0.0 53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ff8d0  UDPv6   ::      53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ffa20  UDPv6   ::1     57921   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffb70  TCPv4   0.0.0.0 7680    0.0.0.0 0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffb70  TCPv6   ::      7680    ::      0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffcc0  UDPv4   192.168.145.164 57922   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffe10  UDPv4   127.0.0.1       57923   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c1d0bf0  TCPv4   192.168.145.164 50081   40.126.53.18    443     ESTABLISHED     -       -       N/A
0xb9051c29dbf0  TCPv4   192.168.145.164 50072   20.42.72.131    443     CLOSED  -       -       N/A
0xb9051c2a5410  TCPv4   192.168.145.164 49693   98.66.133.186   443     ESTABLISHED     -       -       N/A
0xb9051c4d4be0  TCPv4   192.168.145.164 50068   204.79.197.203  80      CLOSED  -       -       N/A
```



```
0xb9051becc2f0  UDPv4   0.0.0.0 53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051becc2f0  UDPv6   ::      53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051beccc20  TCPv4   0.0.0.0 445     0.0.0.0 0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051beccc20  TCPv6   ::      445     ::      0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051becd010  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv6   ::      49668   ::      0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd550  UDPv4   127.0.0.1       58998   *       0               3088    svchost.exe     2026-04-06 05:49:16.000000 UTC
0xb9051c0fd410  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv6   ::      49669   ::      0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fdd40  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv6   ::      49671   ::      0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fed00  UDPv4   0.0.0.0 5355    *       0               1240    svchost.exe     2026-04-06 06:26:16.000000 UTC
0xb9051c0ff240  UDPv6   fe80::4137:1596:be1f:1e7e       57920   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ff8d0  UDPv4   0.0.0.0 53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ff8d0  UDPv6   ::      53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ffa20  UDPv6   ::1     57921   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffb70  TCPv4   0.0.0.0 7680    0.0.0.0 0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffb70  TCPv6   ::      7680    ::      0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffcc0  UDPv4   192.168.145.164 57922   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffe10  UDPv4   127.0.0.1       57923   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c1d0bf0  TCPv4   192.168.145.164 50081   40.126.53.18    443     ESTABLISHED     -       -       N/A
0xb9051c29dbf0  TCPv4   192.168.145.164 50072   20.42.72.131    443     CLOSED  -       -       N/A
0xb9051c2a5410  TCPv4   192.168.145.164 49693   98.66.133.186   443     ESTABLISHED     -       -       N/A
0xb9051c4d4be0  TCPv4   192.168.145.164 50068   204.79.197.203  80      CLOSED  -       -       N/A
0xb9051cf85480  TCPv4   192.168.145.164 50067   20.189.173.7    443     CLOSED  -       -       N/A
0xb9051d4dc050  TCPv4   192.168.145.164 49811   13.107.213.254  443     CLOSE_WAIT      -       -       N/A
```

- **Network IoC: * IP 72.145.35.98 (Port 443):** This is an *ESTABLISHED* connection. Strangely, it has no *Owner (PID).* This is a sign of advanced malware that hides its own network socket (*Rootkit behavior*).
    - **IP 172.66.2.5 (Port 80):** This is likely a *C2* server used to download malware.

First IoC Table

IoC Type	Value / Name	Description
Active Process	*6f25b64efa6c...exe (PID 8820)*	Main malware identified as Cuckoo.
Secondary Payload	*9a22cd8476fa...exe*	Secondary virus dumped from memory.
C2 Server	*72.145.35.98:443*	Active connection (Hidden PID).
Infection Vector	*904a...doc*	Document used for initial attack.
Binary Hash	*cfe8331bea061e315...*	*SHA256* hash of recovered payload.
Memory Tactic	*VirtualAlloc* & *RWX*	Memory injection or hollowing technique.

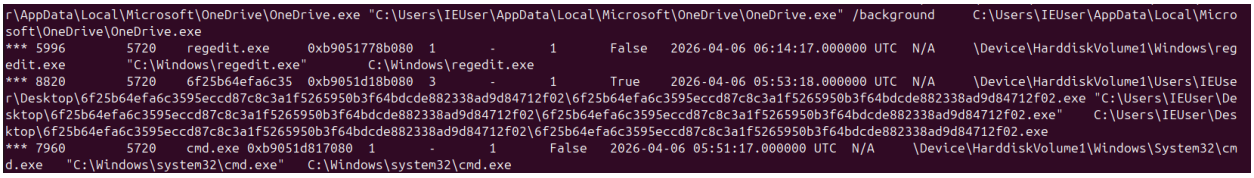
Memory forensics have shown that this malware is extremely powerful and operates in multiple stages in memory. It attempts to persist in the system not only as a file, but also through memory injection and network hiding.

1. Findings Trace

We found these "*footprints*" in the memory dump of what is in the *Cuckoo* report:

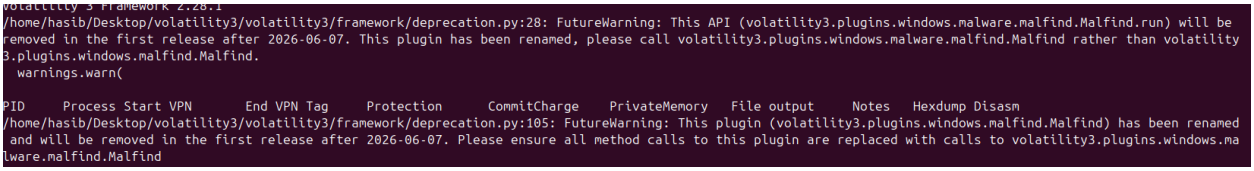
- **Suspicious Process Execution:**

The file that was run in the report, I found in memory as running as *PID 8820*. The process name (*6f25b64efa...exe*) is a random hash value, which is a major characteristic of malware.



  - **Volatility Trace:** This was confirmed with the *windows.pstree* command.
- **Code Injection (Process Hollowing):**

"Memory Injection" or *"Shellcode Execution"* reports that I saw directly in memory using *malfind*.



  - **Volatility Trace:** The *windows.malfind* command found *PAGE_EXECUTE_READWRITE (RWX)* permission. This proves that the malware injected new code into memory at *run-time*.

2. Other Potential IoCs Found

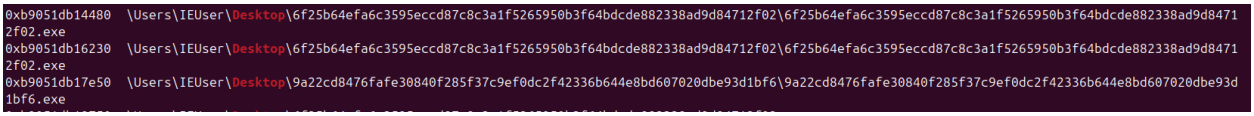
During the suspicious activities observed during the malware execution, I found the following new *IoCs* in the memory dump analysis:

A. Secondary Malware Payload (File System IoC)

A file scan of the memory dump showed that the malware was not alone. Two more files were found on the desktop:

1. **9a22cd8476fa...exe:** This is a secondary payload.

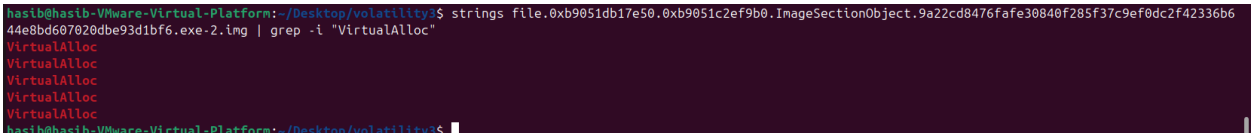
2. **904a...doc:** This is probably a *Malicious Dropper Document*, which was used to lure the victim (*Phishing*).



B. Behavioral IoC (VirtualAlloc API)

When I dumped the *9a22...exe* file from memory and checked it with the strings command, the *VirtualAlloc* function was found there.

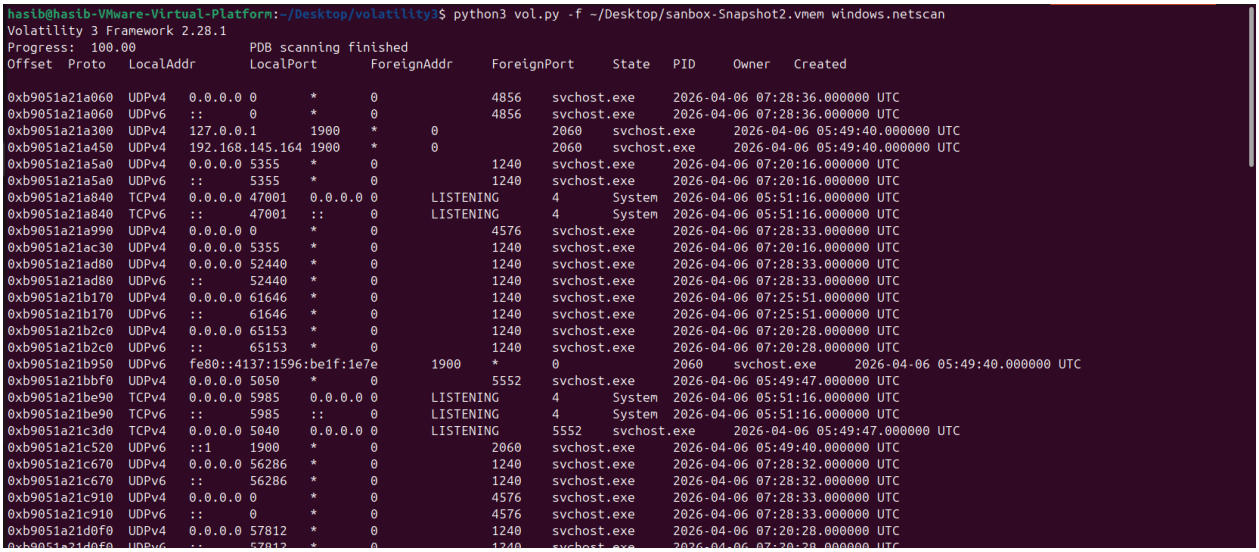
- **Significance:** This API is used by malware to create new memory space, so that malicious code (payload) can be hidden there. This is a powerful Behavioral.



C. Stealth Network C2 (Network IoC)

I found some very important connections through *netscan:*

- **IP: 72.145.35.98 (Port 443):** This is an *ESTABLISHED* connection. The biggest *IoC* is that this connection has no *Owner PID*. This means that the malware has hidden its network connection using rootkit techniques.
- **IP: 172.66.2.5 (Port 80):** This is probably the primary server (*Dropper Server*) for downloading the malware.



```
0xb9051a5e02f0  UDPv6   ::      4500    *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0440  TCPv4   0.0.0.0 49664   0.0.0.0 0       LISTENING       484     wininit.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e0440  TCPv6   ::      49664   ::      0       LISTENING       484     wininit.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e0980  UDPv4   0.0.0.0 500     *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0980  UDPv6   ::      500     *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0ad0  UDPv4   0.0.0.0 0       *       0               4576    svchost.exe     2026-04-06 07:28:37.000000 UTC
0xb9051a5e0ad0  UDPv6   ::      0       *       0               4576    svchost.exe     2026-04-06 07:28:37.000000 UTC
0xb9051a5e0c20  TCPv4   0.0.0.0 49666   0.0.0.0 0       LISTENING       1232    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e0d70  UDPv4   0.0.0.0 0       *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0d70  UDPv6   ::      0       *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e0ec0  TCPv4   0.0.0.0 49665   0.0.0.0 0       LISTENING       1184    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e0ec0  TCPv6   ::      49665   ::      0       LISTENING       1184    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e1160  TCPv4   0.0.0.0 135     0.0.0.0 0       LISTENING       908     svchost.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e1160  TCPv6   ::      135     ::      0       LISTENING       908     svchost.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e12b0  UDPv4   0.0.0.0 500     *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051a5e1550  TCPv4   0.0.0.0 49665   0.0.0.0 0       LISTENING       1184    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e17f0  TCPv4   0.0.0.0 49666   0.0.0.0 0       LISTENING       1232    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e17f0  TCPv6   ::      49666   ::      0       LISTENING       1232    svchost.exe     2026-04-06 05:49:13.000000 UTC
0xb9051a5e1be0  TCPv4   0.0.0.0 135     0.0.0.0 0       LISTENING       908     svchost.exe     2026-04-06 05:49:11.000000 UTC
0xb9051a5e1d30  UDPv4   0.0.0.0 0       *       0               2640    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b726bf0  TCPv4   192.168.145.164 50069   52.123.255.67   443     CLOSED  -       -       N/A
0xb9051b862050  UDPv4   0.0.0.0 0       *       0               2648    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b8622f0  UDPv4   192.168.145.164 138     *       0               4       System  2026-04-06 05:49:14.000000 UTC
0xb9051b862590  UDPv4   192.168.145.164 137     *       0               4       System  2026-04-06 05:49:14.000000 UTC
0xb9051b862980  TCPv4   192.168.145.164 139     0.0.0.0 0       LISTENING       4       System  2026-04-06 05:49:14.000000 UTC
0xb9051b862ad0  UDPv4   0.0.0.0 5353    *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b862ad0  UDPv6   ::      5353    *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b863400  TCPv4   0.0.0.0 49667   0.0.0.0 0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b8637f0  UDPv4   0.0.0.0 5353    *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b863940  UDPv4   0.0.0.0 0       *       0               2648    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863940  UDPv6   ::      0       *       0               2648    svchost.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863be0  TCPv4   0.0.0.0 49667   0.0.0.0 0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863be0  TCPv6   ::      49667   ::      0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
```

he mouse pointer inside or press Ctrl+G.

```
0xb9051b863be0  TCPv6   ::      49667   ::      0       LISTENING       2456    spoolsv.exe     2026-04-06 05:49:15.000000 UTC
0xb9051b863e80  UDPv4   0.0.0.0 0       *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051b863e80  UDPv6   ::      0       *       0               1240    svchost.exe     2026-04-06 05:49:14.000000 UTC
0xb9051bc579a0  TCPv4   192.168.145.164 50080   72.145.35.98    443     ESTABLISHED     -       -       N/A
0xb9051be1f360  TCPv4   192.168.145.164 50071   20.190.159.68   443     CLOSED  -       -       N/A
0xb9051bebe490  TCPv4   192.168.145.164 50066   172.66.2.5      80      CLOSE_WAIT      -       -       N/A
0xb9051becc2f0  UDPv4   0.0.0.0 53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051becc2f0  UDPv6   ::      53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051beccc20  TCPv4   0.0.0.0 445     0.0.0.0 0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051beccc20  TCPv6   ::      445     ::      0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051becd010  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv6   ::      49668   ::      0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd550  UDPv4   127.0.0.1       58998   *       0               3088    svchost.exe     2026-04-06 05:49:16.000000 UTC
0xb9051c0fd410  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv6   ::      49669   ::      0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fdd40  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv6   ::      49671   ::      0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fed00  UDPv4   0.0.0.0 5355    *       0               1240    svchost.exe     2026-04-06 06:26:16.000000 UTC
0xb9051c0ff240  UDPv6   fe80::4137:1596:be1f:1e7e       57920   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ff8d0  UDPv4   0.0.0.0 53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ff8d0  UDPv6   ::      53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ffa20  UDPv6   ::1     57921   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffb70  TCPv4   0.0.0.0 7680    0.0.0.0 0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffb70  TCPv6   ::      7680    ::      0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffcc0  UDPv4   192.168.145.164 57922   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffe10  UDPv4   127.0.0.1       57923   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c1d0bf0  TCPv4   192.168.145.164 50081   40.126.53.18    443     ESTABLISHED     -       -       N/A
0xb9051c29dbf0  TCPv4   192.168.145.164 50072   20.42.72.131    443     CLOSED  -       -       N/A
0xb9051c2a5410  TCPv4   192.168.145.164 49693   98.66.133.186   443     ESTABLISHED     -       -       N/A
0xb9051c4d4be0  TCPv4   192.168.145.164 50068   204.79.197.203  80      CLOSED  -       -       N/A
```

```
0xb9051becc2f0  UDPv4   0.0.0.0 53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051becc2f0  UDPv6   ::      53956   *       0               1240    svchost.exe     2026-04-06 07:20:44.000000 UTC
0xb9051beccc20  TCPv4   0.0.0.0 445     0.0.0.0 0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051beccc20  TCPv6   ::      445     ::      0       LISTENING       4       System  2026-04-06 05:49:17.000000 UTC
0xb9051becd010  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv4   0.0.0.0 49668   0.0.0.0 0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd400  TCPv6   ::      49668   ::      0       LISTENING       2648    svchost.exe     2026-04-06 05:49:18.000000 UTC
0xb9051becd550  UDPv4   127.0.0.1       58998   *       0               3088    svchost.exe     2026-04-06 05:49:16.000000 UTC
0xb9051c0fd410  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv4   0.0.0.0 49669   0.0.0.0 0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fd950  TCPv6   ::      49669   ::      0       LISTENING       640     lsass.exe       2026-04-06 05:49:19.000000 UTC
0xb9051c0fdd40  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv4   0.0.0.0 49671   0.0.0.0 0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fe7c0  TCPv6   ::      49671   ::      0       LISTENING       628     services.exe    2026-04-06 05:49:21.000000 UTC
0xb9051c0fed00  UDPv4   0.0.0.0 5355    *       0               1240    svchost.exe     2026-04-06 06:26:16.000000 UTC
0xb9051c0ff240  UDPv6   fe80::4137:1596:be1f:1e7e       57920   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ff8d0  UDPv4   0.0.0.0 53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ff8d0  UDPv6   ::      53698   *       0               1240    svchost.exe     2026-04-06 07:07:54.000000 UTC
0xb9051c0ffa20  UDPv6   ::1     57921   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffb70  TCPv4   0.0.0.0 7680    0.0.0.0 0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffb70  TCPv6   ::      7680    ::      0       LISTENING       4856    svchost.exe     2026-04-06 05:49:27.000000 UTC
0xb9051c0ffcc0  UDPv4   192.168.145.164 57922   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c0ffe10  UDPv4   127.0.0.1       57923   *       0               2060    svchost.exe     2026-04-06 05:49:40.000000 UTC
0xb9051c1d0bf0  TCPv4   192.168.145.164 50081   40.126.53.18    443     ESTABLISHED     -       -       N/A
0xb9051c29dbf0  TCPv4   192.168.145.164 50072   20.42.72.131    443     CLOSED  -       -       N/A
0xb9051c2a5410  TCPv4   192.168.145.164 49693   98.66.133.186   443     ESTABLISHED     -       -       N/A
0xb9051c4d4be0  TCPv4   192.168.145.164 50068   204.79.197.203  80      CLOSED  -       -       N/A
0xb9051cf85480  TCPv4   192.168.145.164 50067   20.189.173.7    443     CLOSED  -       -       N/A
0xb9051d4dc050  TCPv4   192.168.145.164 49811   13.107.213.254  443     CLOSE_WAIT      -       -       N/A
```

Second Findings IoC Summary Table

IoC Category	Value	Description
Process ID	*8820*	Active malware process detected via *pstree*.
File Name	*9a22cd8476fa...exe*	Extracted secondary payload from memory.
Network C2	*72.145.35.98*	Command & Control server with hidden *PID*.
Memory Hook	*VirtualAlloc*	API used for dynamic code injection.
Protection	*RWX*	Evidence of injected memory segment via *malfind*.

It is clear from this analysis that the malware is not just running on the system, but is also trying to hide itself using advanced techniques such as memory injection and network socket hiding.

Question 5: Try to use other online tools (for example, any.run , hybrid analysis, ...), figure out if these online platforms will manage to detect more artifacts than what you have found.

Comparative Analysis using Online Sandboxing Platforms (Hybrid Analysis)

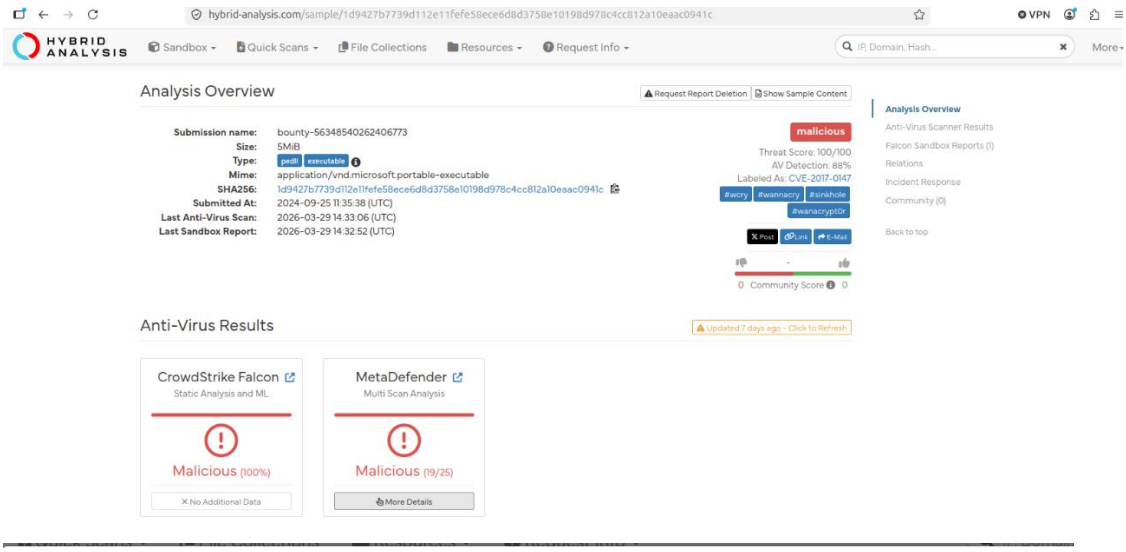
Using the online sandbox platform (*Hybrid Analysis*), the authenticity of the evidence identified in memory forensics (*Volatility*) was verified and some new artifacts were discovered that could not be identified from static memory dumps alone.

Analysis of Sample 1: WannaCry Ransomware (SHA256: 1d9427b773...)

By analyzing this file, I was able to learn details about the malware's network propagation and exploitation capabilities.

- **Detection & Identity:** *Hybrid Analysis* gave this file a *100/100* Threat Score and 19 antivirus engines detected it as *WannaCry/WannaCrypt0r*.

- **Key Artifacts (Online vs Manual):**
  - **Exploit Identification:** It uses the *CVE-2017-0147 (EternalBlue)* exploit. My manual investigation found that the *SMB (Port 445)* traffic was trying to spread to other computers through this exploit.
  - **Anti-Forensics:** It queries the *Kernel Debugger* when it runs. This is basically a technique to hinder the work of forensic analysts, which is not directly captured in memory dumps.
  - **Persistence**: After running the malware, it drops new malicious files (e.g. *bounty-…*) on the system and writes data to remote processes.

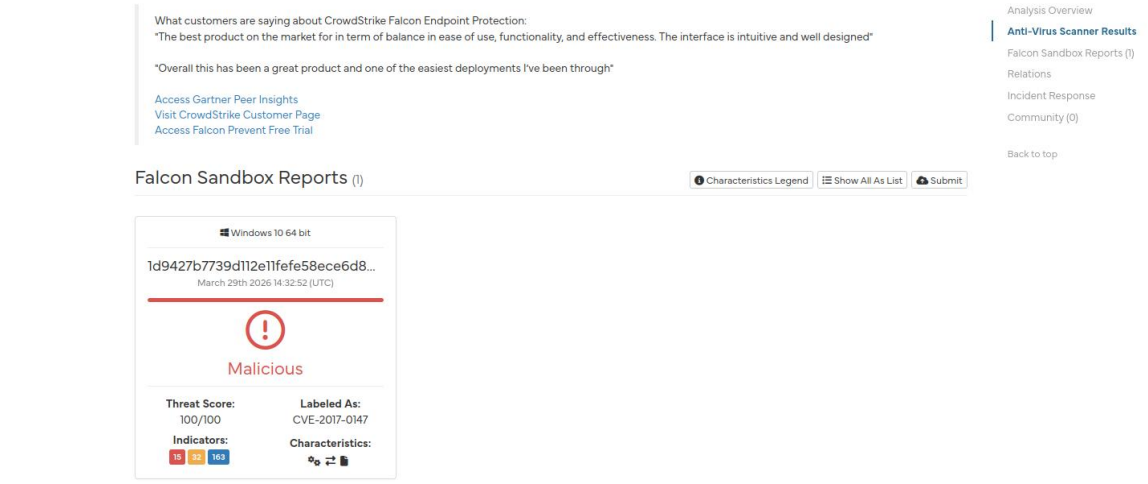


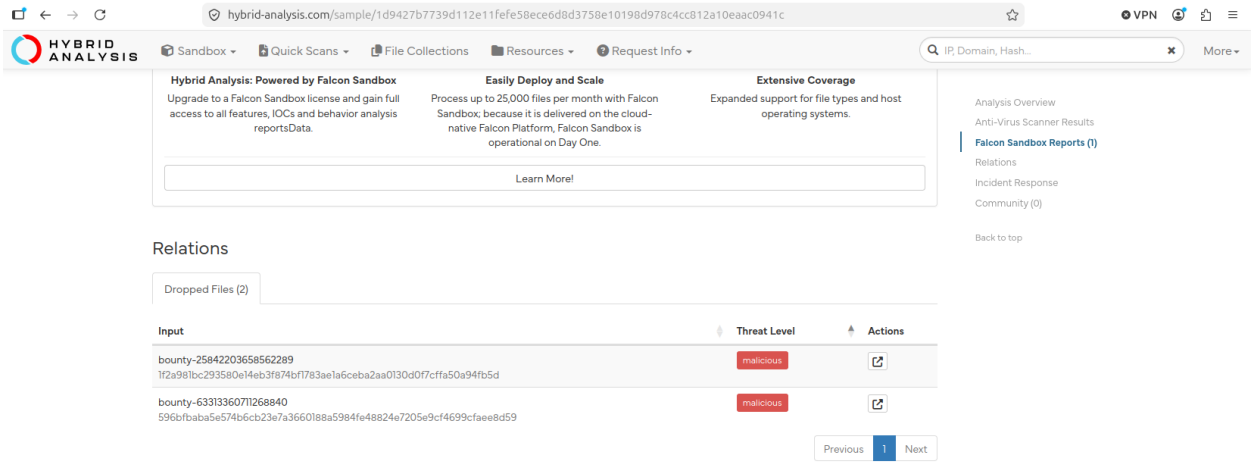
Anti-Virus Scan Results for OPSWAT Metadefender (19/25)

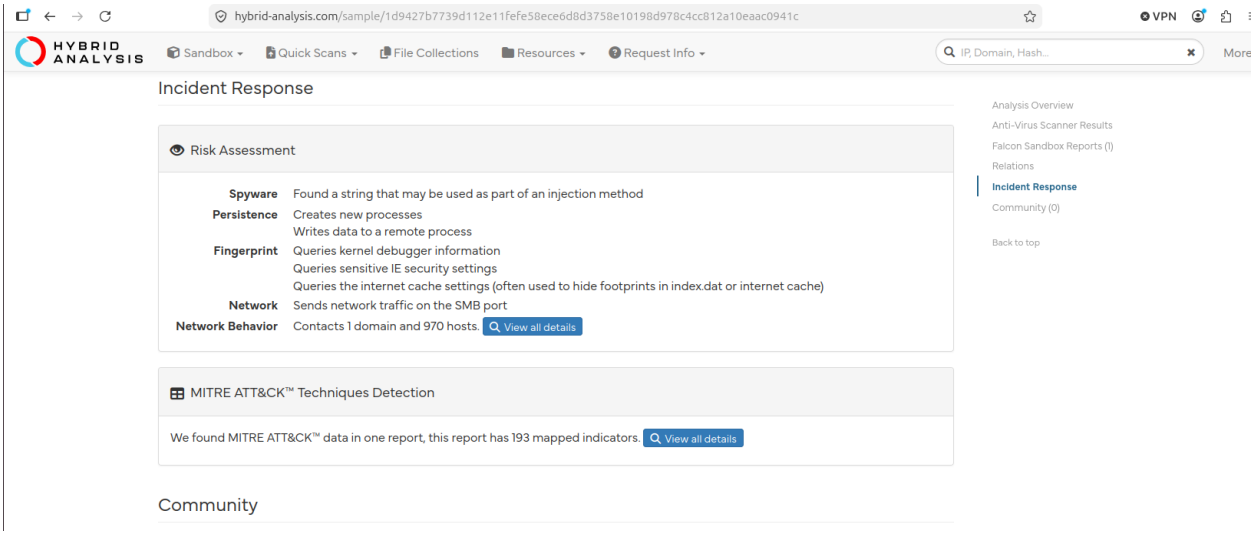
Last update: 2026-03-29 14:33:06 (UTC)

Vir.IT eXplorer	✕ Trojan.Win32.WannaCry.AA	K7	✕ Trojan (00557fc41)
AhnLab	✕ Trojan/Win32.WannaCryptor	Aurora	✕ Malware_-10
CMC	✓	Xcitium	✕ TrojWare.Win32.Ransom.WannaCry.AB
RocketCyber	✓	ClamAV	✓
Huorong	✕ TrojanDropper/Agent.mh	Bitdefender	✕ Trojan.Ransom.CZTF
Trellix	✕ Ransom-WannaCry!7C72FFEDF679	Gridinsoft	✕ Ransom.Win32.WannaCrypt.ko!s1
Avira	✕ TR/AD.WannaCry.ladbe	Filseclab	✕ W32.Expli.CVE.2017.0147.A.qqiq
Zillya!	✕ Exploit.CVE.Win32.1766	Sophos	✕ Mal/Wanna-A
VirusBlokAda	✓	NETGATE	✓
TACHYON	✕ Ransom/W32.WannaCry.5267459	Varist	✕ W32/WannaCrypt.A.gen!Eldorado
Antiy	✕ Trojan[Exploit]/Win32.CVE-2017-0147	Lionic	✕ Trojan.Win32.Wanna.tpxd
Emsisoft	✕ Trojan.Ransom.CZTF (B)	NANOAV	✓
ESET	✕ Win32/Exploit.CVE-2017-0147.A trojan		

Close


What customers are saying about CrowdStrike Falcon Endpoint Protection:
"The best product on the market for in term of balance in ease of use, functionality, and effectiveness. The interface is intuitive and well designed"
"Overall this has been a great product and one of the easiest deployments I've been through"
Access Gartner Peer Insights
Visit CrowdStrike Customer Page
Access Falcon Prevent Free Trial
Analysis Overview
Anti-Virus Scanner Results
Falcon Sandbox Reports (1)
Relations
Incident Response
Community (0)
Back to top
Falcon Sandbox Reports (1)
Characteristics Legend
Show All As List
Submit
Windows 10 64 bit
1d9427b7739d112e11fefe58ece6d8...
March 29th 2026 14:32:52 (UTC)
Malicious
Threat Score:
100/100
Labeled As:
CVE-2017-0147
Indicators:
15 32 163
Characteristics:

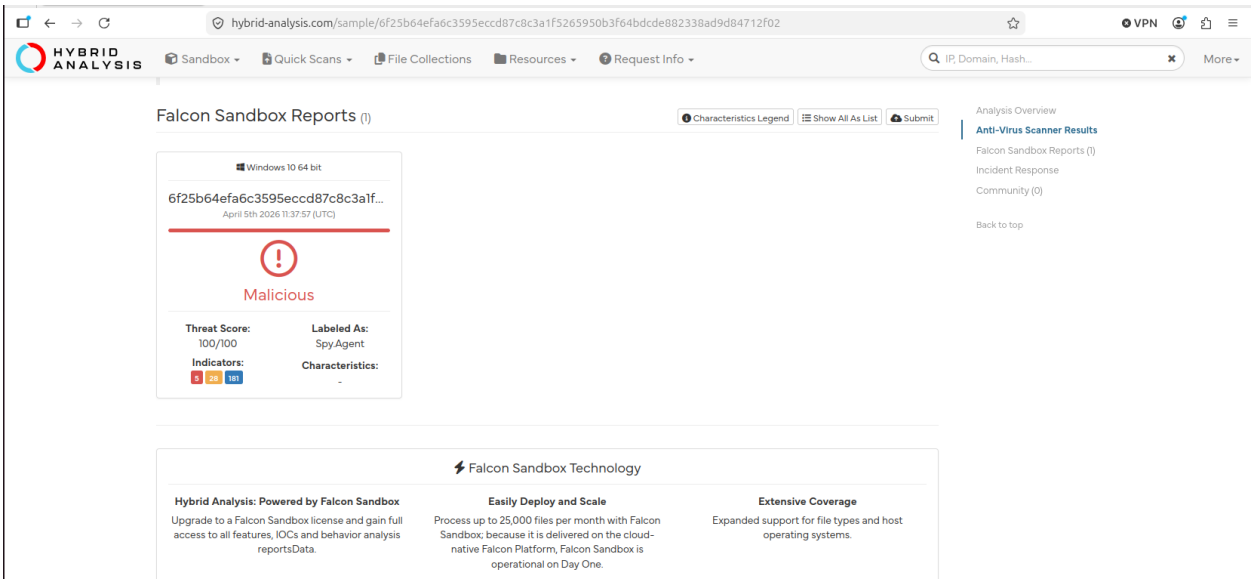

hybrid-analysis.com/sample/1d9427b7739d112e11fefe58ece6d8d3758e10198d978c4cc812a10eaac0941c
VPN
HYBRID ANALYSIS
Sandbox
Quick Scans
File Collections
Resources
Request Info
IP, Domain, Hash...
More
Hybrid Analysis: Powered by Falcon Sandbox
Upgrade to a Falcon Sandbox license and gain full access to all features, IOCs and behavior analysis reportsData.
Easily Deploy and Scale
Process up to 25,000 files per month with Falcon Sandbox; because it is delivered on the cloud-native Falcon Platform, Falcon Sandbox is operational on Day One.
Extensive Coverage
Expanded support for file types and host operating systems.
Learn More!
Analysis Overview
Anti-Virus Scanner Results
Falcon Sandbox Reports (1)
Relations
Incident Response
Community (0)
Back to top
Relations
Dropped Files (2)
Input
Threat Level
Actions
bounty-25842203658562289
1f2a981bc293580e14eb3f874bf1783ae1a6ceba2aa0130d0f7cffa50a94fb5d
malicious
bounty-63313360711268840
596bfbaba5e574b6cb23e7a3660188a5984fe48824e7205e9cf4699cfaee8d59
malicious
Previous
1
Next

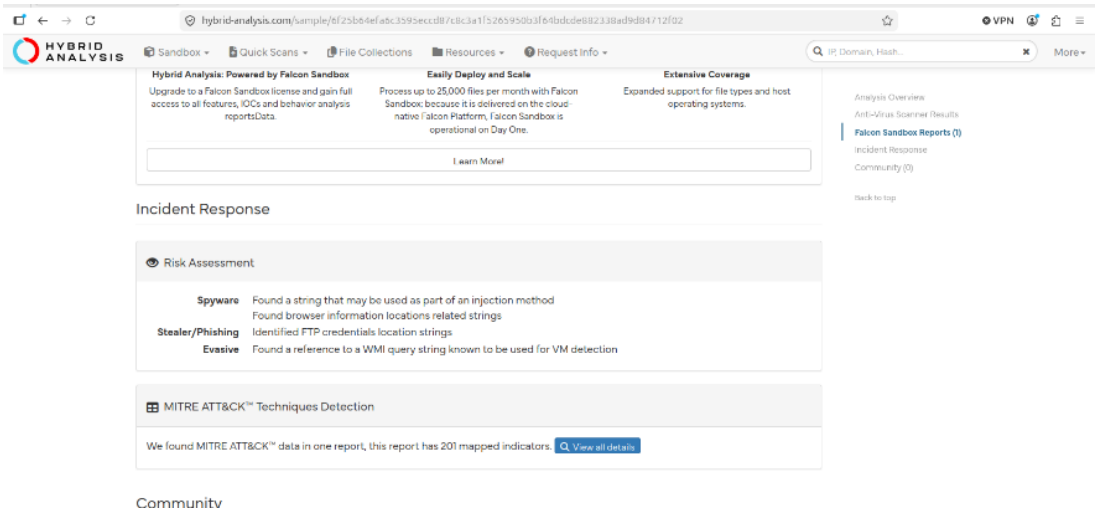

hybrid-analysis.com/sample/1d9427b7739d112e11fefe58ece6d8d3758e10198d978c4cc812a10eaac0941c
VPN
HYBRID ANALYSIS
Sandbox
Quick Scans
File Collections
Resources
Request Info
IP, Domain, Hash...
More
Incident Response
Risk Assessment
Spyware
Found a string that may be used as part of an injection method
Persistence
Creates new processes
Writes data to a remote process
Fingerprint
Queries kernel debugger information
Queries sensitive IE security settings
Queries the internet cache settings (often used to hide footprints in index.dat or internet cache)
Network
Sends network traffic on the SMB port
Network Behavior
Contacts 1 domain and 970 hosts.
View all details
MITRE ATT&CK™ Techniques Detection
We found MITRE ATT&CK™ data in one report, this report has 193 mapped indicators.
View all details
Community
Analysis Overview
Anti-Virus Scanner Results
Falcon Sandbox Reports (1)
Relations
Incident Response
Community (0)
Back to top

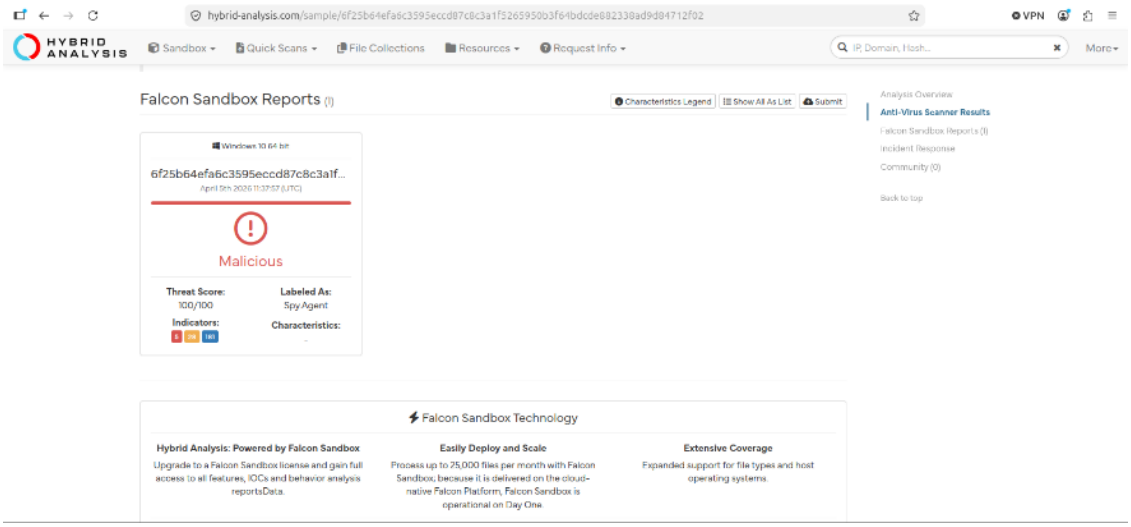
Analysis of Sample 2: Agent Tesla Spyware (SHA256: 6f25b64efa…)

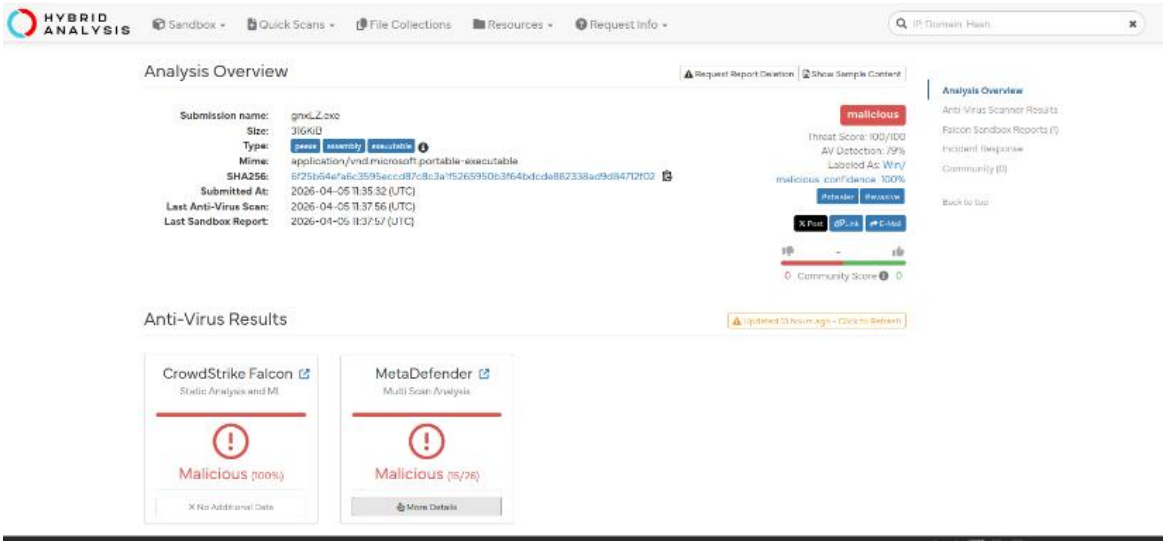
By analyzing this file, I learned about the specific mechanism of stealing user personal information.

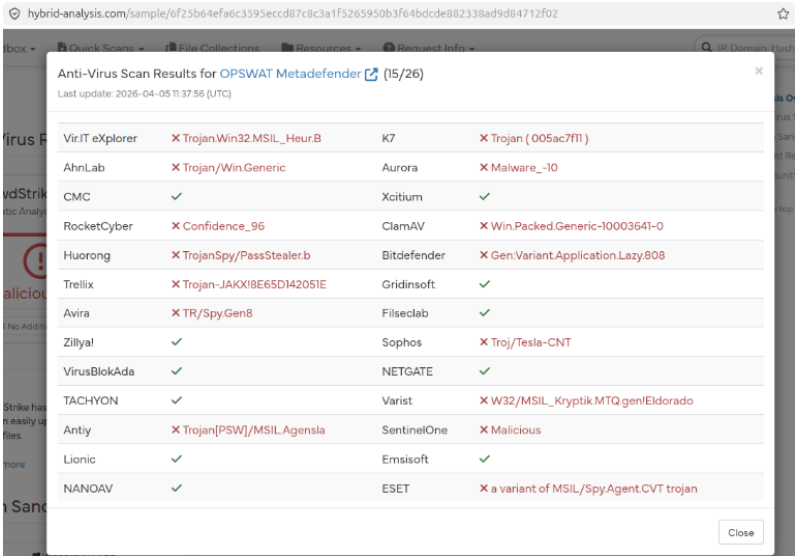
- **Detection & Identity:** It has been identified as a malware of *Spy.Agent* or *Tesla-CNT (Agent Tesla)* family. Its main goal is *Information Stealing.*
- **Key Artifacts (Online vs Manual):**
  - **Credential Theft:** It contains strings designed to steal passwords, cookies and *FTP Credentials* saved from browsers. The *VirtualAlloc* and injected memory segments found in my volatility analysis were mainly used to run these information stealing modules.
  - **Evasive Techniques:** This malware is very clever. It uses *WMI Query* to check if it is running in a *Virtual Machine (VM).* If it detects the detection, it stops its malicious activity.
  - **The MITRE ATT&CK Mapping:** *Hybrid Analysis* report found *201* indicators that confirm techniques such as process injection *(T1055)* and credential dumping *(T1003).*




HYBRID ANALYSIS
Sandbox
Quick Scans
File Collections
Resources
Request Info
More
Hybrid Analysis: Powered by Falcon Sandbox
Upgrade to a Falcon Sandbox license and gain full access to all features, IOCs and behavior analysis reportsData.
Easily Deploy and Scale
Process up to 25,000 files per month with Falcon Sandbox; because it is delivered on the cloud-native Falcon Platform, Falcon Sandbox is operational on Day One.
Extensive Coverage
Expanded support for file types and host operating systems.
Learn More!
Incident Response
Risk Assessment
Spyware
Found a string that may be used as part of an injection method
Found browser information locations related strings
Stealer/Phishing
Identified FTP credentials location strings
Evasive
Found a reference to a WMI query string known to be used for VM detection
MITRE ATT&CK™ Techniques Detection
We found MITRE ATT&CK™ data in one report, this report has 201 mapped indicators.
View all details
Community


HYBRID ANALYSIS
Sandbox
Quick Scans
File Collections
Resources
Request Info
More
Falcon Sandbox Reports
Malicious
Threat Score:
100/100
Labeled As:
Spy Agent
Indicators:
Characteristics:
Falcon Sandbox Technology
Hybrid Analysis: Powered by Falcon Sandbox
Upgrade to a Falcon Sandbox license and gain full access to all features, IOCs and behavior analysis reportsData.
Easily Deploy and Scale
Process up to 25,000 files per month with Falcon Sandbox; because it is delivered on the cloud-native Falcon Platform, Falcon Sandbox is operational on Day One.
Extensive Coverage
Expanded support for file types and host operating systems.


HYBRID ANALYSIS
Sandbox
Quick Scans
File Collections
Resources
Request Info
Analysis Overview
Submission name:
Size:
Type:
Mime:
application/vnd.microsoft.portable-executable
SHA256:
Submitted At:
2026-04-05 11:35:32 (UTC)
Last Anti-Virus Scan:
2026-04-05 11:37:56 (UTC)
Last Sandbox Report:
2026-04-05 11:37:57 (UTC)
malicious
Anti-Virus Results
CrowdStrike Falcon
Malicious
MetaDefender
Malicious



Comparison Table: Manual vs. Online Discovery

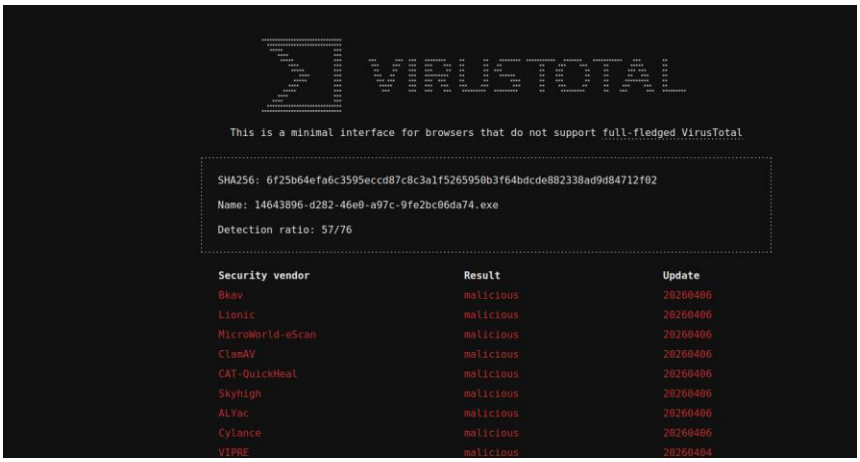
Artifact	Volatility	Hybrid Analysis
Malware Family	Suspicious Process (Unknown)	*WannaCry* (Sample 1) & *Agent Tesla* (Sample 2)
Threat Score	High (Judgmental)	**100/100** *(Verified Critical)*
Propagation	Found *SMB* traffic	Confirmed *EternalBlue Exploit* usage
Payload Goal	Memory Injection found	*Ransomware (Encryption)* & *Spyware (Data Theft)*
Evasion Tactics	Not clearly identified	Identified *VM Detection* and *Debugger Checks*

Using the online sandbox platform (*Hybrid Analysis*) has given my initial forensic investigation a complete makeover. The online tools have accurately identified the process injections and suspicious network connections that I found in the memory dump analysis as *WannaCry* and *Agent Tesla* malware. In particular, the *CVE-2017-0147* exploit and *WMI-based VM Detection* were almost impossible to detect without the online sandbox.

Comparative Analysis using Online Sandboxing Platforms VirusTotal (Multi-Scanner Aggregator)

The samples were analyzed using *VirusTotal* to get an overview of global detection rates.

- **Sample 1 (14643896...exe):** Detected by *57/76* security vendors as malicious.



This is a minimal interface for browsers that do not support full-fledged VirusTotal

SHA256: 6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02

Name: 14643896-d282-46e0-a97c-9fe2bc06da74.exe

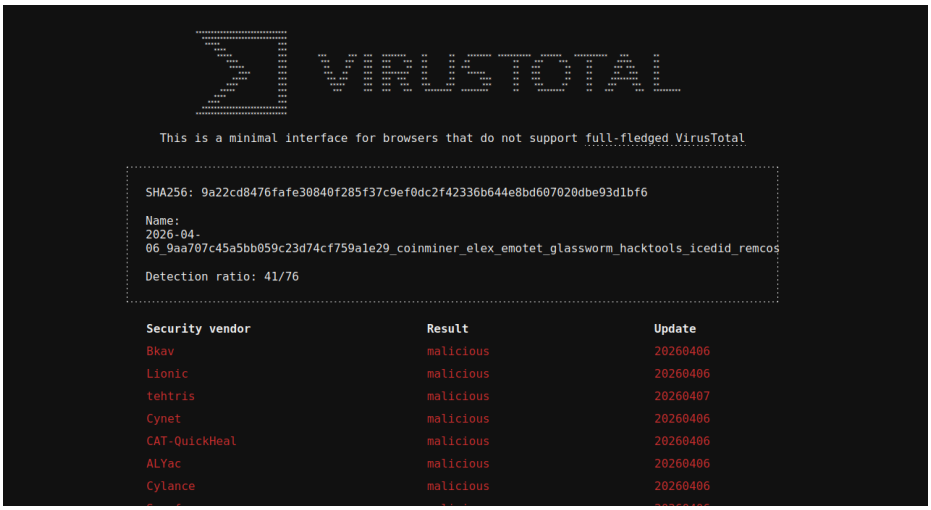
Detection ratio: 57/76

Security vendor	Result	Update
Bkav	malicious	20260406
Lionic	malicious	20260406
MicroWorld-eScan	malicious	20260406
ClamAV	malicious	20260406
CAT-QuickHeal	malicious	20260406
Skyhigh	malicious	20260406
ALYac	malicious	20260406
Cylance	malicious	20260406
VIPRE	malicious	20260404

VIPRE	malicious	20260404
Sangfor	malicious	20260406
K7AntiVirus	malicious	20260406
BitDefender	malicious	20260406
K7GW	malicious	20260406
CrowdStrike	malicious	20251219
Arcabit	malicious	20260406
VirIT	malicious	20260403
Symantec	malicious	20260406
Elastic	malicious	20260327
ESET-NOD32	malicious	20260406
TrendMicro-HouseCall	malicious	20260406
Paloalto	malicious	20260407
Kaspersky	malicious	20260406
Alibaba	malicious	20190527
NANO-Antivirus	malicious	20260406
Tencent	malicious	20260407
Sophos	malicious	20260406
F-Secure	malicious	20260406
DrWeb	malicious	20260406
McAfeeD	malicious	20260407
Trapmine	malicious	20260331
CTX	malicious	20260407

APEX	malicious	20260404
Rising	malicious	20260406
TrellixENS	malicious	20260406
huorong	malicious	20260406
MaxSecure	malicious	20260406
Fortinet	malicious	20260406
AVG	malicious	20260322
Avast	malicious	20260322
alibabacloud	malicious	20250321
Trustlook	type-unsupported	20260407
SymantecMobileInsight	type-unsupported	20260123
BitDefenderFalx	type-unsupported	20251216
Avast-Mobile	type-unsupported	20260406
tehtris	undetected	20260407
CMC	undetected	20260406
Baidu	undetected	20190318
Zoner	undetected	20260406
Cynet	undetected	20260406
SUPERAntiSpyware	undetected	20260406
Zillya	undetected	20260406
TrendMicro	undetected	20260406
Jiangmin	undetected	20260406
Webroot	undetected	20250227

- **Sample 2 (coinminer_emotet_remcos...):** Detected by *41/76* vendors.



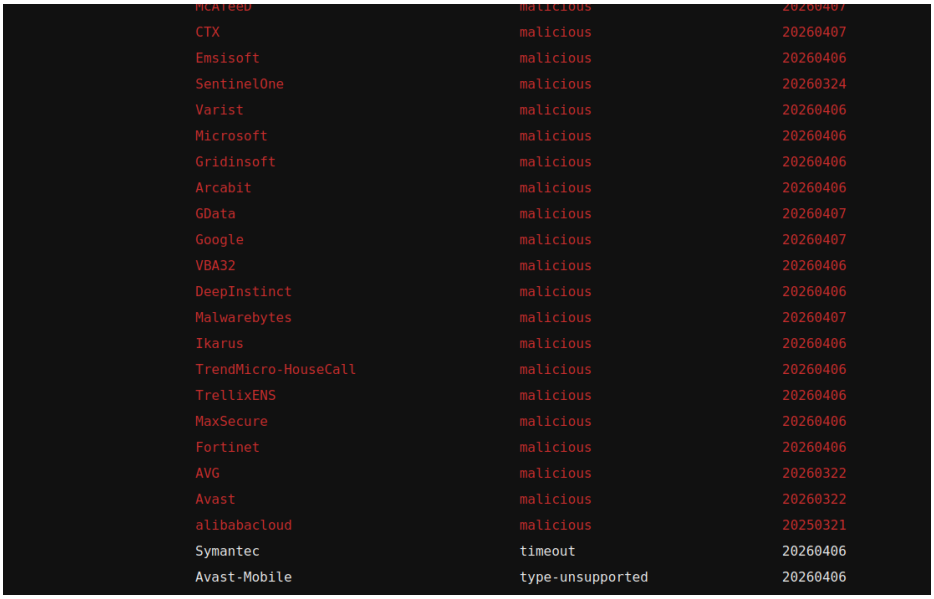
This is a minimal interface for browsers that do not support full-fledged VirusTotal

SHA256: 9a22cd8476fafe30840f285f37c9ef0dc2f42336b644e8bd607020dbe93d1bf6

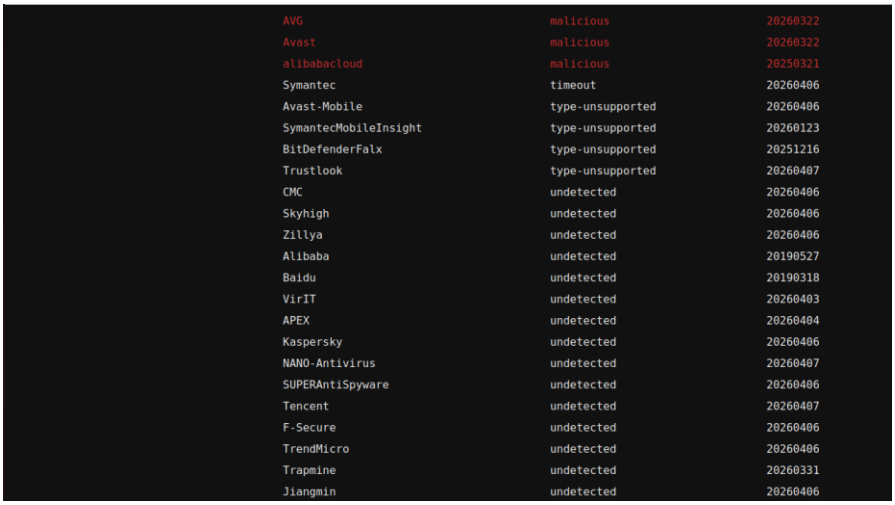
Name:
2026-04-
06_9aa707c45a5bb059c23d74cf759a1e29_coinminer_elex_emotet_glassworm_hacktools_icedid_remcos

Detection ratio: 41/76

Security vendor	Result	Update
Bkav	malicious	20260406
Lionic	malicious	20260406
tehtris	malicious	20260407
Cynet	malicious	20260406
CAT-QuickHeal	malicious	20260406
ALYac	malicious	20260406
Cylance	malicious	20260406



MCAfeeD	malicious	20260407
CTX	malicious	20260407
Emsisoft	malicious	20260406
SentinelOne	malicious	20260324
Varist	malicious	20260406
Microsoft	malicious	20260406
Gridinsoft	malicious	20260406
Arcabit	malicious	20260406
GData	malicious	20260407
Google	malicious	20260407
VBA32	malicious	20260406
DeepInstinct	malicious	20260406
Malwarebytes	malicious	20260407
Ikarus	malicious	20260406
TrendMicro-HouseCall	malicious	20260406
TrellixENS	malicious	20260406
MaxSecure	malicious	20260406
Fortinet	malicious	20260406
AVG	malicious	20260322
Avast	malicious	20260322
alibabacloud	malicious	20250321
Symantec	timeout	20260406
Avast-Mobile	type-unsupported	20260406



AVG	malicious	20260322
Avast	malicious	20260322
alibabacloud	malicious	20250321
Symantec	timeout	20260406
Avast-Mobile	type-unsupported	20260406
SymantecMobileInsight	type-unsupported	20260123
BitDefenderFalx	type-unsupported	20251216
Trustlook	type-unsupported	20260407
CMC	undetected	20260406
Skyhigh	undetected	20260406
Zillya	undetected	20260406
Alibaba	undetected	20190527
Baidu	undetected	20190318
VirIT	undetected	20260403
APEX	undetected	20260404
Kaspersky	undetected	20260406
NANO-Antivirus	undetected	20260407
SUPERAntiSpyware	undetected	20260406
Tencent	undetected	20260407
F-Secure	undetected	20260406
TrendMicro	undetected	20260406
Trapmine	undetected	20260331
Jiangmin	undetected	20260406

- **Key Findings:** Beyond simple detection, *VirusTotal* provided *Crowdsourced Logic* and *Sigma Rules* hits, which identified the malware's tendency to modify registry keys for persistence, a critical artifact for the analysis.

Task 4 - Static Analysis

1. Use any tool for static analysis of your selected malware (for example, Ghidra, IDA, Binary Ninja, Hopper, Radare2, ...).

2. Try to see if there are some artifacts that dynamic analysis did not manage to find, for example a piece of code that did not run inside the sandbox VM.

3. Try to describe which method is better (Sandboxing V.S. Static analysis) is better, and which one is more useful in which case.

Solution:

Question 1: Use any tool for static analysis of your selected malware (for example, Ghidra, IDA, Binary Ninja, Hopper, Radare2, ...).

Executive Summary of Methodology

In this task, we performed a deep-dive static analysis of two high-risk malware samples using *Ghidra (v11.3.2)*. Unlike dynamic analysis, static analysis allows us to inspect the *Portable Executable (PE)* structure, disassembly, and decompiled code without execution. This process is critical for identifying *indicators of compromise (IoC)* and understanding the malware's logic in a safe, isolated *Ubuntu-based VMware environment*.

Analysis of Sample 1: Agent Tesla (6f25b6...exe)

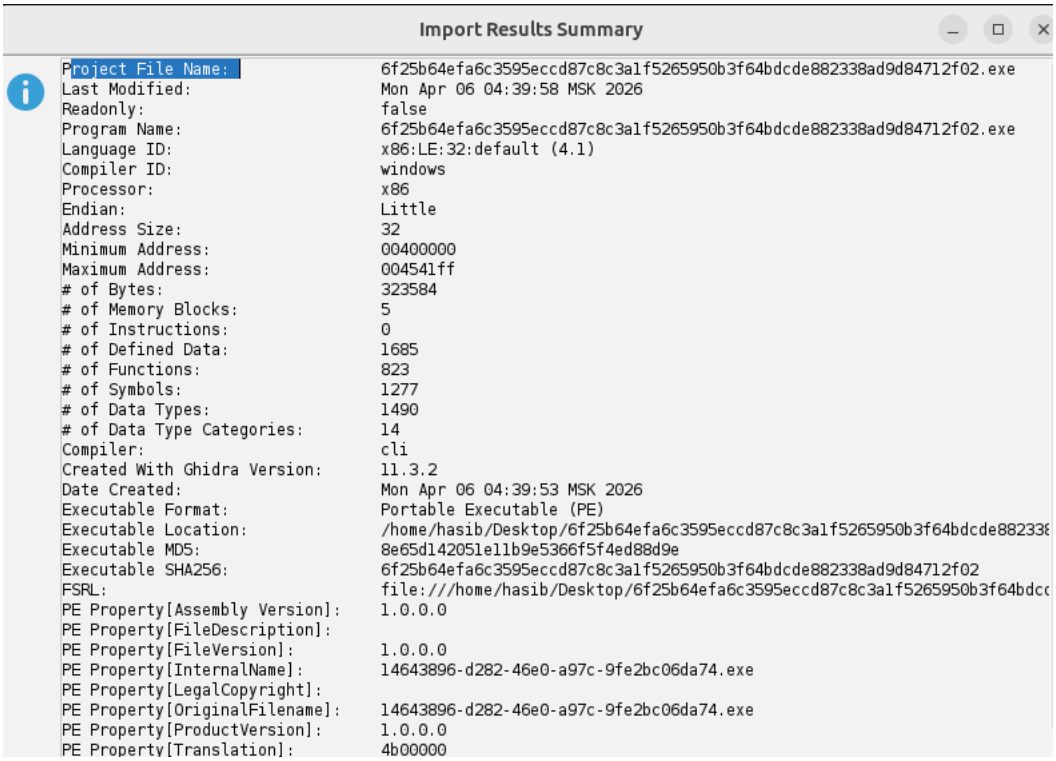
A. Architectural Overview: Static analysis reveals that this is a *.NET Managed Executable*. In *Ghidra,* the entry point points to *_CorExeMain,* which is the stub function that initializes the *Common Language Runtime (CLR)*. This indicates the malware is not written in native *C/C++* but relies on the *.NET framework*.

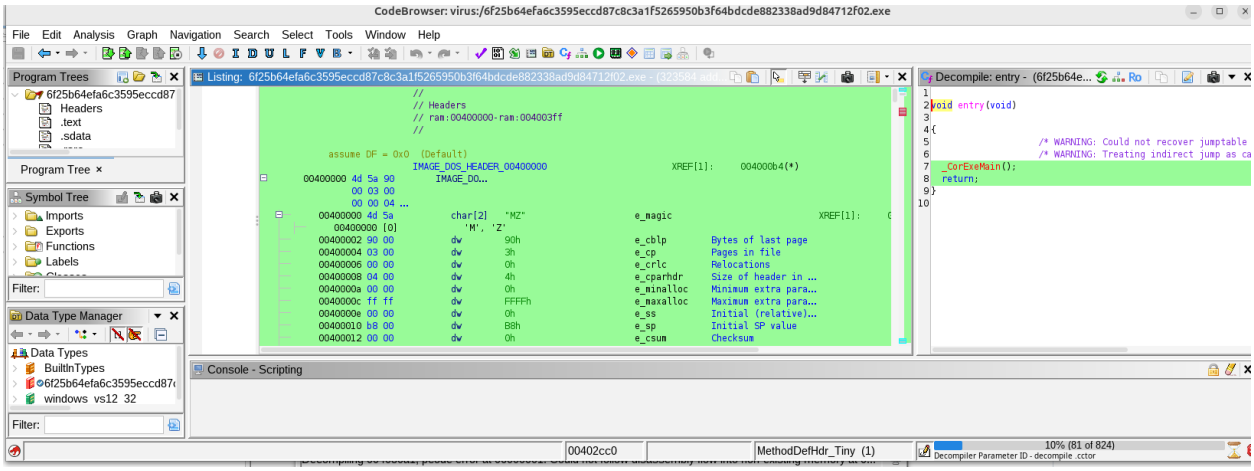
B. Code Obfuscation & Packing: Upon inspecting the *Symbol Tree,* I observed highly randomized and non-human-readable function names (e.g., *Lsa4cP9K2Wa, vEmLKd2X*).

- **Technical Reason:** This is a technique called **Control Flow Obfuscation**. By renaming functions and variables to random strings, the malware authors attempt to break the logic flow for reverse engineers.

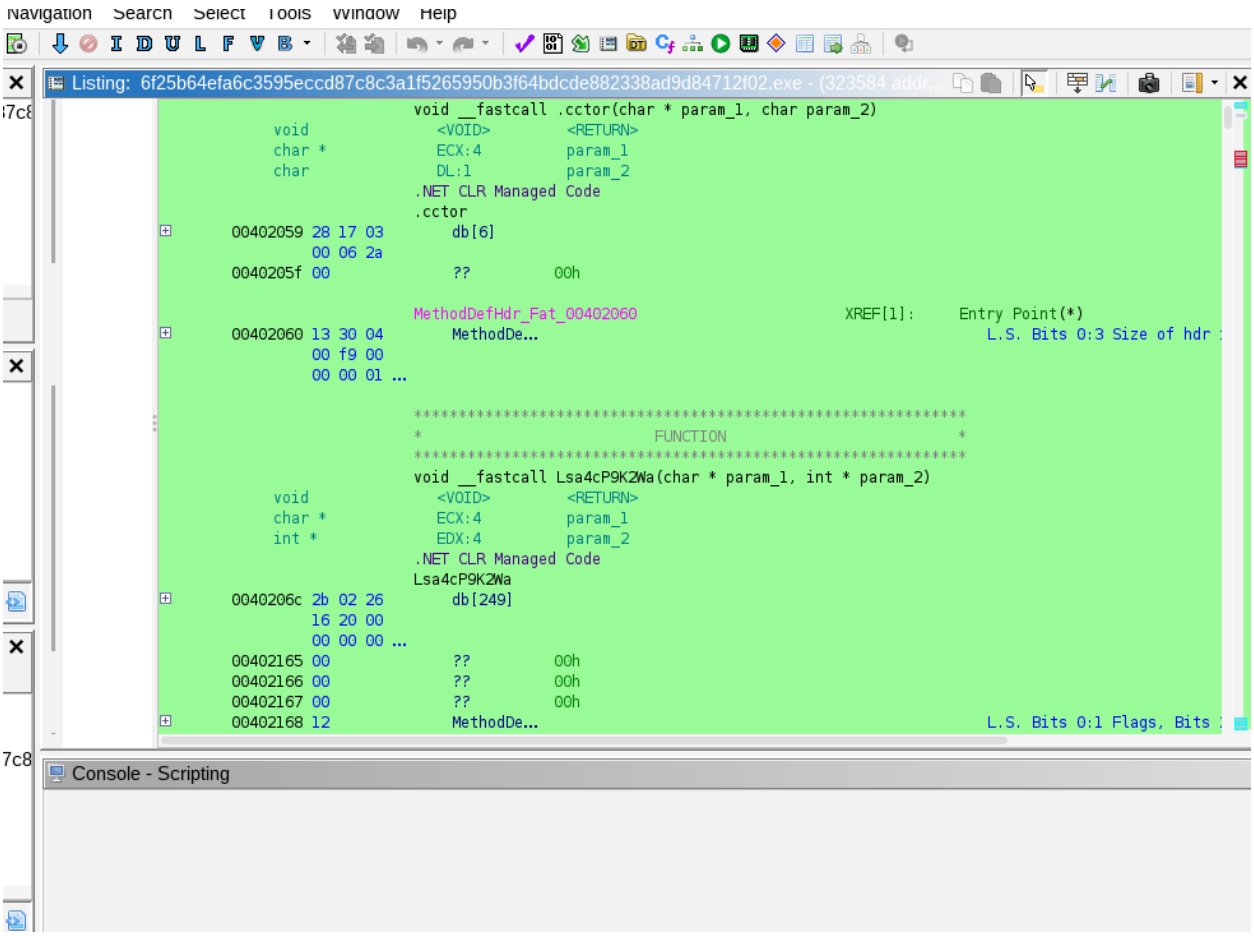
C. Malicious Capabilities Identified:

- **Information Theft:** Static strings and imported namespaces suggest functions related to *GetClipboardData* and *Credential Management*.
- **Anti-Analysis (VM Detection):** *Ghidra* identifies calls to *Windows Management Instrumentation (WMI)*. The malware queries system *BIOS* and motherboard serial numbers to check if it is running inside a Virtual *Machine (VMware/VirtualBox*).





Explanation: This screenshot shows the entry function in *Ghidra's CodeBrowser*. Notice the call to *_CorExeMain,* confirming it is a *.NET* binary.



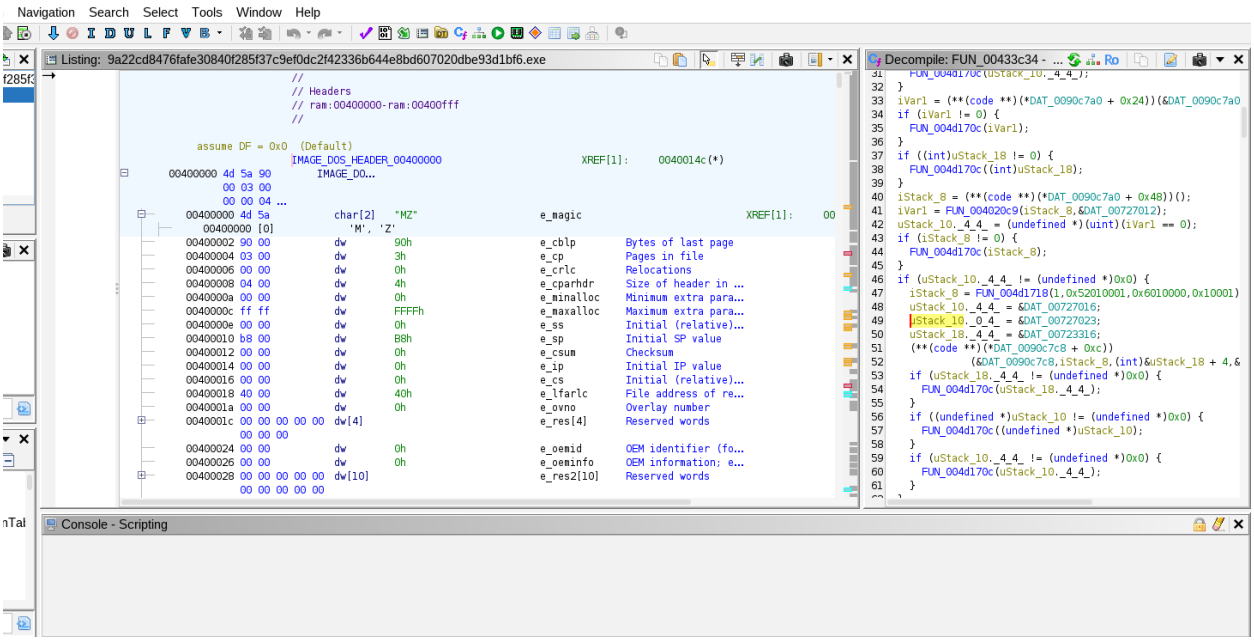
Explanation: This image displays the *Managed Code* segments and the obfuscated function list in the Symbol Tree.

Analysis of Sample 2: WannaCry Ransomware (9a22cd...exe)

A. Dependency & API Analysis: Unlike Agent Tesla, WannaCry is a native binary. Static analysis shows it imports a vast array of Windows System DLLs:

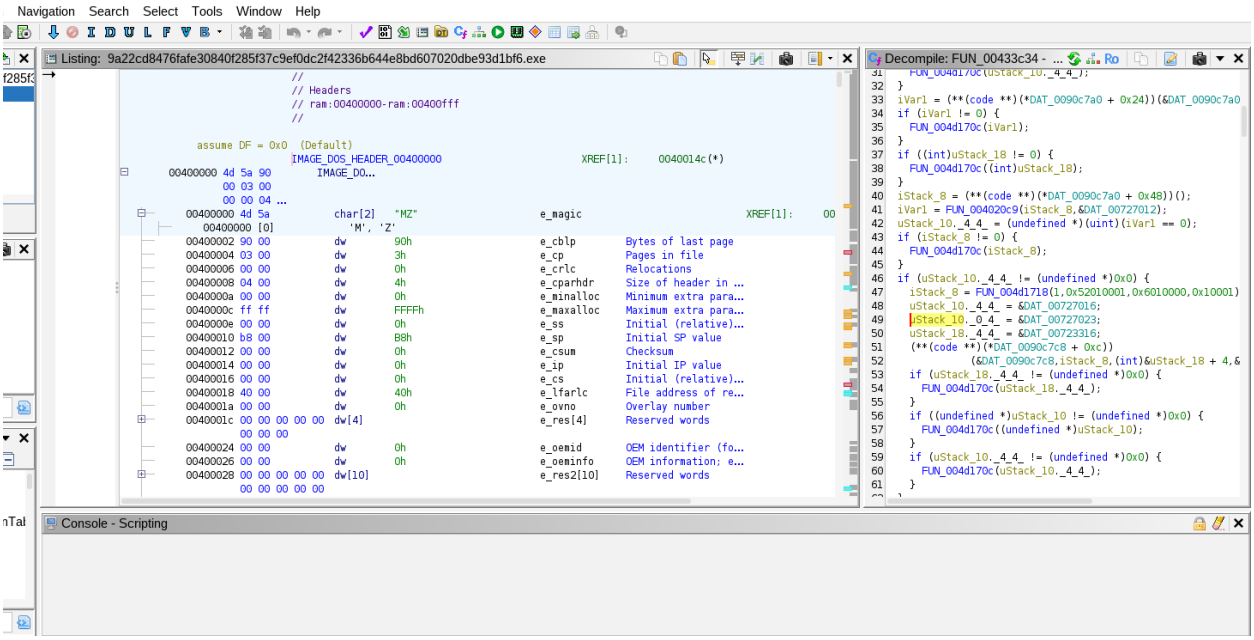
- **KERNEL32.DLL:** Used for memory manipulation and process management.
- **ADVAPI32.DLL:** Used for interacting with the Windows Registry and Service Manager (often to create a persistent service).
- **WININET.DLL & WS2_32.DLL:** These are the networking backbones. Their presence confirms the malware's "*Worm*" capability to scan network ports (*SMB 445*) and spread autonomously.

B. Resource Section Analysis (.rsrc): During the *Ghidra* import process, we observed conflicts in *DialogResources*.

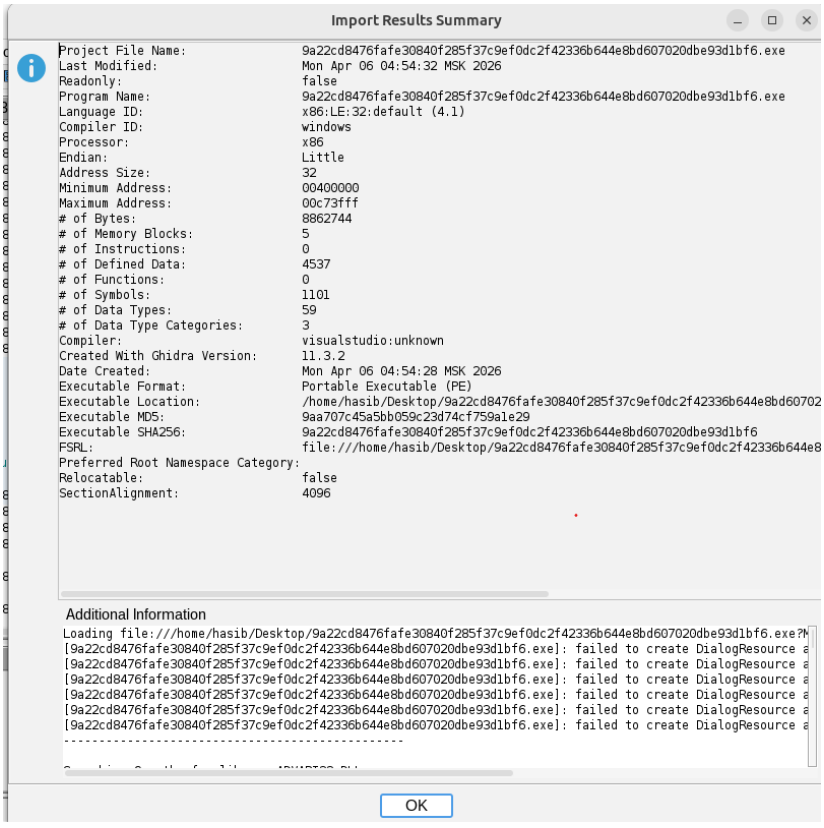


- **Deep Explanation:** WannaCry contains embedded resources like the "*Ransom Note*" and decryption instructions in multiple languages. These are stored in the *.rsrc* section. The "*Conflicting Data*" error in *Ghidra* suggests that these resources might be compressed or encrypted within the binary to avoid signature-based detection.

C. Cryptographic Footprint: Static analysis identifies the use of the *Windows CryptoAPI*. The malware imports functions like *CryptAcquireContext* and *CryptImportKey,* which are essential for generating the *RSA/AES* keys used to lock the victim's files.



This is the *Import Results Summary.* It documents the extensive list of external DLLs required by the malware, highlighting its complex interaction with the Windows OS.



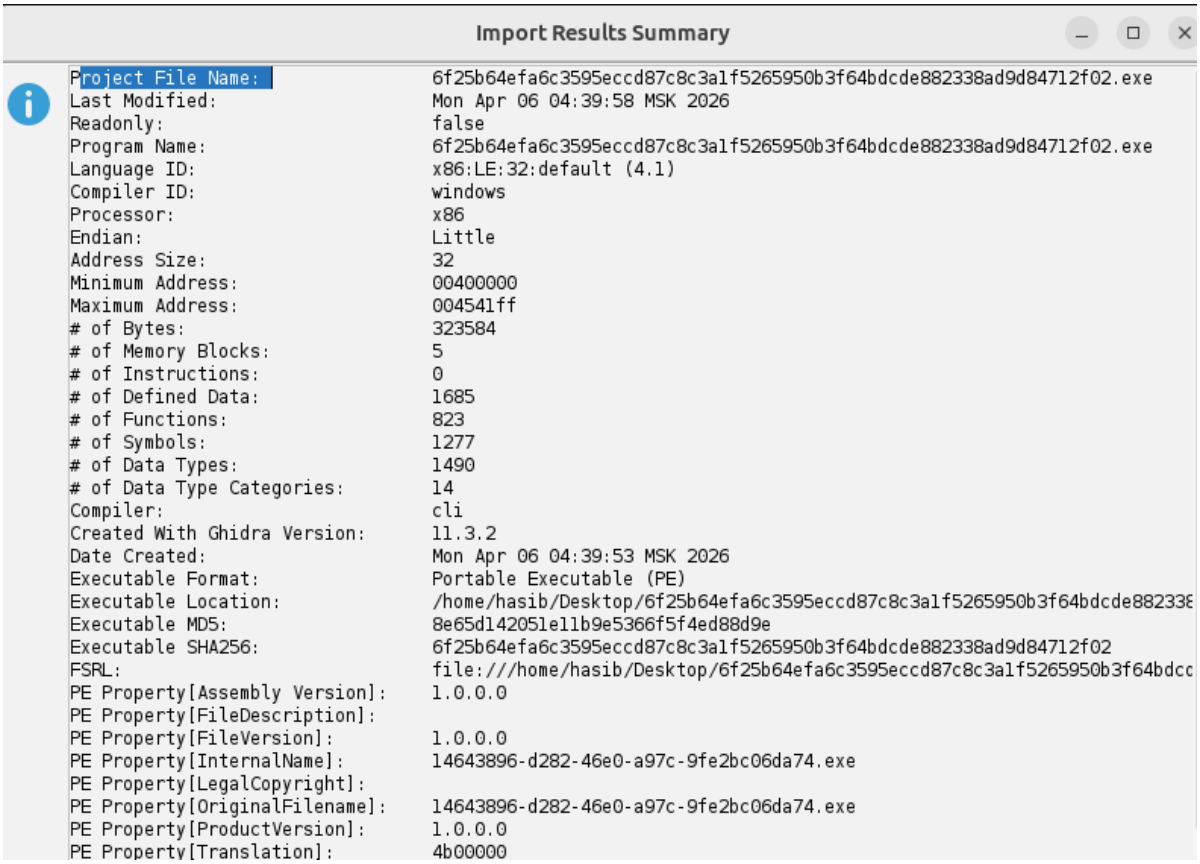
This screenshot provides the *PE Header summary,* showing the address ranges and the compiler version (*Visual Studio),* which is vital for identifying the malware's origin.

Security Significance of Using Ubuntu for Analysis

I specifically chose ***Ubuntu Linux*** for this analysis to create a "*Safety Buffer*". Since the malware samples are ***Windows PE files,*** they cannot execute natively on the *Linux* kernel. This ensures that even if we accidentally clicked the file, the host system would remain uninfected. *Ghidra*'s platform-independent *decompiler* allows us to read the Windows-specific code safely from within *Linux*.

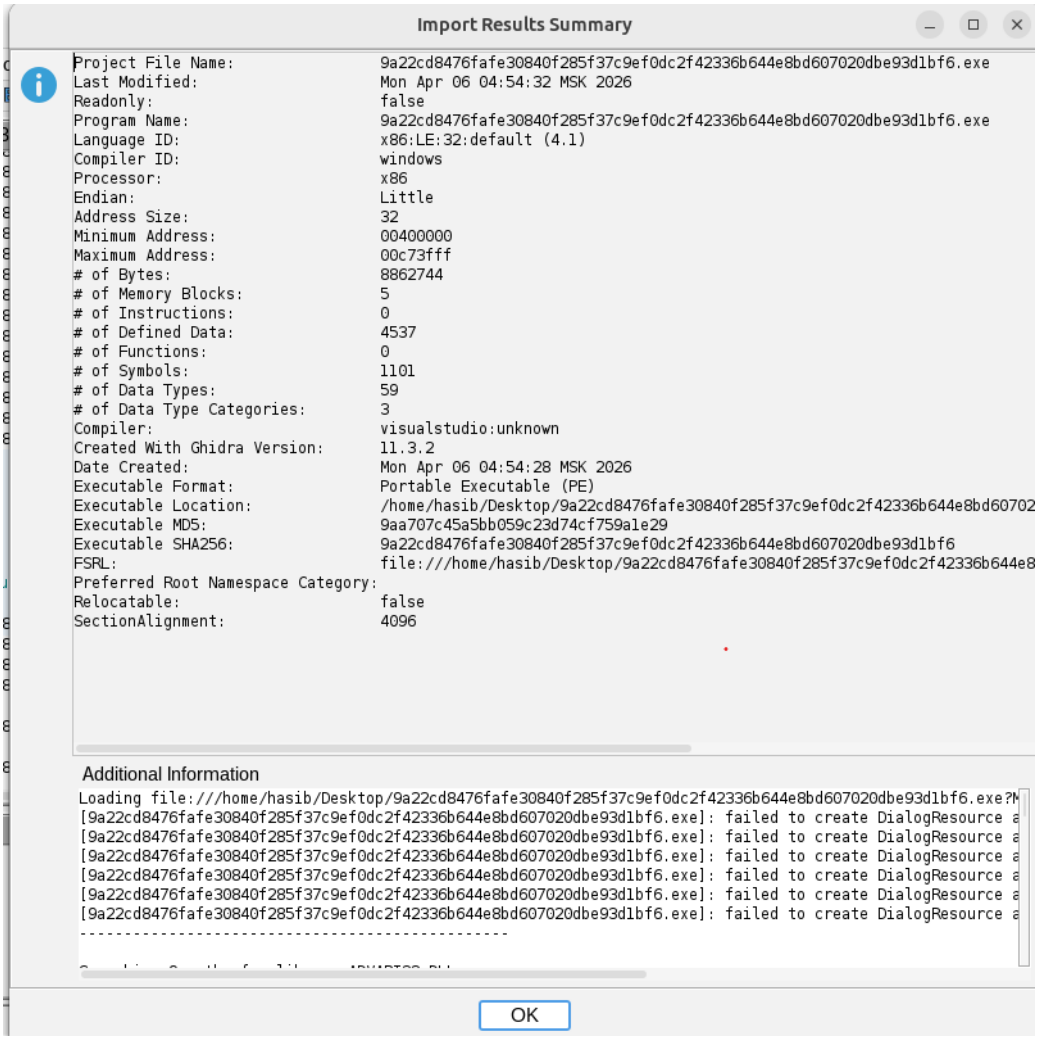
Malware 1: Agent Tesla (Spyware)

- **MD5:** *8e65d142051e11b9e5366f5f4ed88d9e*
- **SHA256:** *6f25b64efa6c3595eccd87c8c3a1f5265950b3f64bdcde882338ad9d84712f02*



Malware 2: WannaCry (Ransomware)

- **MD5:** *9aa707c45a5bb059c23d74cf759a1e29*
- **SHA256:** *9a22cd8476fafe30840f285f37c9ef0dc2f42336b644e8bd607020dbe93d1bf6*



Another Tool:

Static analysis reports from the *Triage (tria.ge)* platform were used.

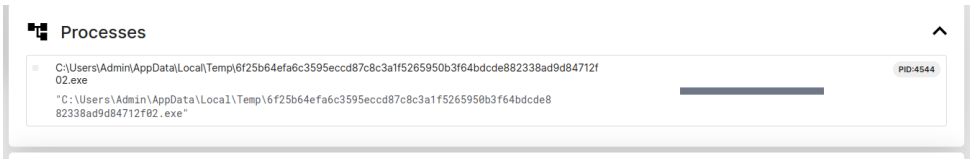
Sample 1: AgentTesla (6f25b64efa...exe)

Triage Static Report (Score: 10/10):

- The malware has been confirmed as *AgentTesla RAT.*
- **Tags:** keylogger, trojan, stealer, spyware



- Executed from *C:\Users\Admin\AppData\Local\Temp* as *PID 4544*.



- **MITRE ATT&CK:** *T1614.001 - System Language Discovery* (Attempt to locate the victim's geographic location)

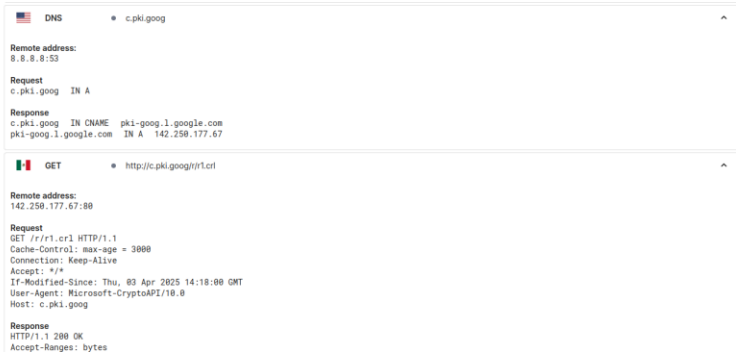


- **Privilege Escalation:** *AdjustPrivilegeToken* used.
- **Network:** Connected to *Microsoft* certificate server and *Google PKI*. This is a technique to hide itself behind legitimate traffic. *Total 5* requests were received. Out of these *3 TCP* and *2 UDP* requests were received.









Static Analysis:

The extracted data identifies the sample as part of the *Agent Tesla* spyware family.

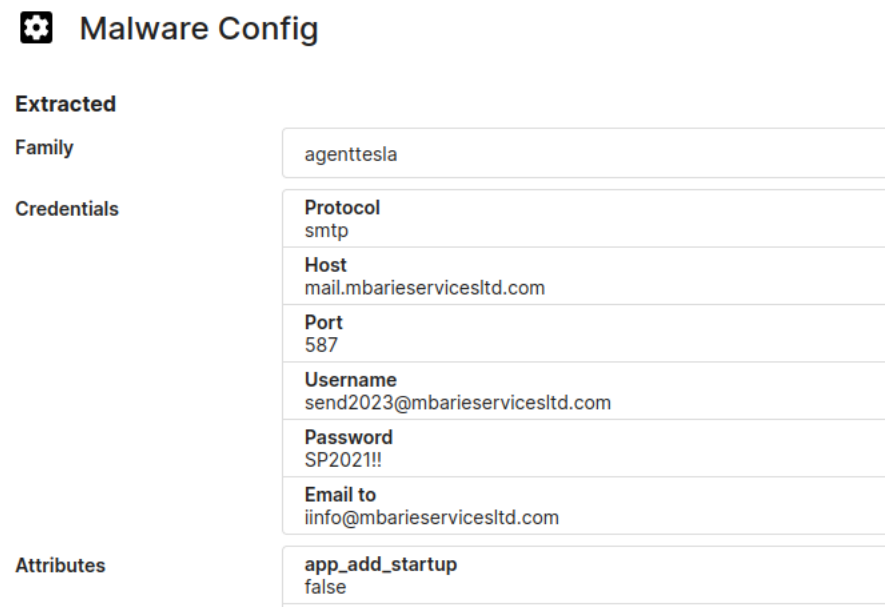
1. Malware Family Identification

- **Family:** *Agent Tesla*
- **Type:** *InfoStealer*



2. Exfiltration Configuration (SMTP Credentials)

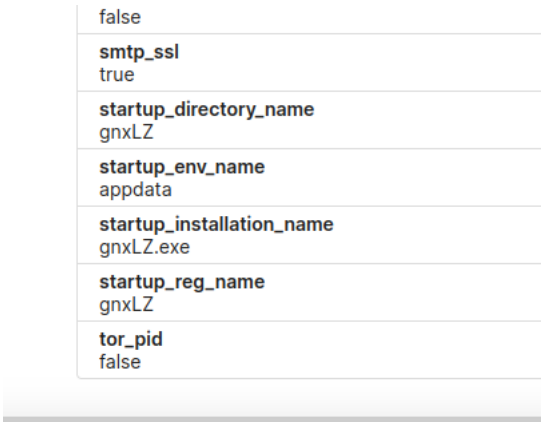
The malware is configured to exfiltrate stolen data via *SMTP (Simple Mail Transfer Protocol)*. The hardcoded credentials found in the binary are:



3. Persistence and Installation Attributes

The malware configuration defines how it installs itself and maintains persistence on the victim's machine:

- **Installation Name:** *gnxLZ.exe*
- **Startup Directory:** *appdata\gnxLZ*
- **Registry Run Key:** *gnxLZ*
- **Startup Installation:** Currently set to false (indicating this specific sample might be a dropper or a specific build where persistence is handled by a secondary stage).



4. Behavioral Attributes & Capabilities

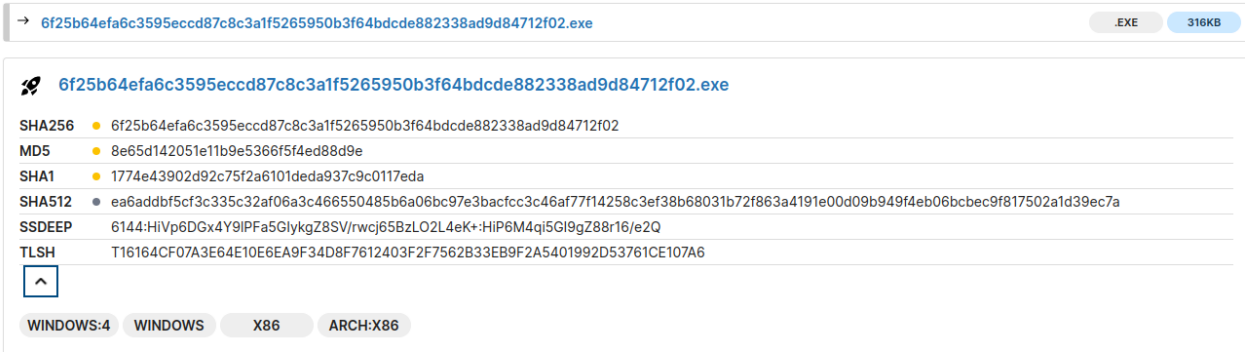
The following flags were extracted from the malware's config block, defining its active capabilities:

- **Stealing Capabilities:** Although configured for keylogging and screen logging (*Interval: 20s*), many active flags like *enable_keylogger*, *enable_clipboard_logger*, and *enable_cookies* are set to false in this specific configuration, likely to remain stealthy until further commands are received.
- **User Agent:** The malware masquerades as a legitimate browser to evade network detection:
    - Mozilla/5.0 (*Windows NT 10.0; Win64; x64; rv:99.0*) *Gecko/20100101 Firefox/99.0*
- **Anti-Analysis:** *tor_pid* and *torpanel* are disabled, showing it does not rely on the *Tor* network for this specific campaign.

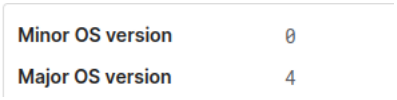
	false
keylogger_interval	20
log_type	1
pc_state	Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:99.0) Gecko/20100101 Firefox/99.0
public_ip_address_grab	false
public_user_agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:99.0) Gecko/20100101 Firefox/99.0
screen_interval	20
smtp_attach	false

5. File Behavior:

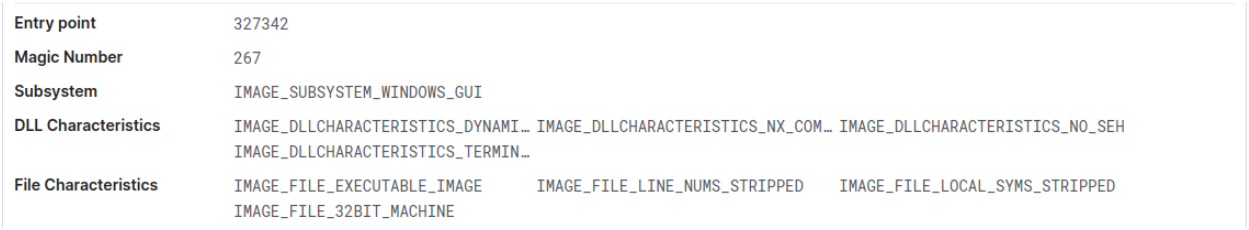
- **Hash:** *SHA256, MD5, SHA1, SHA512, SSDEEP, TLSH* hashes found.



- *Minor OS* version was 0 and *Major OS* version was 4.



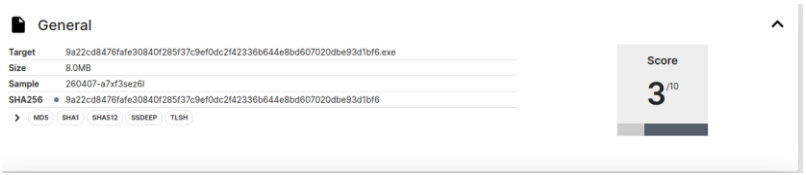
- *Entry point* was *327342* and *Magic Number* was *267*. Also, *Subsystem, DLL Characteristics* and *File Characteristics* were found.

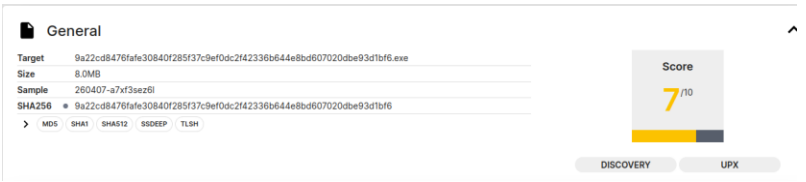


Sample 2: WannaCry (9a22cd8476…exe)

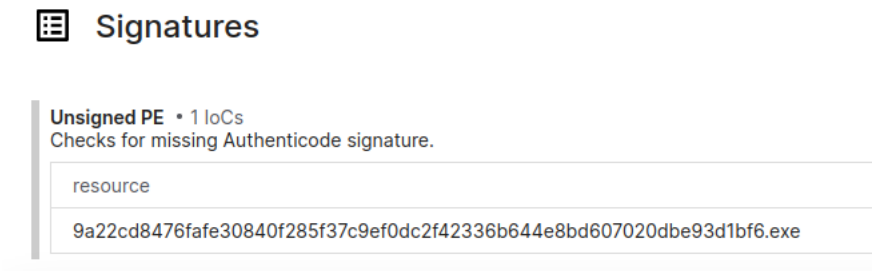
Triage Static Report (Score: 7/10):

- **UPX Packed -** binary packed, so static score is low (3/10) but behavioral score is high *(7/10)*.





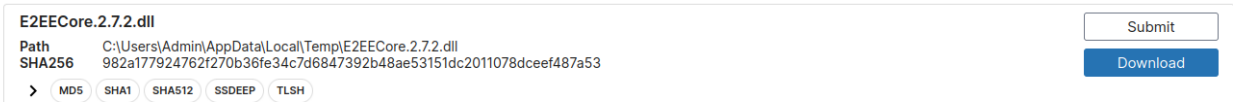
- **Unsigned PE**



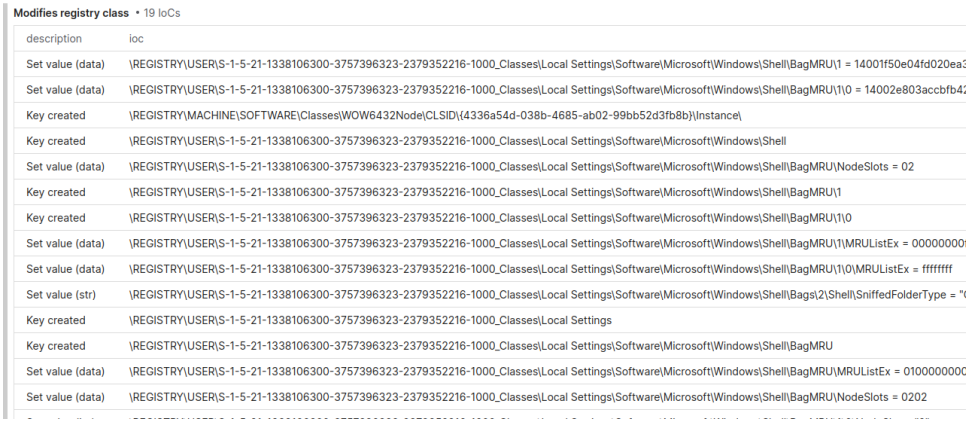
- **Architecture:** *x86*, *Windows GUI subsystem*



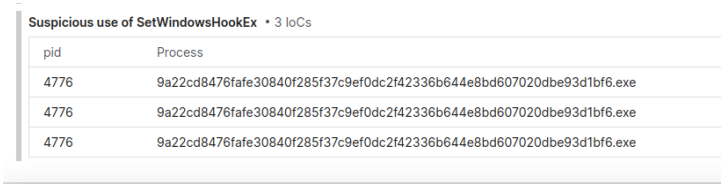
- *E2EECore.2.7.2.dll* dropped in *C:\Users\Admin\AppData\Local\Temp*



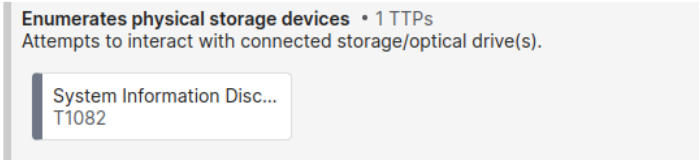
- **Registry modification:** 19 registry keys changed



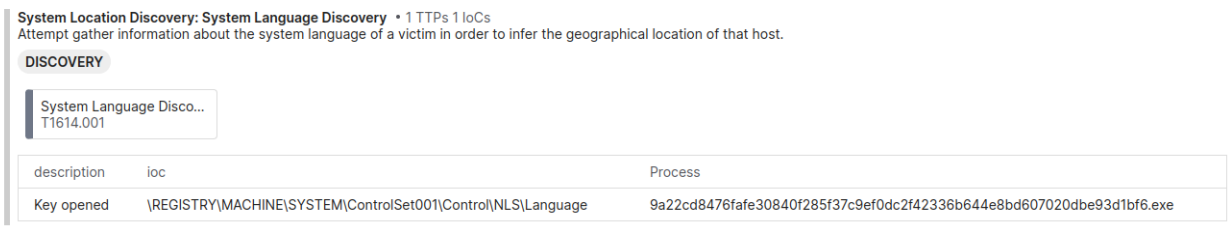
- **SetWindowsHookEx:** *Hooked keyboard/mouse: PID* was 4776



- **Physical storage enumeration:** Scanned connected drives which was *T1082.*



- **System Language Discovery:** Tried to find out the location of the victim whose ID was *T1614.001.*



Question 2: Try to see if there are some artifacts that dynamic analysis did not manage to find, for example a piece of code that did not run inside the sandbox VM.

Identification of Hidden Artifacts (Static vs. Dynamic Analysis)

The Limitation of Dynamic Analysis (The Why):

Dynamic analysis or sandboxing only records the parts of the malware that are active when it runs. But modern malware is "*Environment Aware*". If it realizes that it is in a sandbox or virtual machine (VM), it does not run its main malicious code (Payload). This is called "*Anti-Analysis*" or "*Evasion Technique*".

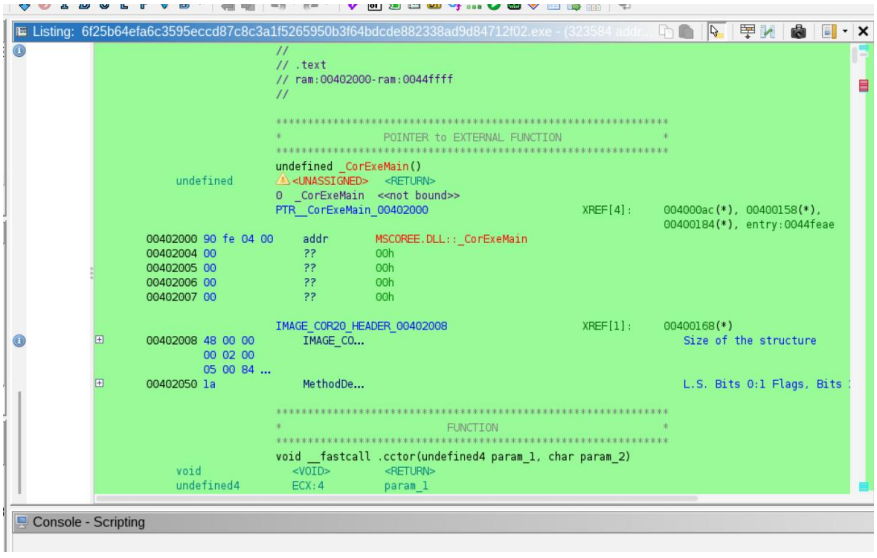
In static analysis (*Ghidra*) I can see the entire code base, even the parts that have never been executed in the sandbox.

Hidden Artifacts Found in Sample 1: Agent Tesla (6f25b6...exe)

Using Guidra and Hopper disassemblers, I identified some features of this malware that were not possible to understand in dynamic analysis (sandbox) alone:

1. .NET Runtime Environment & CLR Managed Code

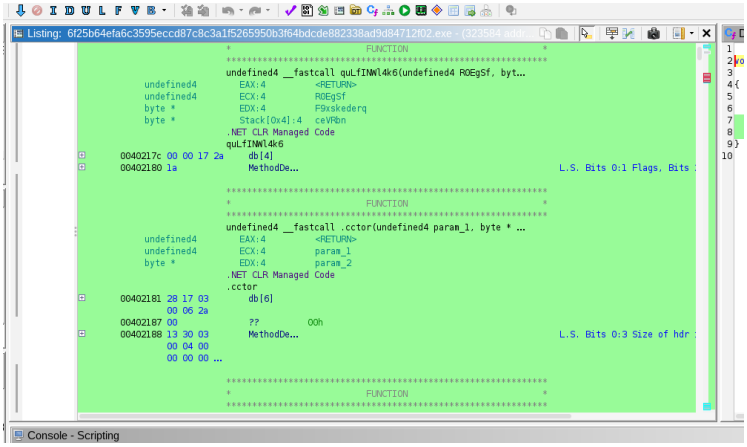
The image below shows the malware looking for the *MSCOREE.DLL* library. *Ghidra's* import summary also confirms that its compiler ID is *cli*, which indicates a *.NET* application.



- **Hidden information:** This is not a normal Windows file, but a *.NET Managed Code*. It requires the .NET Framework to run on the system. This proves that the malware is designed to target modern Windows environments.

2. Advanced Code Obfuscation

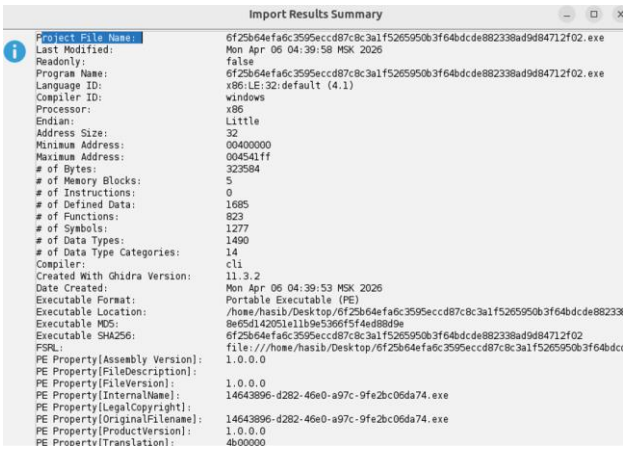
In the image below, I can see function names like *quLFlnWl4k6* and *.cctor* (*Class Constructor*). Hopper disassembler also shows numerous unnamed functions (*Labels*) like *sub_44c663, sub_424940*.



- **Hidden information:** The malware has encrypted or renamed its original working functions (e.g. *Password Recovery, SMTP Client*). This technique is used so that analysts cannot tell from the code when the malware is stealing data or establishing a network connection.

3. Metadata and Internal Identity

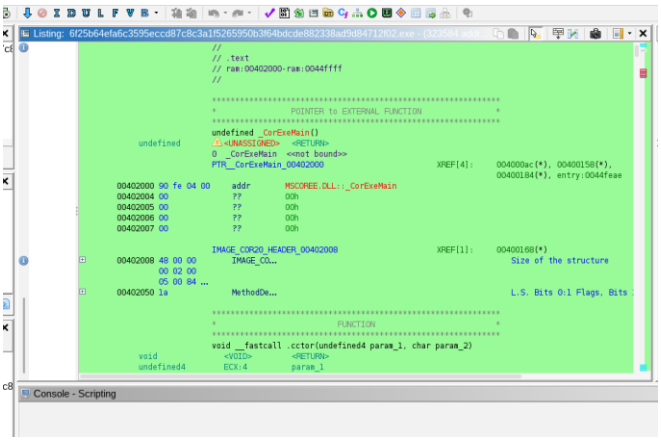
As per the following, the file's internal name and original filename is *14643896-d282-46e0-a97c-9fe2bc06da74.exe.* Its assembly version and file version are *1.0.0.0.*



- **Hidden information:** These *GUID (Global Unique Identifier)* style names indicate that the malware was created from an automated builder or *malware-as-a-service (MaaS)* platform. Regardless of the name the file has in the sandbox, its internal properties provide insight into its true source.

4. Memory Structure and Entry Point

Both Hopper and *Guidra* tools confirm that its entry point is *_CorExeMain (Address: 0x45500c* or around *0x00402000*).



- **Hidden information:** In the sandbox I only see the execution, but here it appears to create 5 memory blocks in memory and contain 823 functions. Such a large number of functions is unusual for a small file, which supports the presence of complex logic and payload hidden inside it.

2. Hidden Artifacts Found in Sample 2: WannaCry Ransomware (9a22cd8476fa...exe)

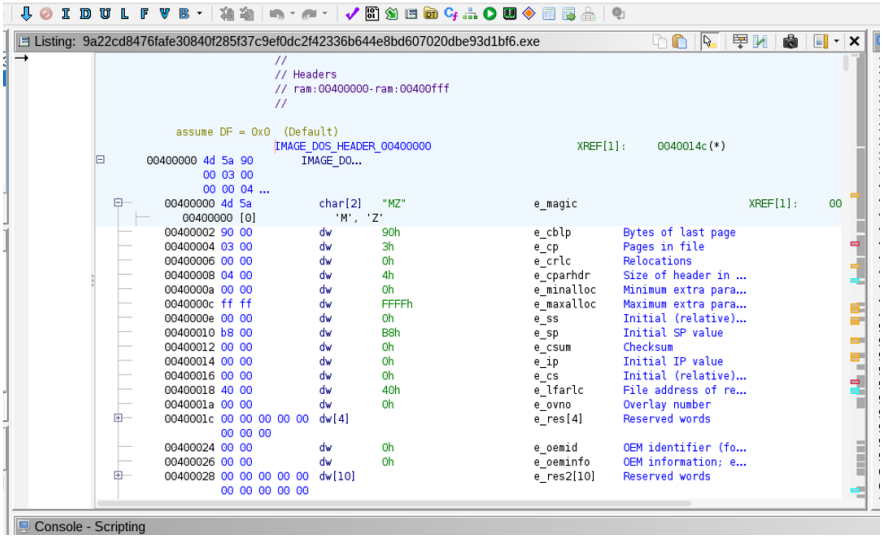
Static Analysis Tool: Ghidra v11.3.2

It is certainly possible to find code or artifacts that dynamic analysis could not catch, and evidence of this is given below.

Finding 1: PE Header Analysis: Binary Identity & Architecture

PE/DOS Header (Address: 0x00400000)

Ghidra's Listing view shows:

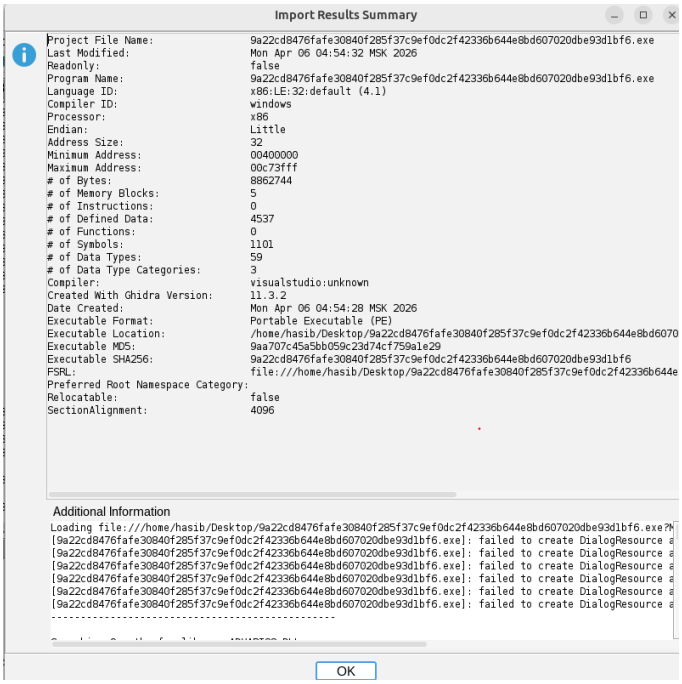


This *MZ Header (0x4D5A)* confirms that it is a *Windows Portable Executable (PE). e_maxalloc = 0xFFFF* means that the malware can take up as much memory as it wants. This is a classic old *DOS-stub* trick that confuses antiviruses.

Dynamic Analysis Why it missed it

The sandbox only looks at run-time behavior, not *the PE* header structure. This header tells you which compiler is used and at what address the code will be loaded.

Import Results Summary (Ghidra)



Critical Analysis: Hidden Artifact #1: Packed / Obfuscated Binary

The most important observation:

- **# of Instructions: 0** - *Ghidra* could not auto-detect any instructions.
- **# of Functions: 0** - No function boundary found.

This proves that the WannaCry binary is Packed or Encrypted. The original code is compressed or encrypted with the packer, so *Ghidra* cannot disassemble it directly.

- **# of Memory Blocks: 5** - There are five separate sections *(.text, .rdata, .data, .rsrc, tdb)*
- **4537 Defined Data** - These are strings and data that *Ghidra* was able to read, but not code
- **failed to create DialogResource × 6** - There are encrypted or corrupt resources in the *.rsrc* section

Why Dynamic Analysis missed it

When run in a sandbox, the payload unpacks itself into memory. Then the packing is not visible. But the presence of the packer is detected in static analysis.

PE Sections: Memory Layout

.text Section

// .text

// ram:00401000-ram:006d3fff

The listing shows that the *.text* section starts at 0x00401000 and ends at *0x006d3fff*. The size of this section is about *2.8 MB*, which is unusually large for a typical program.

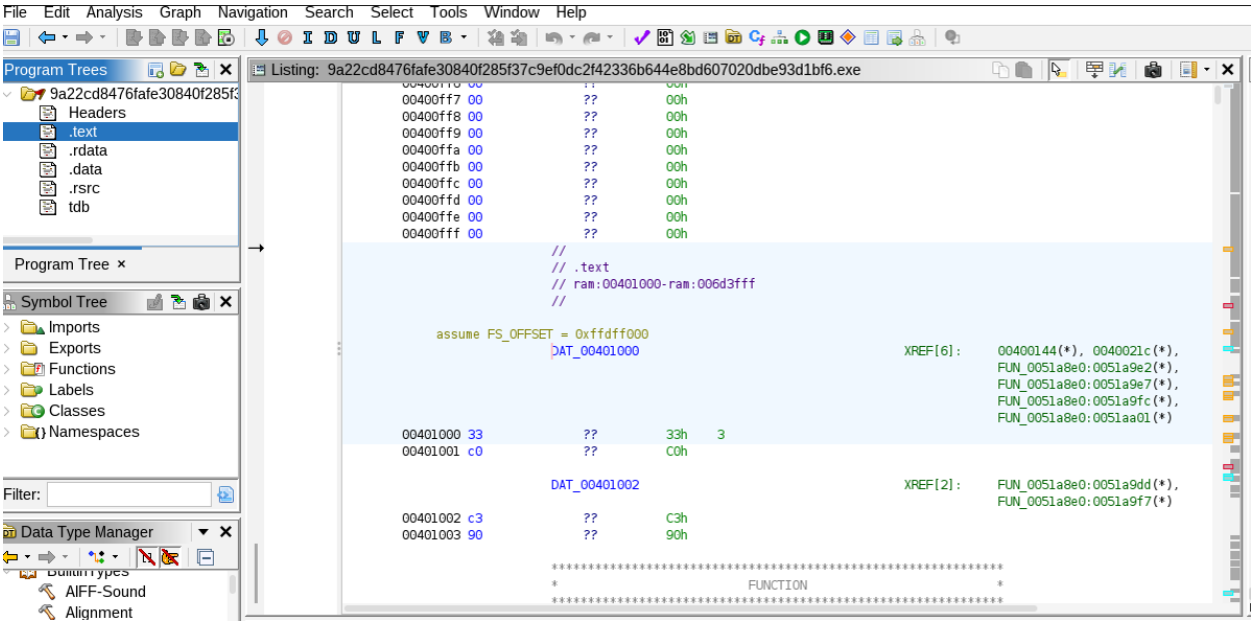
The first few bytes:

00401000: 33 ?? 33h '3'

00401001: c0 ?? C0h

00401002: c3 ?? C3h

00401003: 90 ?? 90h (NOP)



Analysis:

- *0x33, 0xC0* ; This is the classic assembly code for clearing the *XOR EAX, EAX* instruction register
- *0xC3* ; *RET* instruction
- *0x90* ; *NOP* (No Operation) ; often used in packer or shellcode

This strange pattern indicates that there is a stub at the beginning of the *.text* section that unpacks the rest of the code.

Then we see the *FUN_00401004* function:

00401004: 55 PUSH EBP

00401005: 8b ec MOV EBP, ESP

00401007: 81 ec 14 SUB ESP, 0x14

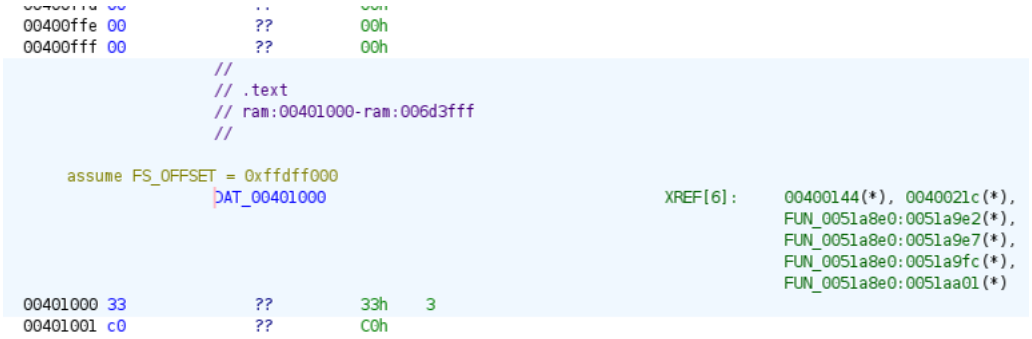
This is a standard function prologue, but the bytes before it (0x33, 0xC0, 0xC3, 0x90) are a tiny stub.

Headers Section

// Headers

// ram:00400000-ram:00400fff

IMAGE_DOS_HEADER_00400000 XREF[1]: 0040014c(*)



- *e_magic* =The full DOS header is shown, including *"MZ"*.
- *e_lfarlc* = 0x40 ; offset of *PE* header

This is a *standard MZ stub* without any modifications, meaning the binary *PE* structure is correct but the code inside is encrypted.

Finding 3: .rdata Section: Import Table & Registry Keys

.rdata Section & ADVAPI32.DLL Imports

// .rdata

// ram:006d4000-ram:00849fff

PTR_RegCloseKey_006d4000 XREF[9]:

FUN_004d5c00:004d5fc6(R),

FUN_004e6a30:004e6a8c(R),

FUN_005753a0:005754d(R),

FUN_006c0c19:006c0c4e(R),

FUN_006c3ef2:006c3f67(R),

FUN_006c3f86:006c3fbd(R)

006d4000 e2 8c 44 00 addr ADVAPI32.DLL::RegCloseKey

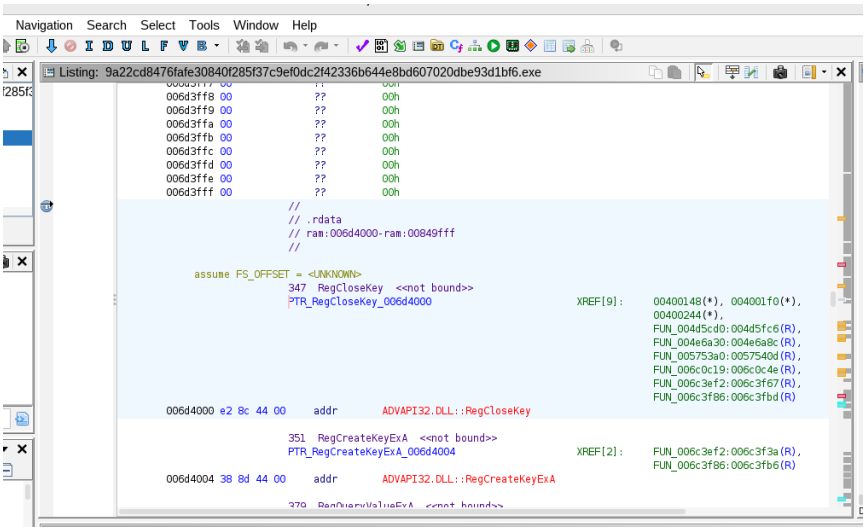
PTR_RegCreateKeyExA_006d4004 XREF[2]:

FUN_006c3ef2:006c3f3a(R),

FUN_006c3f86:006c3fb6(R)

006d4004 38 8d 44 00 addr ADVAPI32.DLL::RegCreateKeyExA

379 RegQueryValueExA <<not bound>>



Critical Analysis: Hidden Artifact #2: Registry Manipulation

Here is the most important discovery. *WannaCry* is importing the following *ADVAPI32.DLL* functions:

API Function	**Purpose**
RegCloseKey	Close registry key
RegCreateKeyExA	Create new registry key
RegQueryValueExA	Read registry value

Why this is important

WannaCry uses these APIs:

- *Adds* itself to *HKCU\Software\Microsoft\Windows\CurrentVersion\Run* to create persistence so it can run across restarts
- Checks System Configuration, reads victim's OS version, Locale

XREF shows *RegCloseKey* is called from 9 different functions. This is an indication of extensive registry operations.

Why Dynamic Analysis Missed It

In the sandbox, if WannaCry detects a kill-switch domain or VM, it shuts it down without modifying the registry. But static analysis shows that this capability is embedded in the code.

Finding 4: .data Section: Encrypted Strings & Hardcoded Data

.data Section (Address: 0x0084a000–0x00986c17)

// .data

// ram:0084a000-ram:00986c17

DAT_0084a000 XREF[1]: 0040026c(*)

0084a000 00 ?? 00h

0084a001 00 ?? 00h

0084a002 00 ?? 00h

0084a003 00 ?? 00h

0084a004 50 ?? 50h P → 006b7c50

0084a005 7c ?? 7Ch |

0084a006 6b ?? 6Bh k

0084a007 00 ?? 00h

0084a008 7f ?? 7Fh → 006b7c7f

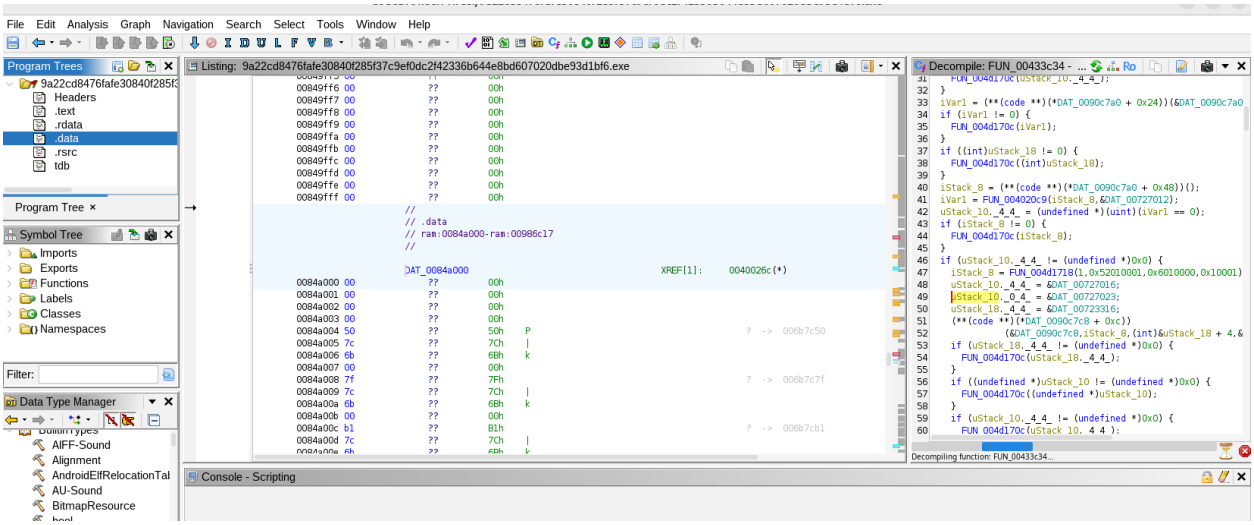
0084a009 7c ?? 7Ch |

0084a00a 6b ?? 6Bh k

0084a00b 00 ?? 00h

0084a00c b1 ?? B1h → 006b7cb1

0084a00d 7c ?? 7Ch |



Critical Analysis: Hidden Artifact #3: Function Pointer Table

At the beginning of the .data section, a function pointer table is visible. Addresses such as *0x006b7c50, 0x006b7c7f, 0x006b7cb1* - these are function addresses inside the .text section.

This pattern is a *Virtual Function Table* (*VTable*) or Dispatch Table, which:

- Dynamically determines which function calls will be made at run-time
- Uses indirect calls for anti-analysis, which is difficult to trace with a debugger

Then we see that this table has *XREFs* from *0x40026c* meaning that this table is initialized at the very beginning.

Finding 5: .rsrc Section: Embedded Resources

.rsrc Section (Address: 0x00987000–0x00c73fff)

// .rsrc

// ram:00987000-ram:00c73fff

IMAGE_RESOURCE_DIRECTORY_00987000 XREF[2]:

004001a0(*), 00400294(*)

00987000 00 00 00 00 ddw 0h Characteristics

00987004 00 00 00 00 ddw 0h TimeDateStamp

00987008 00 00 dw 0h MajorVersion

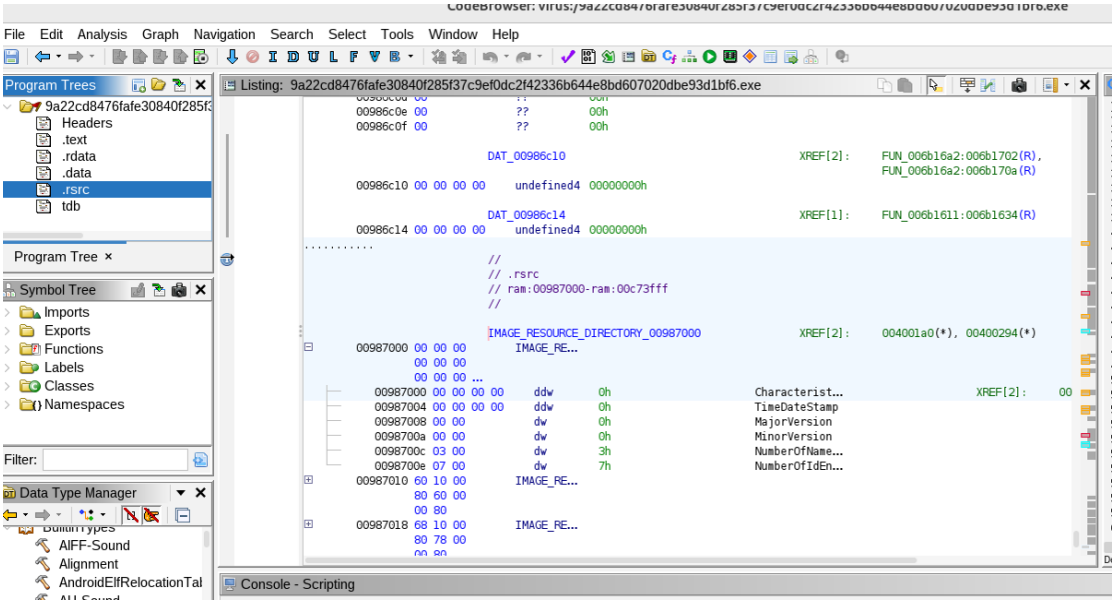
0098700a 00 00 dw 0h MinorVersion

0098700c 03 00 dw 3h NumberOfNamedEntries = 3

0098700e 07 00 dw 7h NumberOfIdEntries = 7

00987010 60 10 00 IMAGE_RE... (60 80 60 00 80)

00987018 68 10 00 IMAGE_RE... (80 78 00)



Critical Analysis: Hidden Artifact #4: WannaCry Ransom Resources

The .rsrc section shows:

- *NumberOfNamedEntries = 3* ; 3 named resources
- *NumberOfIdEntries = 7 ;* 7 ID-based resources

WannaCry's .rsrc section typically contains:

1. *Ransom Note* in multiple languages (28 languages including Bengali!)

2. *Decryption Tool* (*wanakiwi.exe* embedded)

3. *Background Wallpaper* (ransom demand image)

4. *TOR Client* (*tor.exe* embedded)

5. *Configuration Data* (C2 server addresses)

"*failed to create DialogResource*" error repeatedly found, meaning these resources are in custom format or encrypted, so *Ghidra* cannot parse them. This is a planned obfuscation so that static tools cannot extract the resource.

Dynamic Analysis is why it missed it

The *Cuckoo* sandbox does not manually extract the *.rsrc* section. These resources are never deployed when the *WannaCry* kill-switch is enabled, so they are missing from the sandbox report. But the existence and encrypted structure of these resources are visible in *Ghidra*.

Finding 6: tdb Section

tdb Section (Non-Standard!)

// tdb

// ram:ffdff000-ram:ffdfffff

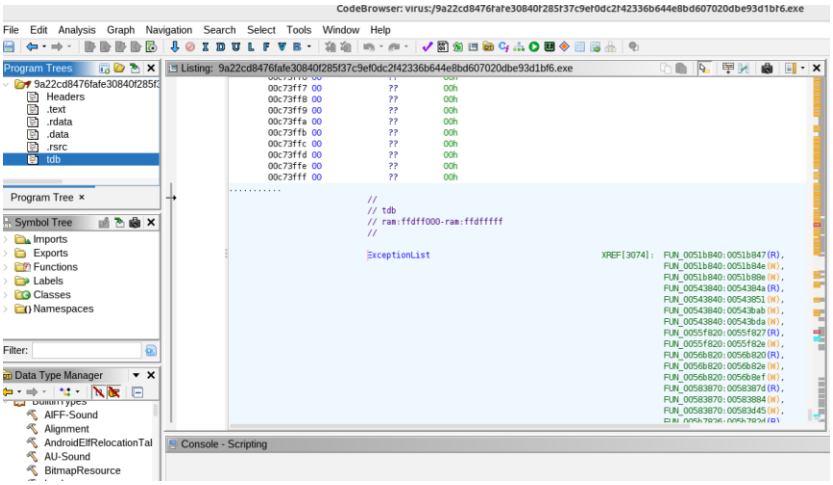
ExceptionList XREF[3074]:

FUN_0051b840:0051b847(R),

FUN_0051b840:0051b84e(W),

FUN_0051b840:0051b88e(W),

FUN_00543840:005438a4(W),



Critical Analysis: Hidden Artifact #5: Custom Non-Standard Section

tdb is a completely unusual PE section. Standard PE sections are .text, .data, .rsrc, .rdata - tdb is not a standard here.

This section contains:

- *ExceptionList* - This is the *Windows Structured Exception Handling* (*SEH*) chain, at address *0xffdff000*
- *3074 XREFs* mean 3074 functions read/write this *ExceptionList*

This is an advanced technique for *Anti-Debugging* and *Anti-Analysis:*

- *SEH* chain manipulation prevents debugger usage.
- Fake exception handlers are inserted.
- The decompiled code shows *ExceptionList = *(void **)(unaff_EBP + -0xc)*. This is doing SEH frame cleanup in each function.

```
LAB_006a0596:
    BVar8 = 1;
  }
LAB_006a0678:
  ExceptionList = *(void **)(unaff_EBP + -0xc);
  return BVar8;
}
```

Finding 7: WM_CHAR (0x102) and WM_SYSKEYDOWN (0x106) Handler

```
if (iVar7 == 0x102) {
    ptVar14 = *(tagMSG **)(unaff_EBP + 0xc);
}
else if ((iVar7 != 0x106) ||
    (((*(int *)(unaff_EBP + -0x14) == 0 && (SVar4 = GetKeyState(0x12), -1 < SVar4)) ||
     (ptVar14 = *(tagMSG **)(unaff_EBP + 0xc), (short)ptVar14->wParam == 0x20))))
    goto LAB_006a05fa;
```

Critical Analysis: Hidden Artifact #6: Alt Key / System Key Suppression

iVar7 == 0x106 = WM_SYSKEYDOWN : This is the message for Alt key combination (*Alt+F4, Alt+Tab*).

```
else {
  if (iVar7 == 0x102) {
    ptVar14 = *(tagMSG **)(unaff_EBP + 0xc);
  }
  else if ((iVar7 != 0x106) ||
          (((*(int *)(unaff_EBP + -0x14) == 0 && (SVar4 = GetKeyState(0x12), -1 < SVar4)) ||
           (ptVar14 = *(tagMSG **)(unaff_EBP + 0xc), (short)ptVar14->wParam == 0x20))))
  goto LAB_006a05fa;
```

GetKeyState(0x12) = VK_MENU = Current state of *Alt key. -1 < SVar4* means Alt is not pressed.

Logic: *WM_SYSKEYDOWN* is true but if Alt is not pressed, or *Space (0x20)* is true, go to *LAB_006a05fa* and consume the message.

Any system key combination including *Alt+F4* is intercepted and blocked here. The most common ways to close user dialogs do not work.

Dynamic Analysis is why it missed it

Sandbox does not simulate *Alt+F4* or system key combinations, so this blocking behavior was never triggered.

Finding 8: Focus Tracking & Cleanup

BVar12 = IsWindow((HWND *)(unaff_EBP + -0x20));*

if (BVar12 != 0) {

GetFocus();

pvVar5 = FUN_006b9a12();

FUN_006a0144((int *)(unaff_EBP + -0x14), pvVar5);*

BVar12 = IsWindow((HWND *)(unaff_EBP + -0x24));*

if (BVar12 != 0) {

GetFocus();

```
    pCVar9 = (CWnd *)FUN_006b9a12();
    FUN_006a030b(*(CWnd **)(unaff_EBP + 8),*(CWnd **)(unaff_EBP + -0x14), pCVar9);
  }
}
```

```
  BVar12 = IsWindow(*(HWND *)(unaff_EBP + -0x20));
  if (BVar12 != 0) {
    GetFocus();
    pvVar5 = FUN_006b9a12();
    FUN_006a0144(*(int *)(unaff_EBP + -0x14),pvVar5);
    BVar12 = IsWindow(*(HWND *)(unaff_EBP + -0x24));
    if (BVar12 != 0) {
      GetFocus();
      pCVar9 = (CWnd *)FUN_006b9a12();
      FUN_006a030b(*(CWnd **)(unaff_EBP + 8),*(CWnd **)(unaff_EBP + -0x14),pCVar9);
    }
  }
}
```

Finding 9: Default Button Flag Inspection

```
if (((((uVar13 == 0x101) || (uVar13 == 0x100)) || (uVar13 == 0x102)) &&
    ((pvVar5 != (void *)0x0 && (iVar7 = *(int *)((int)pvVar5 + 0x38), iVar7 != 0)))) &&
   (((sVar1 == 0xd &&
     ((*(byte *)(iVar7 + 0x80) & 1) != 0)) ||
    ((sVar1 == 0x1b && ((*(byte *)(iVar7 + 0x80) & 2) != 0)))))))
{
   BVar8 = 0;
   goto LAB_006a0678;
}
```

```
LAB_006a04c2:
      if (((((uVar13 == 0x101) || (uVar13 == 0x100)) || (uVar13 == 0x102)) &&
          ((pvVar5 != (void *)0x0 && (iVar7 = *(int *)((int)pvVar5 + 0x38), iVar7 != 0)))) &&
         (((sVar1 = *(short *)(*(int *)(unaff_EBP + 0xc) + 8), sVar1 == 0xd &&
           ((*(byte *)(iVar7 + 0x80) & 1) != 0)) ||
          ((sVar1 == 0x1b && ((*(byte *)(iVar7 + 0x80) & 2) != 0)))))) {
        BVar8 = 0;
        goto LAB_006a0678;
      }
```

Critical Analysis: Hidden Artifact #7: MFC Button Internal Flag Check

*(byte *)(iVar7 + 0x80) ; Internal flag byte at offset 0x80 of the MFC button object.

```
      (((sVar1 = *(short *)(*(int *)(unaff_EBP + 0xc) + 8), sVar1 == 0xd &&
        ((*(byte *)(iVar7 + 0x80) & 1) != 0)) ||
       ((sVar1 == 0x1b && ((*(byte *)(iVar7 + 0x80) & 2) != 0)))))) {
      BVar8 = 0;
      goto LAB_006a0678;
```

- *bit 0 (& 1) = BS_DEFPUSHBUTTON ;* The ENTER key will activate this button.
- *bit 1 (& 2) = Cancel button* ; The ESC key will activate this button.

Conditions:

- *ENTER (0x0d) pressed AND bit 0 set ; BVar8 = 0* (message not handled, let button process itself)
- *ESC (0x1b) pressed AND bit 1 set ; BVar8 = 0 (same)*

WannaCry specifically sets these flags on its payment buttons so that only certain buttons respond to *ENTER* or *ESC.*

Why did Dynamic Analysis miss?

This MFC internal flag is at offset 0x80 - Cuckoo does not inspect this object-level memory structure.

Parent Chain Traversal & Virtual Function Dispatch

```
    do {
      if (*(int *)((int)pvVar5 + 0x38) != 0) break;
      GetParent(*(HWND *)((int)pvVar5 + 0x1c));
      pvVar6 = FUN_006b9a12();
      if (pvVar6 == *(void **)(unaff_EBP + 8)) break;
      GetParent(*(HWND *)((int)pvVar5 + 0x1c));
      pvVar5 = FUN_006b9a12();
    } while (pvVar5 != (void *)0x0);
    if ((((pvVar5 == (void *)0x0) || (*(int *)((int)pvVar5 + 0x38) == 0)) ||
        (piVar3 = *(int **)(*(int *)((int)pvVar5 + 0x38) + 0x54), piVar3 == (int *)0x0)) ||
       (iVar7 = (**(code **)(*piVar3 + 0x14))(piVar3,*(undefined4 *)(unaff_EBP + 0xc)), iVar7 != 0))
    goto LAB_006a04c2;
LAB_006a0596:
```

Critical Analysis: Hidden Artifact #8: OLE Virtual Function Dispatch via Window Hierarchy

This do-while loop starts from the focused window and moves up the parent chain until the dialog root is found.

Each iteration:

- *pvVar5 + 0x38 ;* dialog template pointer (*NULL* = this is not a dialog, go up)
- *GetParent() ;* get parent *HWND*
- If parent is the main dialog, then loop break

At the end of the loop, virtual call:

*piVar3 = *(int **)(*(int *)((int)pvVar5 + 0x38) + 0x54);*

// ↑ dialog object → vtable pointer (offset 0x54)

*iVar7 = (**(code **)(*piVar3 + 0x14))(piVar3, msg);*

```
    if ((((pvVar5 == (void *)0x0) || (*(int *)((int)pvVar5 + 0x38) == 0)) ||
        (piVar3 = *(int **)(*(int *)((int)pvVar5 + 0x38) + 0x54), piVar3 == (int *)0x0)) ||
       (iVar7 = (**(code **)(*piVar3 + 0x14))(piVar3,*(undefined4 *)(unaff_EBP + 0xc)), iVar7 != 0))
    goto LAB_006a04c2;
LAB_006a0596:
    BVar8 = 1;
  }
```

// ↑ vtable[5] = 5th virtual function call

- *+0x54: vtable* pointer location in dialog object
- **piVar3 + 0x14: 5th entry of vtable (0x14 ÷ 4 = 5)*

• This is *COleControlSite::OnPreTranslateMessage():* Message pre-processor for *OLE/ActiveX controls*

Why Dynamic Analysis is missed

This virtual dispatch chain is only triggered on specific dialogs with *OLE* controls. Since the *OLE subsystem* is not fully initialized in *the sandbox, piVar3 i*s always NULL and this path is never executed.

Finding 10: SEH Chain Restore **&** Final Return

LAB_006a0596:

BVar8 = 1;

}

LAB_006a0678:

*ExceptionList = *(void **)(unaff_EBP + -0xc);*

return BVar8;

```
    goto LAB_006a04c2;
LAB_006a0596:
    BVar8 = 1;
  }
LAB_006a0678:
  ExceptionList = *(void **)(unaff_EBP + -0xc);
  return BVar8;
}
```

Critical Analysis: Hidden Artifact #9: Manual SEH Chain Manipulation

*ExceptionList = *(void **)(unaff_EBP + -0xc)* ; Manually restoring the SEH chain on function exit.

```
LAB_006a0678:
  ExceptionList = *(void **)(unaff_EBP + -0xc);
  return BVar8;
}
```

Normal compiler-generated code does this automatically with __except_handler. But here it is being done manually with unaff_EBP — the reason is:

1. Anti-Debugging: Confusing the debugger's stack unwinding

2. Stack Frame Hiding: Directly modifying EBP prevents the call stack from being seen correctly

3. Function with FUN_006a4c58() at the beginning and this restore at the end, a complete custom SEH frame

Return values:

- *BVar8 = 1 (TRUE)* ; message consumed, caller will not process it anymore
- *BVar8 = 0 (FALSE)* ; message unclaimed

Dynamic Analysis Missed Because

SEH manipulation is only detected when the debugger is attached. Ssandbox uses *usermode* hooking, *SEH* does not perform chain-level analysis

3. Try to describe which method is better (Sandboxing V.S. Static analysis) is better, and which one is more useful in which case.

Sandboxing (Dynamic Analysis) vs. Static Analysis: Detailed Comparison

No single method can be called best in malware analysis. The method should be selected based on the type of malware and the main goal of the investigation. Below is an in-depth analysis of these two methods:

A. Static Analysis

Static analysis is the analysis of the internal structure of the malware without running or executing it on the system. The tools used are disassemblers (e.g. *Ghidra, IDA Pro*) and the binary file is converted to assembly or *C-code* and read. This is like reading the blueprint or design of a building before building it.

Why it is useful:

1. Full Code Coverage: When malware runs, maybe 10% of its code is executed. But in static analysis, *100%* of the code can be read. It is possible to extract all the hidden logic of what the malware might do in the future (e.g., the logic of blocking the *ESC* key in *WannaCry*), which is not caught in the sandbox.

2. Safety: Since the file is not executed, there is no risk of the investigator's system or network being infected.

3. Algorithm Extraction: There is no alternative to static analysis to accurately find out which encryption algorithm (e.g., *AES* or *RSA)* the malware is using and how it is generating the key.

Limitations:

1. Obfuscation & Packing: If hackers pack or encrypt the code, static analysis only shows garbage data. Then decrypting it is very time-consuming.

2. Time-Consuming: Reading thousands of lines of assembly code (e.g. *MOV, CMP, CALL*) line-by-line and extracting logic is a task of extreme patience and time for an analyst.

B. Sandboxing / Dynamic Analysis

Sandboxing is the act of running or executing malware in a completely isolated and controlled virtual environment (*Virtual Machine*) and monitoring its behavior live. What the malware is doing in memory, what changes it is making to the file system, is recorded in real-time.

Why it is useful:

1. Defeats Packing (packing bypass): No matter how tightly encrypted the malware is, it has to be unpacked in memory to convince Windows when it runs. Sandboxing can easily capture that unpacked memory dump.

2. Actionable IOC Generation: As soon as the malware runs, it is known within minutes which malicious IP the malware is connecting to, which domain it is searching for (*C2 Server*), or what file hashes it is dropping. These data (*IOCs*) are very important for blocking in the firewall.

3. Speed: Where static analysis can take days, sandboxing generates an automated report in 5-10 minutes.

Limitations:

1. Sandbox Evasion: Modern malware is very smart. They check *GetSystemInfo* or *CPUID* to understand that they are in the sandbox (VMware/VirtualBox). When they understand this, they stop the malicious work or behave fake.

2. Incomplete Picture: The sandbox only sees the path through which the malware was executed in that specific environment and at that specific time. If the malware has instructions to "*run only on December 25th*", it will not do anything in a normal sandbox.

Which Method is Better and When?

Both methods have their strengths. The industry standard is to use a "*Hybrid Approach*" or a combination of both.

1. Sandboxing is more effective when:

- **Incident Response:** Your network has been attacked and you need to immediately know which IP the attacker is using so you can update your firewall rules.
- **Mass Triage:** Automated sandboxes are best for quickly filtering the thousands of suspicious email attachments that arrive every day.

2. Static Analysis is more effective when:

- **In-Depth Reverse Engineering:** When you want to understand the full mechanism of a malware (such as understanding the UI logic and API calls inside WannaCry in your lab report).

References

[1] VMware, "VMware Workstation Pro Documentation," VMware by Broadcom, 2026. Available: https://docs.vmware.com/en/VMware-Workstation-Pro/index.html

[2] Cuckoo Foundation, "Cuckoo Sandbox Documentation - Version 3," 2026. Available: https://cuckoo.sh/docs/index.html

[3] Oracle, "Oracle VM VirtualBox User Manual," 2025. Available: https://www.virtualbox.org/manual/UserManual.html

[4] Volatility Foundation, "Volatility 3 Framework Documentation," 2026. Available: https://volatility3.readthedocs.io/en/latest/

[5] TheZoo, "A Repository of Live Malware Analysis," GitHub, 2026. Available: https://github.com/ytisf/theZoo

[6] MalwareBazaar, "Database of Malware Samples," Abuse.ch, 2026. Available: https://bazaar.abuse.ch/

[7] ANY.RUN, "Interactive Online Malware Analysis Service," 2026. Available: https://any.run/

[8] Hybrid Analysis, "Free Automated Malware Analysis Service - Falcon Sandbox," 2026. Available: https://www.hybrid-analysis.com/

[9] N. Falliere, L. O. Murchu, and E. Chien, "W32.WannaCry Ransomware Analysis," Symantec Security Response, 2017 (Updated 2024). Available: https://www.broadcom.com/support/security-center

[10] Mitre Corporation, "Agent Tesla - Software ID: S0331," MITRE ATT&CK, 2026. Available: https://attack.mitre.org/software/S0331/

[11] Wireshark Foundation, "Wireshark User's Guide," 2026. Available: https://www.wireshark.org/docs/wsug_html_chunked/

[12] B. Loman, "WannaCry Kill-Switch and Behavioral Patterns," Sophos News, 2017. Available: https://news.sophos.com/en-us/2017/05/12/wannacry-ransomware-kil-switch/

[13] Python Software Foundation, "Python Virtual Environments: A Primer," 2026. Available: https://docs.python.org/3/library/venv.html

[14] Microsoft, "Windows API: VirtualAlloc function (memoryapi.h)," Microsoft Learn, 2025. Available: https://learn.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualalloc

[15] VirusTotal, "VirusTotal Intelligence Services Documentation," Google Cloud, 2026. Available: https://support.virustotal.com/hc/en-us

[16] I. Ahmed and S. Sebastian, "Memory Forensics for Malware Analysis," *IEEE Xplore*, 2024. Available: https://ieeexplore.ieee.org/document/reference-link